# Politechnika Wrocławska

Wydział Elektroniki, Fotoniki i Mikrosystemów


KIERUNEK: Automatyka i Robotyka (AIR)

# Bazy danych
# PROJEKT

TYTUŁ PROJEKTU:
Sklep hydrauliczny


AUTORZY:
Bartosz Ksel
Szymon Omieljaniuk


PROWADZĄCY:
Dr inż. Paweł Rogaliński


WROCŁAW 2025

# Spis treści

**Specyfikacja projektu** **5**

**1 Opis świata rzeczywistego** **7**

**2 Cel projektu oraz wymagania systemu** **9**
2.1 Cel projektu . . . 9
2.2 Wymagania funkcjonalne . . . 9
2.2.1 Konta użytkowników . . . 9
2.2.2 Identyfikacja i uwierzytelnianie użytkowników . . . 10
2.2.3 Funkcje dostępne dla pracownika . . . 10
2.2.4 Funkcje dostępne dla kierownika . . . 10
2.2.5 Funkcje dostępne dla klienta . . . 11
2.2.6 Funkcje dostępne dla zalogowanego użytkownika . . . 11
2.2.7 Funkcje dostępne dla niezalogowanego użytkownika . . . 11
2.2.8 Funkcje wymagane do obsługi procesu zamówienia . . . 11
2.3 Wymagania niefunkcjonalne . . . 12
2.4 Funkcjonalności implementowane i testowane na zaliczenie . . . 12
2.4.1 Wersja minimalna . . . 12
2.4.2 Wersja standardowa . . . 12
2.4.3 Wersja rozszerzona . . . 12

**3 Diagramy przypadków użycia** **13**
3.1 Identyfikacja aktorów . . . 13
3.2 Przykładowe diagramy przypadków użycia dla poszczególnych modułów . . . 15

**4 Opis przypadków użycia** **19**
4.1 Diagramy przypadków użycia . . . 19
4.2 Przypadki użycia dla gościa . . . 22
4.2.1 PU „Zaloguj się” . . . 23
4.2.2 PU „Przeglądaj asortyment sklepu” . . . 23
4.3 Przypadki użycia dla użytkownika . . . 24
4.3.1 PU „Wyloguj się” . . . 25
4.3.2 PU „Dodaj produkt do koszyka” . . . 28
4.3.3 PU „Zobacz koszyk” . . . 29
4.3.4 PU „Zobacz stan swoich zamówień” . . . 30
4.4 Przypadki użycia dla pracownika . . . 33
4.4.1 PU „Wyszukaj użytkownika” . . . 33
4.4.2 PU „Zarządzaj zamówieniami” . . . 36
4.4.3 PU „Edytuj stan magazynowy” . . . 39

4.5 Przypadki użycia dla kierownika . . . 44
4.5.1 PU „Nadaj/odbierz rolę pracownika” . . . 44
4.5.2 PU „Nadaj/odbierz rolę kierownika” . . . 44
4.5.3 PU „Generuj statystyki dla sklepu w określonym czasie” . . . 45
4.5.4 PU „Generuj rankingi” . . . 47
4.6 Grafiki . . . 49

**5 Modele bazy danych** **51**
5.1 Wersja pierwsza . . . 52
5.2 Wersja druga . . . 54
5.3 Wersja trzecia . . . 56
5.4 Wersja czwarta . . . 57
5.5 Wersja piąta . . . 59
5.6 Wersja szósta . . . 61
5.7 Graf stanów zamówienia . . . 63

**6 Opis implementacji** **67**
6.1 Opis implementacji . . . 67
6.1.1 Technologie . . . 67
6.1.2 Struktura systemu . . . 67
6.1.3 Moduły funkcjonalne . . . 67
6.1.4 Model danych . . . 68
6.1.5 Bezpieczeństwo . . . 68

**7 Główne moduły i klasy** **69**
7.1 Główne moduły i klasy . . . 69
7.1.1 Pakiet `com.kselomiel.app` . . . 69
7.1.2 Pakiet `com.kselomiel.app.base` . . . 69
7.1.3 Pakiet `com.kselomiel.app.model` . . . 69
7.1.4 Pakiet `com.kselomiel.app.repository` . . . 70
7.1.5 Pakiet `com.kselomiel.app.security` . . . 70
7.1.6 Pakiet `com.kselomiel.app.service` . . . 70
7.1.7 Pakiet `com.kselomiel.app.sklep` . . . 70
7.1.8 Widoki użytkownika (`com.kselomiel.app.sklep.ui.view`) . . . 70
7.1.9 Formularze i dialogi . . . 71
7.1.10 Testy . . . 71
7.1.11 Pozostałe pliki . . . 71

**8 Opis wybranego fragmentu kodu źródłowego** **73**
8.0.1 Konstruktor klasy . . . 74
8.0.2 Metoda tworzenia zamówień . . . 75
8.1 Metody akceptacji i odrzucenia zamówienia . . . 76
8.1.1 Kompletowanie zamówienia . . . 78
8.1.2 Odbiór zamówienia, oznaczanie gotowości opłaty oraz opłata zamówienia . . . 79
8.2 Zwrot zamówienia . . . 81
8.3 Metody pomocnicze . . . 83

**9 Podręcznik dla użytkownika** **85**
9.1 Uruchomienie aplikacji . . . 85

9.1.1 Budowanie aplikacji (Tryb produkcyjny) . . . . . 86
9.1.2 Budowanie obrazu Docker . . . . . 86
9.1.3 Uruchamianie bazy danych . . . . . 86
9.1.4 Dostęp do aplikacji . . . . . 86
9.2 Przewodnik po rolach użytkowników . . . . . 87
9.2.1 Gość . . . . . 87
9.2.2 Klient . . . . . 89
9.2.3 Pracownik . . . . . 91
9.2.4 Kierownik . . . . . 95
**10 Wnioski** **97**
**Literatura** **99**
**Spis rysunków** **99**

# Specyfikacja projektu

Celem projektu było stworzenie aplikacji webowej z wykorzystaniem frameworka Vaadin, umożliwiającej zarządzanie danymi przechowywanymi w relacyjnej bazie danych MySQL. Aplikacja udostępnia interfejs dostępny przez przeglądarkę internetową i realizuje operacje związane z obsługą użytkowników, zamówień oraz produktów.

## Wykorzystane technologie i narzędzia

- **Język programowania:** Java
- **Framework:** Vaadin
- **Baza danych:** MySQL
- **System kontroli wersji:** Git
- **Repozytorium kodu:** GitHub
- **Środowisko uruchomieniowe:** Docker
- **System budowania:** Maven
- **Środowisko programistyczne (IDE):** Visual Studio Code

## Główne funkcjonalności aplikacji

- Możliwość logowania użytkowników
- Zarządzanie kontami użytkowników
- Składanie zamówień przez użytkowników
- Zarządzanie zamówieniami
- Dodawanie i edycja produktów

# Uwagi dodatkowe

Projekt był uruchamiany i testowany w środowisku kontenerowym Docker, co umożliwiało szybkie wdrażanie aplikacji wraz z bazą danych w środowisku testowym i produkcyjnym.

# Rozdział 1

# Opis świata rzeczywistego

Projekt ma dotyczyć opracowania i implementacji systemu wspomagającego wydawanie towaru w hurtowni hydraulicznej. Klientami są instalatorzy, którzy korzystają z mechanizmu wydania zewnętrznego towaru. Biorą oni materiał, nie płacąc za niego, często więcej niż im będzie potrzebne. To co nie zostało zamontowane jest przywożone do hurtowni i odpisywane z zamówienia. Po takim odpisaniu można wystawić fakturę, za którą ktoś może zapłacić (niekoniecznie instalator, może być to nawet klient instalatora).

Jednak nie każdy instalator może korzystać z tej opcji. Musi on być stałym, zaufanym klientem z historią rzetelnych opłat za towary. Wtedy pracownicy mogą zaproponować możliwość korzystania z tego mechanizmu. Wówczas nie będzie występować obawa o to, że może dojść do kradzieży. Jeśli jednak by do tego doszło to instalator straciłby swoją renomę nie tylko w jednej hurtowni, ale także w okolicznych. To uniemożliwiłoby korzystanie z tego wygodnego układu, który powoduje wzrost produktywności, a dla sklepu wzrost zysków. Jednak tylko gdy obie strony są uczciwe, niestety zawsze istnieje pewne ryzyko, lecz jest ono przewidziane w kosztach działalności.

Pracownik sklepu zajmuje się akceptacją, przygotowaniem oraz aktualizacją statusu zamówienia złożonego przez klienta. Po otrzymaniu zamówienia oraz jego weryfikacji pracownik przygotowuje zamówienie i uzupełnia dostępność poszczególnych materiałów w spisie inwentarza. W przypadku przekroczenia pewnego dolnego progu dostępności danego produktu pracownik może złożyć zamówienie w celu uzupełnienia asortymentu. Do uprawnień pracownika należy również modyfikacja cen produktów oraz, w uzasadnionych przypadkach, przyznanie klientowi rabatu.

Prezes firmy czasami pracuje obok swoich pracowników, jednak ma dodatkowe uprawnienia. Przede wszystkim może zatrudniać i zwalniać pracowników. Dodatkowo ma dostęp do statystyk przedsiębiorstwa:

- ilość transakcji,
- koszty, obrót, dochód,
- zestawienie klientów z najwyższymi sumarycznymi kosztami zamówień,

Każdy klient jak i pracownik posiada następujące dane:

- imię i nazwisko,
- adres email,
- numer identyfikacyjny.

Każdy pracownik jak i kierownik mają informacje o dniu, w którym zostali zatrudnieni na swoim stanowisku.

Zamówienie można opisać następującymi stanami:

- złożone, ale niezaakcpetowane.
- w przygotowaniu,
- gotowe do odbioru,
- odebrane, oczekujące na ewentualny zwrot,
- zwrócone, w trakcie przetwarzania,
- gotowe do zapłaty,
- opłacone.

Obecnie asortyment sklepu jest widoczny w formie tabelki na stronie internetowej. Każdy produkt, którym handluje sklep ma swoje atrybuty:

- kod producenta,
- cena,
- nazwa,
- zdjęcie,
- jednostkę sprzedaży (sztuka, metr bieżący, itp.),
- minimalny próg ilościowy po przekroczeniu, którego należy natychmiast złożyć zamówienie do uzupełnienia go.

# Rozdział 2

# Cel projektu oraz wymagania systemu

## 2.1 Cel projektu

Celem projektu jest budowa systemu bazodanowego, który umożliwi sprawne funkcjonowanie sklepu hydraulicznego. System ma wspomagać pracowników w zarządzaniu zamówieniami oraz kontrolowaniu asortymentu sklepu. Klienci muszą mieć dostęp do składania, edycji (zależnie od statusu zamówienia) oraz anulowania zamówień. Ponadto powinni też widzieć stan magazynowy asortymentu. Kierownik ma dostęp do okresowych raportów finansowych sklepu.

System powinien być zaimplementowany jako strona internetowa dostępna zarówno dla pracowników oraz klientów. Posiadanie konta nie jest wymagane aby przeglądać asortyment sklepu.

*Uwaga: System nie umożliwia wystawiania faktur i wydań zewnętrznych. System nie prowadzi ewidencji wynagrodzeń pracowników.*

## 2.2 Wymagania funkcjonalne

### 2.2.1 Konta użytkowników

- Użytkownikami systemu są pracownicy i klienci hurtowni.
- Dostęp do systemu mają użytkownicy niezalogowani oraz użytkownicy zalogowani posiadający indywidualne konta imienne.
- Konta użytkowników mogą zostać utworzone jedynie przez kadrę przedsiębiorstwa, która wprowadza podczas tego procesu potrzebne dane (imię, nazwisko, adres email). Domyślnie każdy nowy użytkownik jest klientem.
- W systemie przewidziano następujące role dla użytkowników zalogowanych: klient, pracownik, kierownik.
- W systemie musi być zarejestrowany co najmniej jeden użytkownik z rolą kierownika. Kierownik może nadawać rolę kierownika innym użytkownikom, dodawać nowych pracowników oraz klientów.

- Pracownicy mogą dodawać nowych użytkowników z rolą klienta.

### 2.2.2 Identyfikacja i uwierzytelnianie użytkowników

- Użytkownicy będą identyfikowani na podstawie adresu email. Każdy zarejestrowany użytkownik musi mieć unikalny adres.
- Zastosowane zostanie uwierzytelnianie dwuskładnikowe - przy użyciu własnego hasła oraz generowanego kodu wysłanego innym kanałem komunikacyjnym (np. na adres email). *Uwaga: wysłanie wiadomości email będzie symulowane przez wyświetlenie odpowiedniego komunikatu z kodem podczas logowania.*
- Hasło musi zawierać co najmniej 6 znaków, przy czym musi występować w nim co najmniej jedna: mała litera, wielka litera oraz cyfra.
- Kod dostępu będzie czteroelementowym ciągiem cyfr generowanym losowo dla każdej instancji logowania.

### 2.2.3 Funkcje dostępne dla pracownika

Zalogowany użytkownik posiadający rolę pracownika ma dostępne funkcjonalności:

- Tworzenie oraz deaktywowanie kont klienta.
- Sprawdzanie oraz edycja stanu magazynowego konkretnych produktów.
- Dodawanie nowych pozycji w asortymencie wraz z nadaniem ceny oraz ilości magazynowej (konieczne podanie kodu fabrycznego produktu).
- Wycofywanie istniejących pozycji z asortymentu sklepu.

### 2.2.4 Funkcje dostępne dla kierownika

Zalogowany użytkownik posiadający rolę kierownika ma dostępne wszystkie funkcjonalności roli pracownik oraz:

- Pierwsze konto kierownika jest zakładane przy instalacji systemu.
- Nadawanie oraz odbieranie pracownikom roli kierownika.
- Blokowanie konta pracownika lub klienta.
- Usuwanie blokady innego użytkownika.
- Generowanie statystyk dla sklepu w określonym czasie:
  - całkowity obrót,
  - całkowity zysk,

  - ilość otwartych zamówień,
  - ilość zamkniętych zamówień.

- Generowanie rankingów:
  - zestawienie klientów, których suma kosztów zamówień jest największa,
  - zestawienie najczęściej kupowanych produktów.

### 2.2.5 Funkcje dostępne dla klienta

Zalogowany użytkownik posiadający rolę klienta ma dostępne funkcjonalności:

- Złożenie, anulowanie oraz edycja zamówienia (klient może edytować zamówienia które nie zostało już przygotowane i nie jest jeszcze gotowe do odbioru).
- Podgląd statusu zamówienia.
- Złożenie zawiadomienia o wykonaniu zwrotu towaru, który będzie wtedy odpisany z zamówienia, wraz z podglądem statusu zwrotu (oczekiwanie na przywiezienie towaru do hurtowni, w trakcie zwrotu, gotowy do opłaty).

### 2.2.6 Funkcje dostępne dla zalogowanego użytkownika

- Przegląd danych osobowych, kontaktowych oraz danych przedsiębiorstwa.
- Zmiana hasła.
- Wylogowanie.

### 2.2.7 Funkcje dostępne dla niezalogowanego użytkownika

- Przegląd asortymentu sklepu (stan magazynowy, cena).
- Przegląd danych kontaktowych oraz adresu sklepu.
- Przeczytanie regulaminu hurtowni.
- Logowanie.

### 2.2.8 Funkcje wymagane do obsługi procesu zamówienia

1. Składanie zamówienia przez klienta przez wybór produktów na stronie internetowej.
   - (opcjonalnie) dopóki zamówienie nie zostanie zatwierdzone to może je edytować lub anulować.
2. Zatwierdzenie zamówienia przez pracownika.

3. Kompletowanie zamówienia przez pracownika. Jeżeli w asortymencie brakuje jakiegoś produktu to do zamówienia dodawane jest tyle ile jest na stanie. Po skompletowaniu zostaje oznaczone jako gotowe do odbioru.

4. Odbiór zamówienia przez klienta.

    - (opcjonalnie) klient może poprosić przy odbiorze o wystawienie rachunku od razu. Pomijany jest wtedy proces zwrotu.

5. Zwrot towaru, przez klienta, który zostaje odpisany od zamówienia.

6. Zamknięcie zamówienia przez pracownika, klientowi wystawiany jest rachunek.

    - (opcjonalnie) pracownik może przy wystawianiu rachunku dać klientowi rabat określony procentowo.

## 2.3 Wymagania niefunkcjonalne

- Baza danych typu SQL z bezpłatną licencją do użytku komercyjnego.
- Terminale wszystkich typów użytkowników: aplikacja webowa przystosowana do pracy w trybie desktopowym i mobilnym.
- Przeglądarka Chromium w wersji co najmniej 132, na komputerze z minimum 4 GiB RAM.
- Interfejs użytkownika typu GUI.

## 2.4 Funkcjonalności implementowane i testowane na zaliczenie

### 2.4.1 Wersja minimalna

Stworzenie bazy danych umożliwiającej zarządzanie kontami użytkowników (możliwość logowania, zmiany hasła, wylogowanie, blokowanie konta).

### 2.4.2 Wersja standardowa

Dodawanie, edycja oraz usuwanie produktów z asortymentu sklepu. Tworzenie oraz zarządzanie zamówieniami przez klienta.

### 2.4.3 Wersja rozszerzona

Dostosowanie strony internetowej do urządzeń mobilnych. Tworzenie raportów finansowych dla kierownictwa.

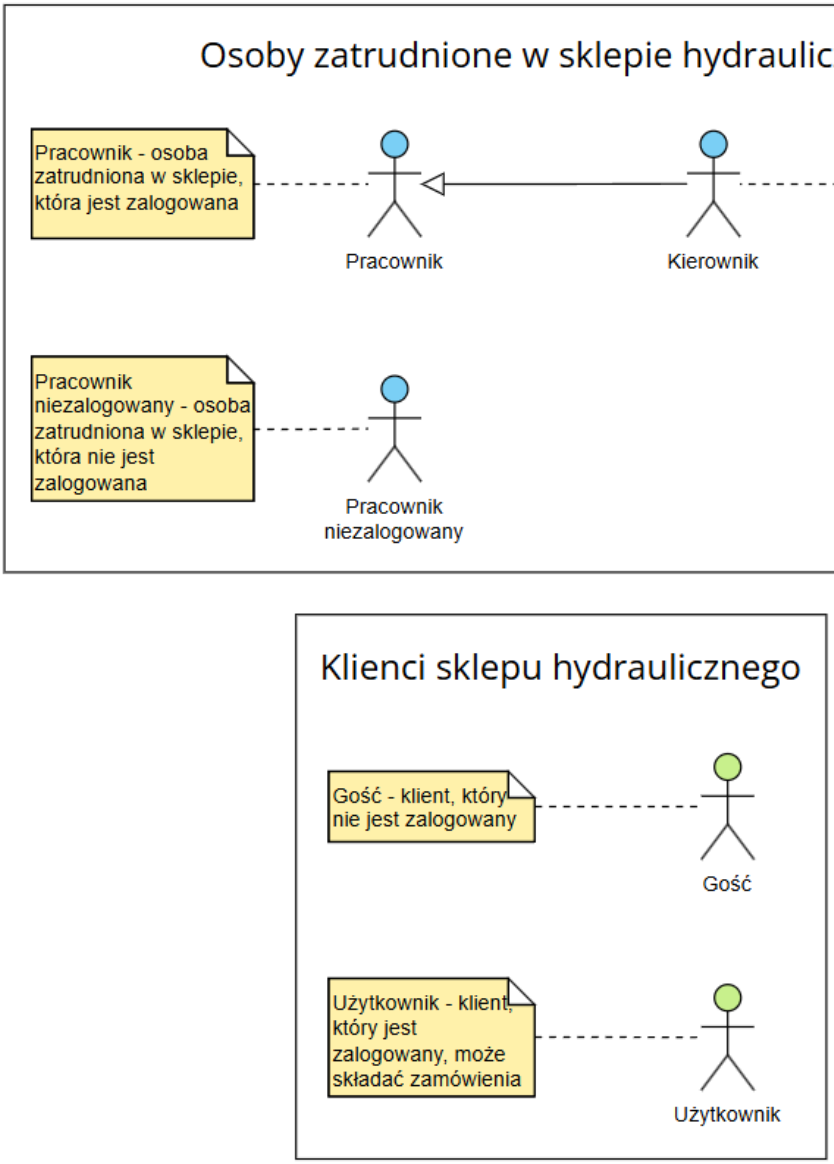
# Rozdział 3

# Diagramy przypadków użycia

## 3.1 Identyfikacja aktorów

- Gość - klient, który nie jest zalogowany,
- Użytkownik - zalogowany klient, który może składać zamówienia
- Pracownik niezalogowany - pracownik, który nie jest zalogowany
- Pracownik - osoba zatrudniona w sklepie, która jest zalogowana
- Kierownik - pracownik pełniący rolę kierownika

Wszyscy aktorzy zostali przedstawieni na poniższym diagramie:




Rysunek 3.1 Aktorzy w systemie zarządzania sklepem hydraulicznym.

## 3.2 Przykładowe diagramy przypadków użycia dla poszczególnych modułów

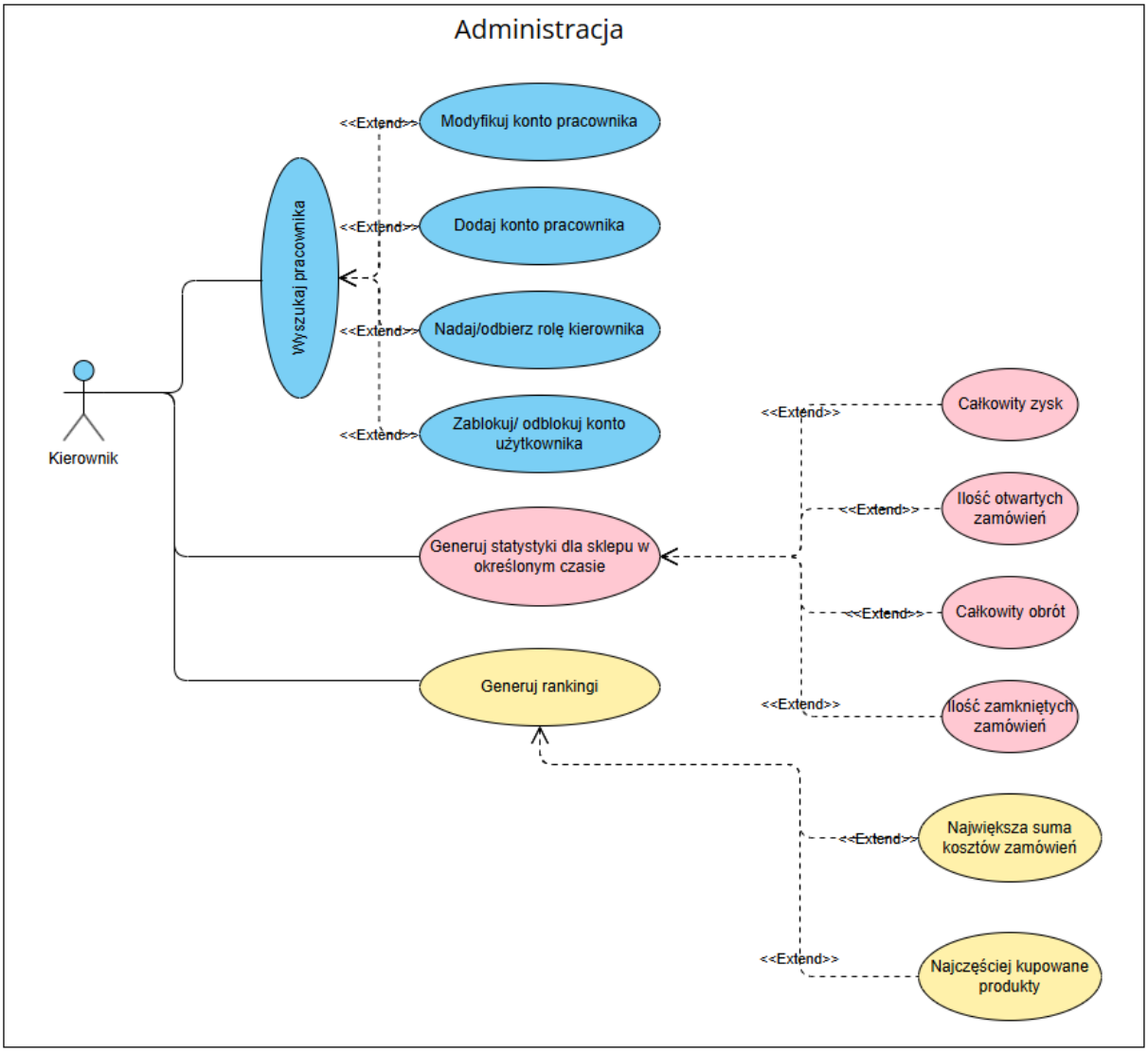
Dla poprawy czytelności system został podzielony na następujące moduły:

⇒ Administracja

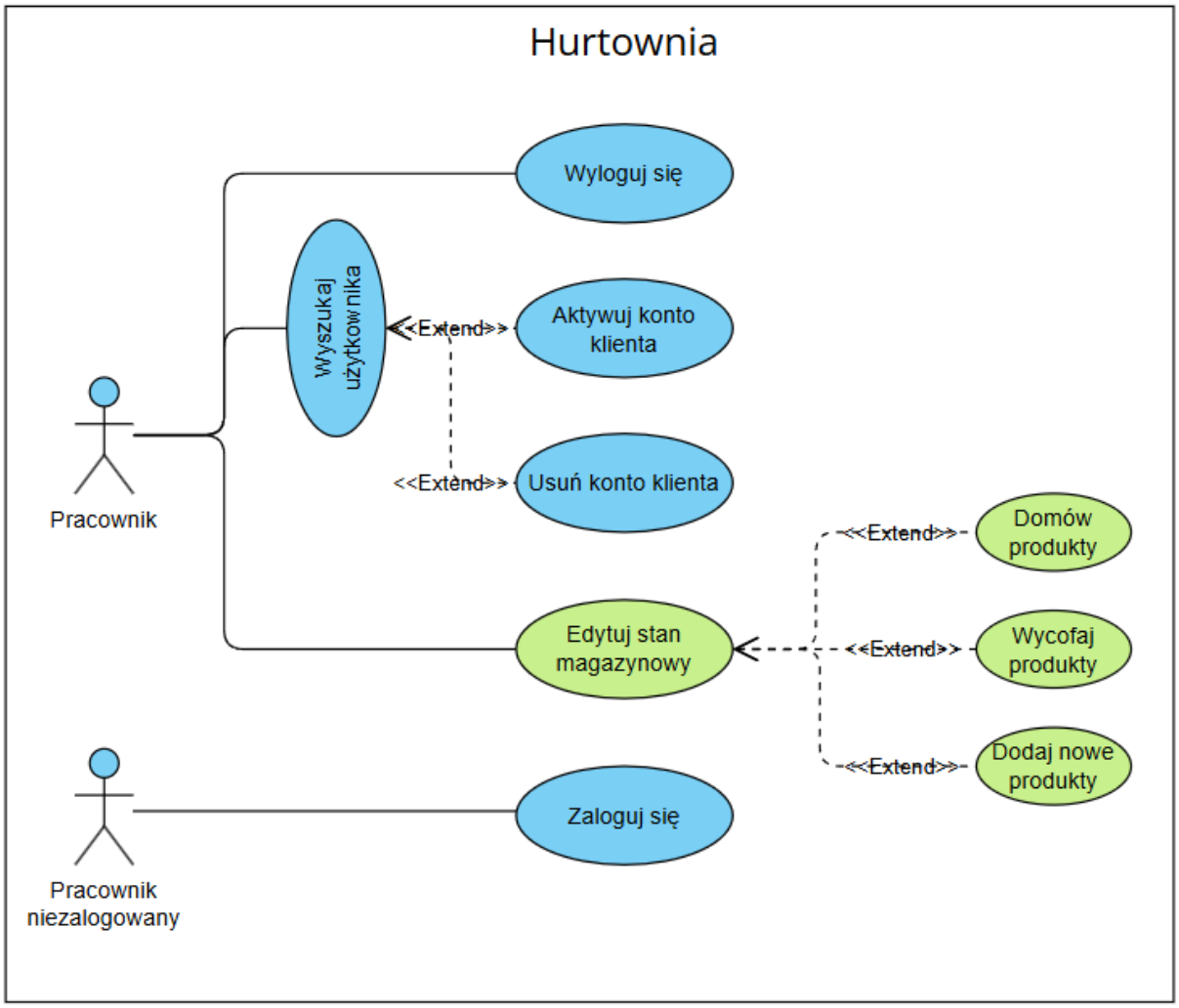
⇒ Hurtownia

⇒ Sklep internetowy

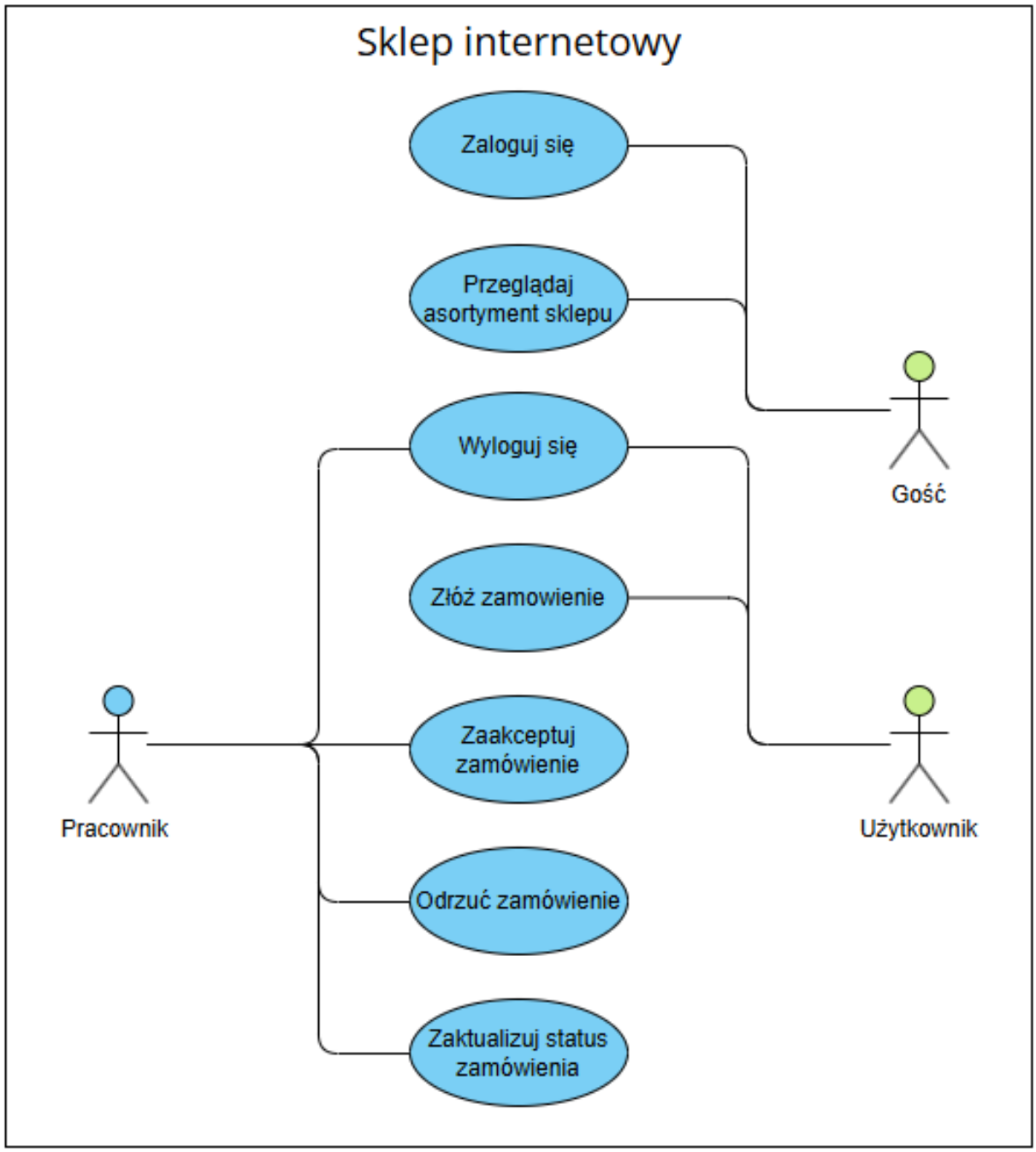
Wszystkie diagramy przypadków użycia zostały przedstawione na poniższym diagramie:




Rysunek 3.2 Diagram przypadków użycia dla modułu *Administracja*.




Rysunek 3.3 Diagram przypadków użycia dla modułu *Hurtownia*.

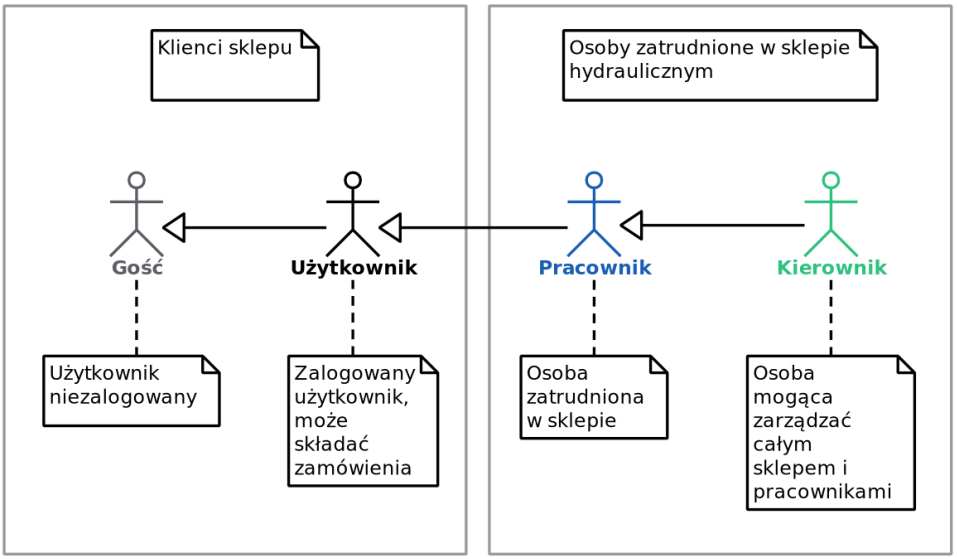



Rysunek 3.4 Diagram przypadków użycia dla modułu *Sklep internetowy*.

# Rozdział 4

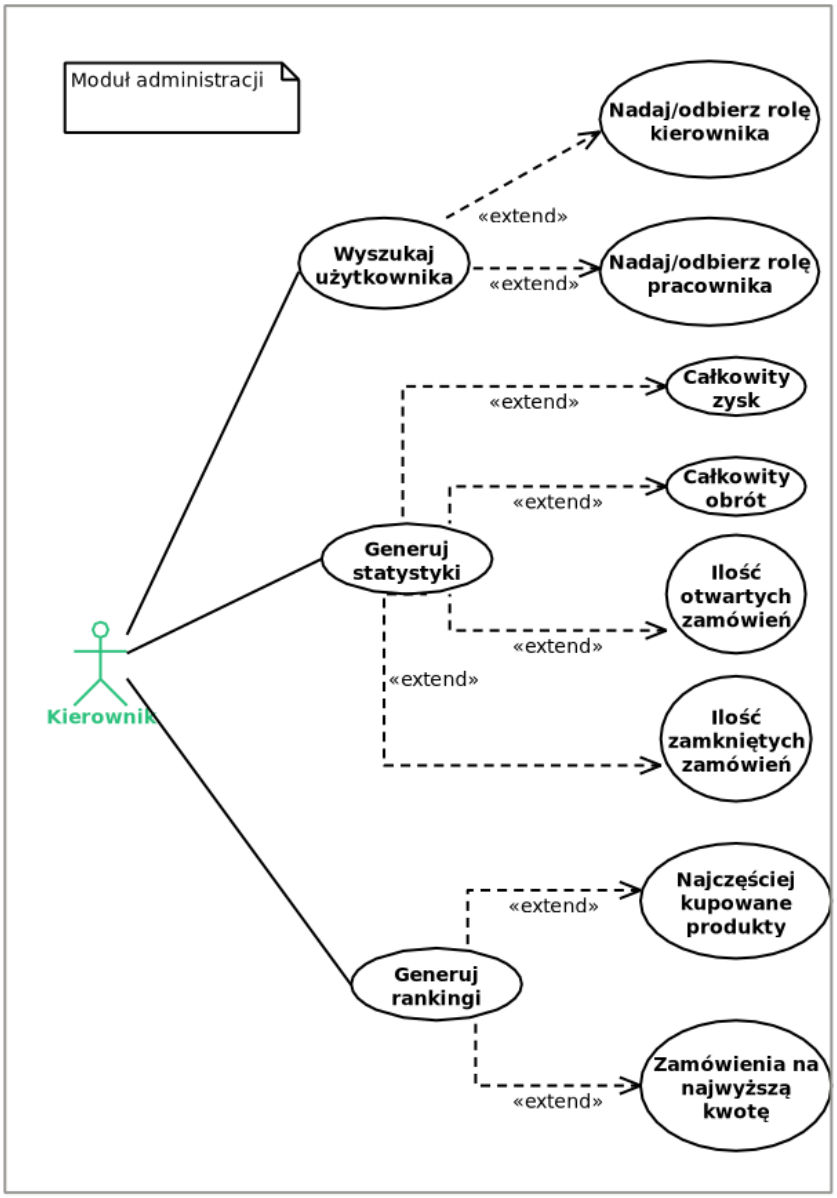
# Opis przypadków użycia

## 4.1 Diagramy przypadków użycia

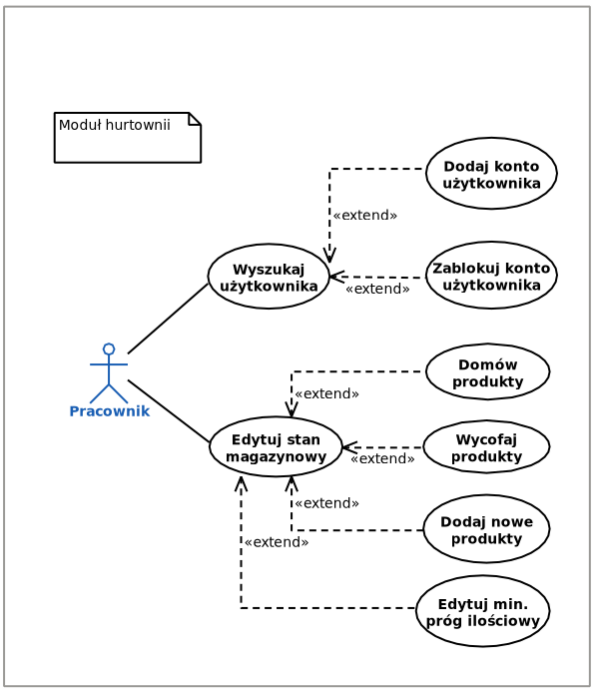



Rysunek 4.1 Aktorzy.

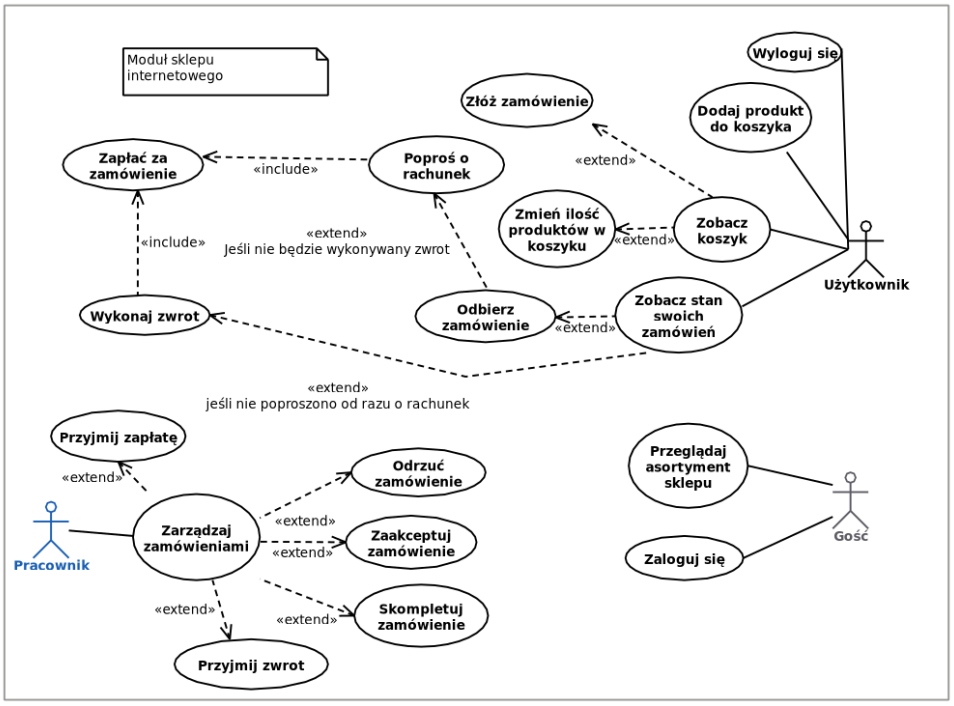
Wykonano kilka drobnych poprawek na diagramie przypadków użycia. Na rysunku 4.4 przedstawiono diagram przypadków użycia dla modułu *Sklep internetowy*. Został zmodyfikowany względem poprzedniej wersji (patrz rysunek 4.25). Wcześniej traktowano przypadki użycia użytkownika „Wykonaj zwrot" i „Złóż zamówienie" jako również przypadki pracownika. Było to błędne, wyodrębnio podobne przypadki dla pracownika o nazwie „Przyjmij zapłatę" oraz „Przyjmij zwrot". Na rysunku 4.3 przedstawiono diagram przypadków użycia dla modułu *Hurtownia*. Został zmodyfikowany względem poprzedniej wersji (patrz rysunek 4.26). Wcześniej brakowało PU „Modyfikuj minimalny próg ilościowy".




Rysunek 4.2 Diagram przypadków użycia dla modułu *Administracja*.

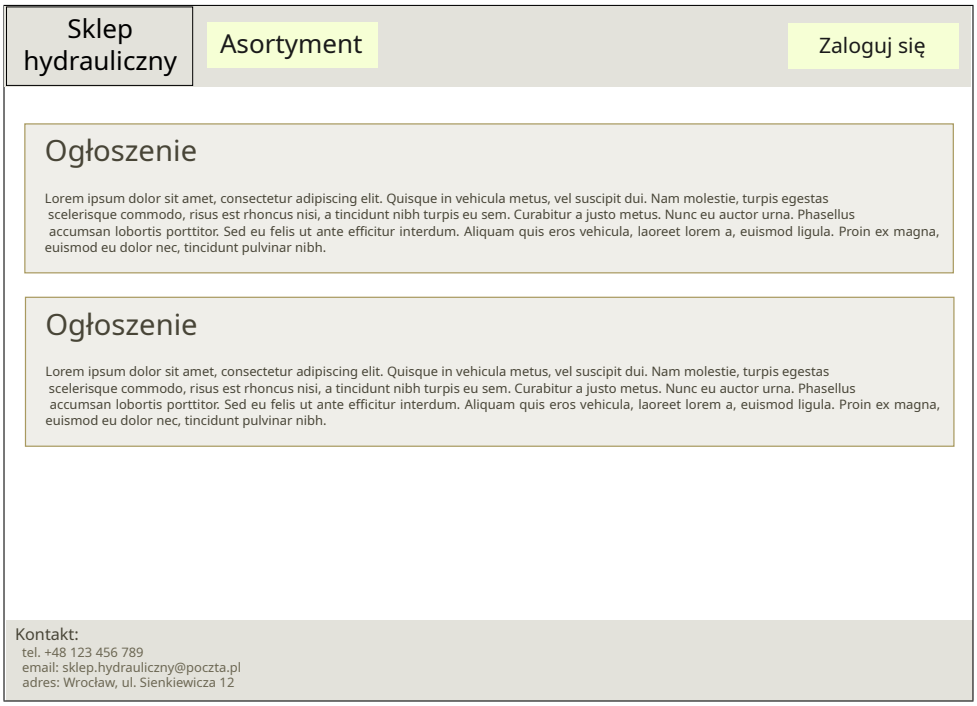



Rysunek 4.3 Diagram przypadków użycia dla modułu *Hurtownia*.

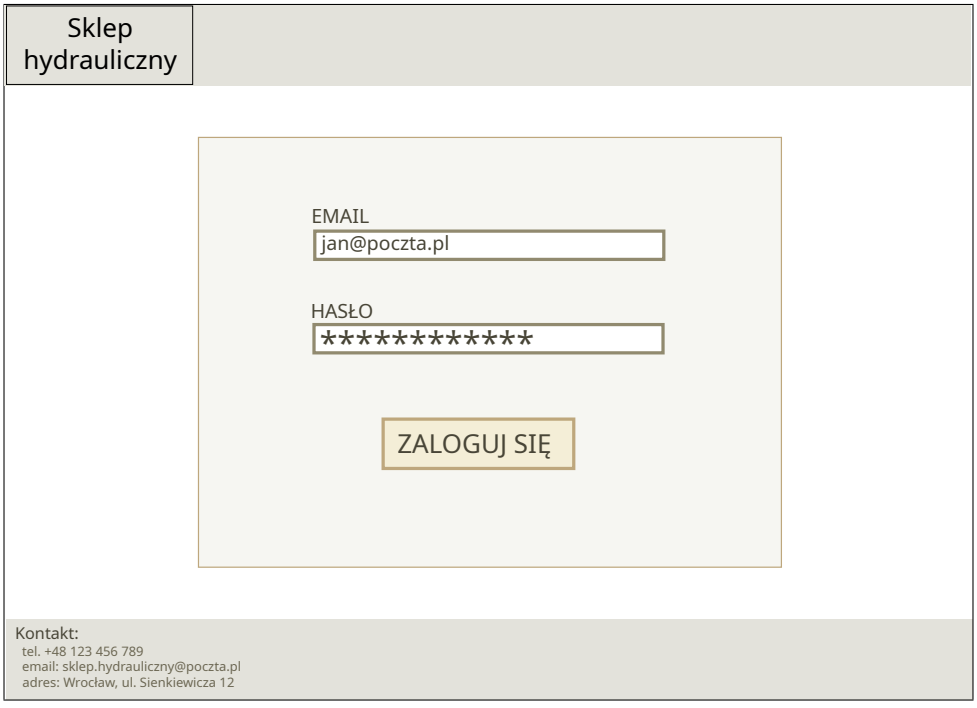



Rysunek 4.4 Diagram przypadków użycia dla modułu *Sklep internetowy*.

## 4.2 Przypadki użycia dla gościa




Rysunek 4.5 Strona tytułowa dla gościa.



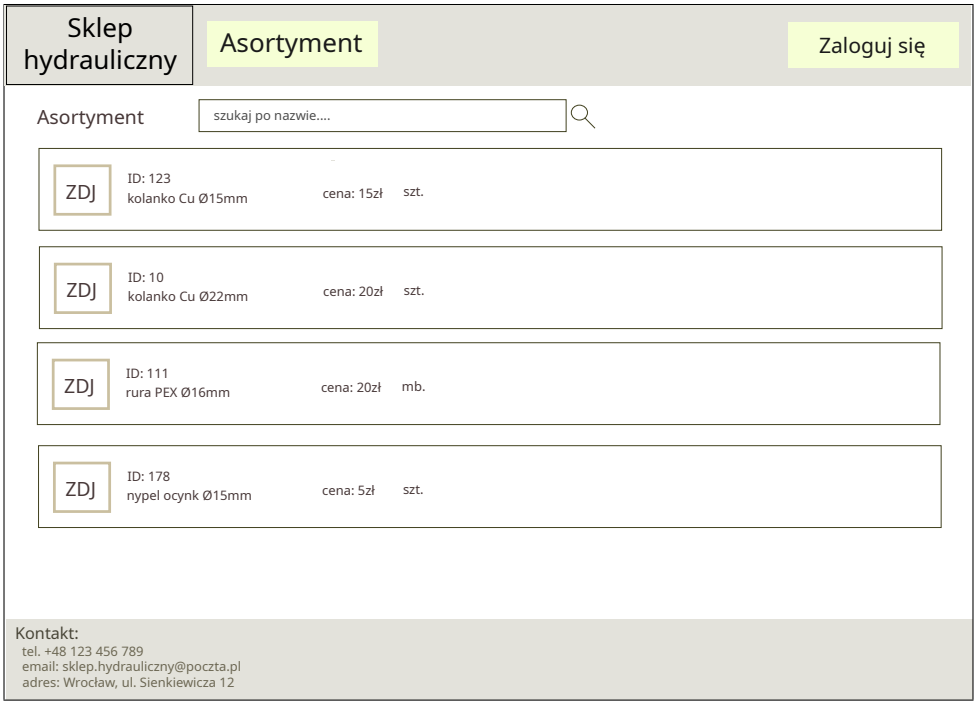

Rysunek 4.6 Ekran logowania

### 4.2.1 PU „Zaloguj się”

- Aktor: gość
- Zdarzenie inicjujące: kliknięcie przycisku „ZALOGUJ SIĘ”
- Warunki początkowe: brak
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „ZALOGUJ SIĘ”, patrz rysunek 4.5.
  2. Wyświetlenie formularza logowania, patrz rysunek 4.6
  3. Wpisanie loginu i hasła.
  4. Kliknięcie przycisku „ZALOGUJ SIĘ”.
  5. Aplikacja sprawdza czy istnieje taki użytkownik, sprawdza poprawność danych.
  6. Zalogowanie użytkownika.
  7. Powrót do strony głównej.
- Przebieg alternatywny realizacji PU: użytkownik o podanych danych nie istnieje. W punkcie 5. pojawia się komunikat o niepowodzeniu. Proces logowania zostaje przerwany.
- Warunki końcowe: Użytkownik zostaje zalogowany.

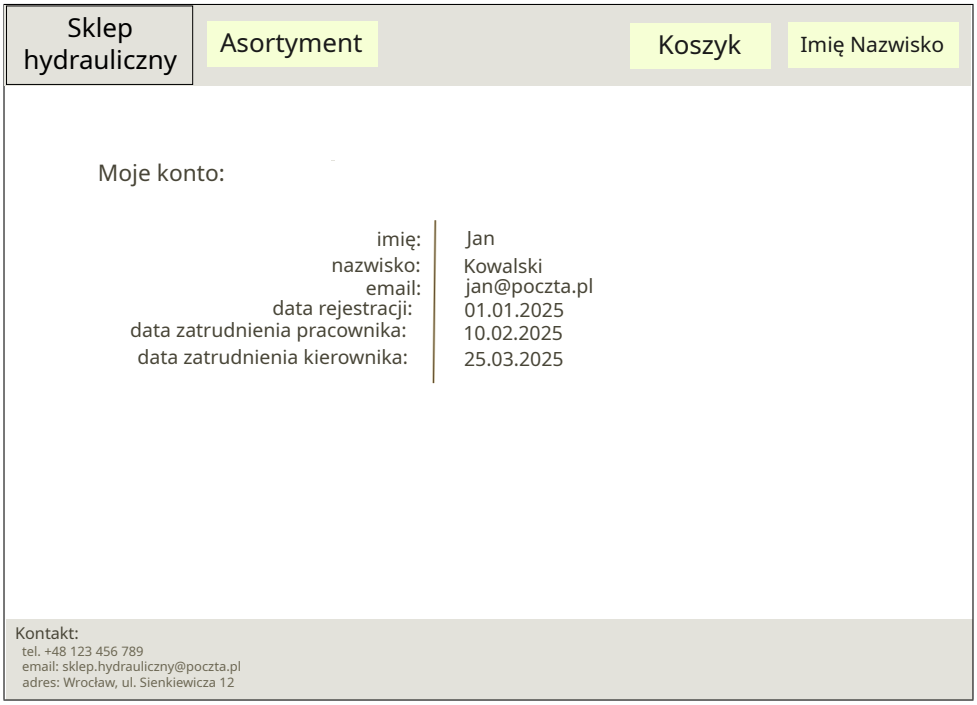
### 4.2.2 PU „Przeglądaj asortyment sklepu”

- Aktor: Gość
- Zdarzenie inicjujące: Kliknięcie przycisku „asortyment”
- Warunki początkowe: Gość w widoku startowym, patrz rysunek 4.5.
- Przebieg podstawowy realizacji PU:
  1. Wciśnięcie przycisku „ASORTYMENT”.
  2. Wyświetlenie widoku asortymentu dla gościa, patrz rysunek 4.7.
- Warunki końcowe: Gość w widoku asortymentu.

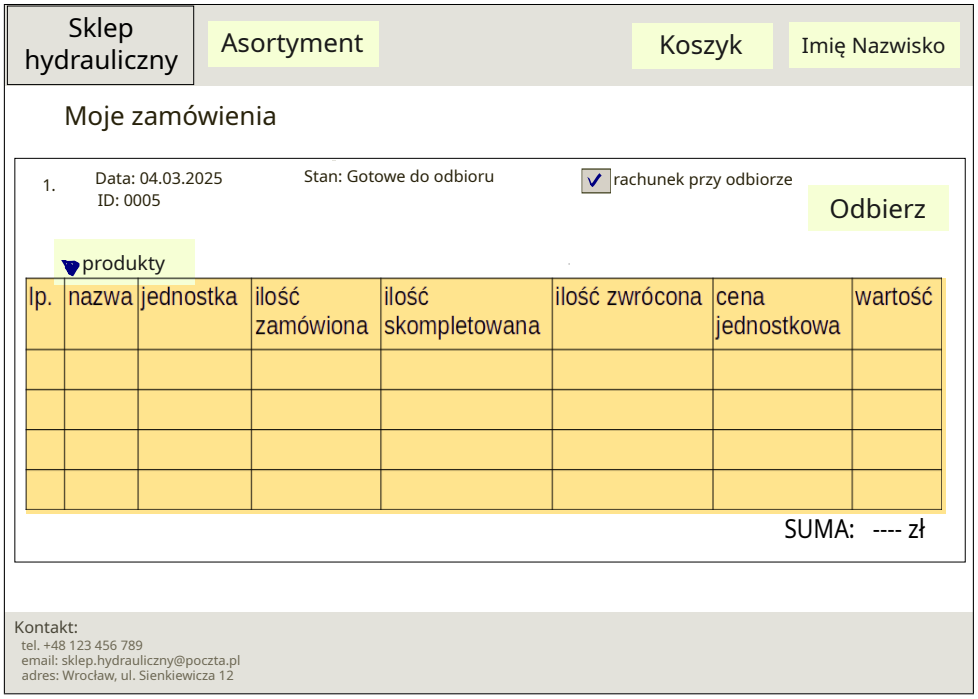



Rysunek 4.7 Przegląd asortymentu przez gościa

## 4.3 Przypadki użycia dla użytkownika




Rysunek 4.8 Ekran z informacjami o koncie



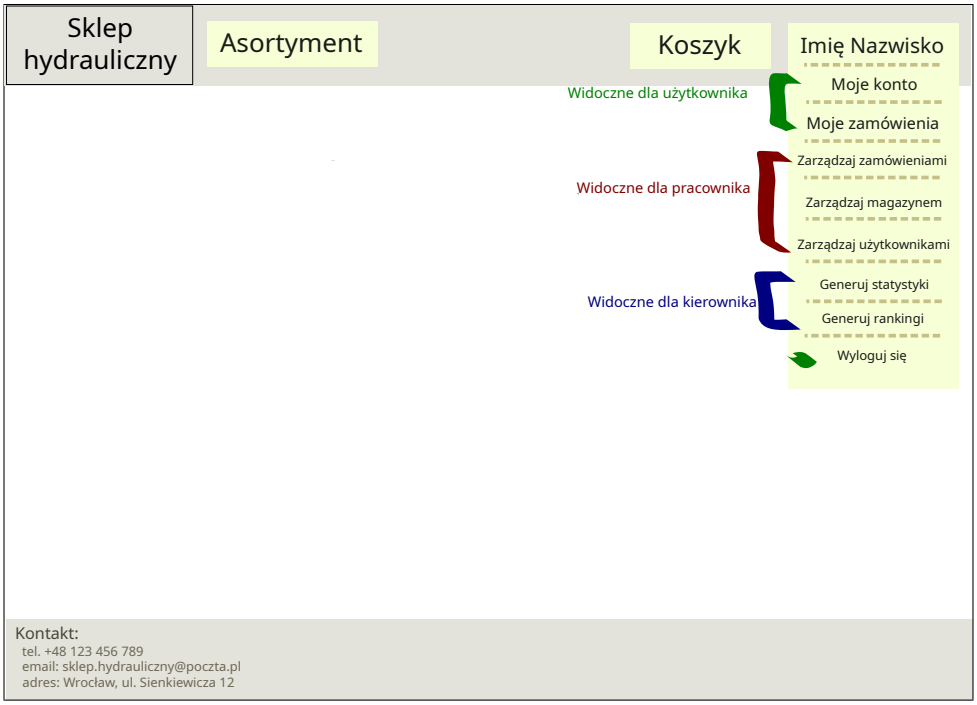

Rysunek 4.9 Ekran z informacjami o zamówieniach użytkownika




Rysunek 4.10 Ekran do wykonywania zwrotu towarów.

### 4.3.1 PU „Wyloguj się”

- Aktor: Użytkownik
- Zdarzenie inicjujące: Kliknięcie przycisku „wyloguj się”




Rysunek 4.11 Uniwersalne rozwijane menu.

- Warunki początkowe: Zalogowany użytkownik na dowolnym ekranie.

- Przebieg podstawowy realizacji PU:

  1. Rozwinięcie uniwersalnego menu w prawym górnym rogu poprzez kliknięcie pola z imieniem i nazwiskiem użytkownika, patrz np. rysunki 4.12 i 4.13.

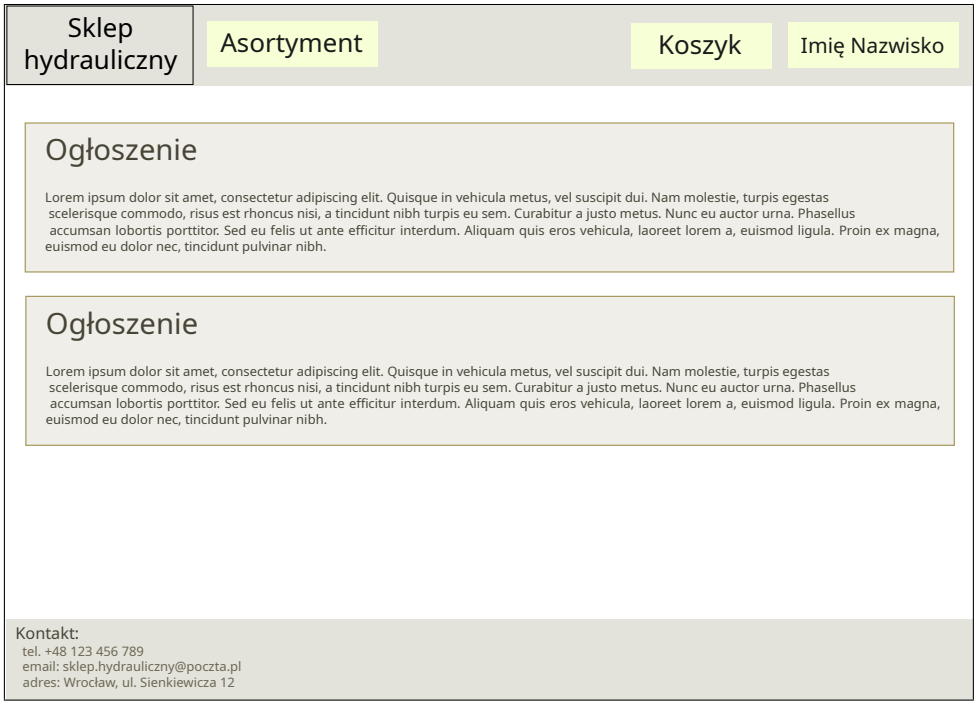
  2. Wyświetlenie się rozwijanego menu, patrz rysunek 4.11.

  3. Kliknięcie przycisku „wyloguj się”

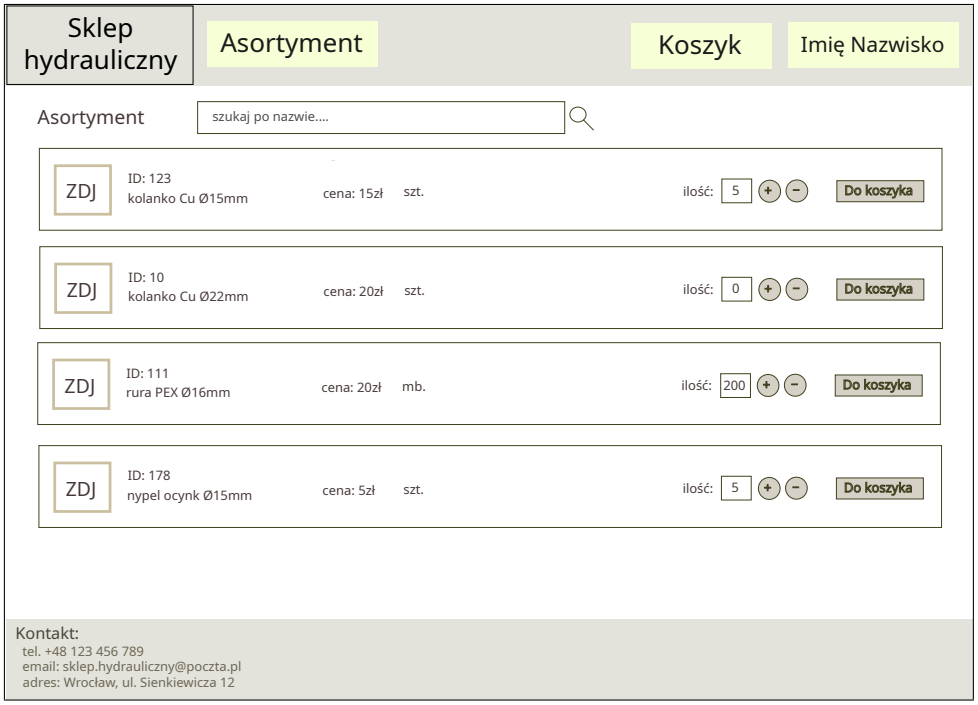
  4. Wylogowanie się.

  5. Powrót do ekranu głównego, patrz rysunek 4.5.

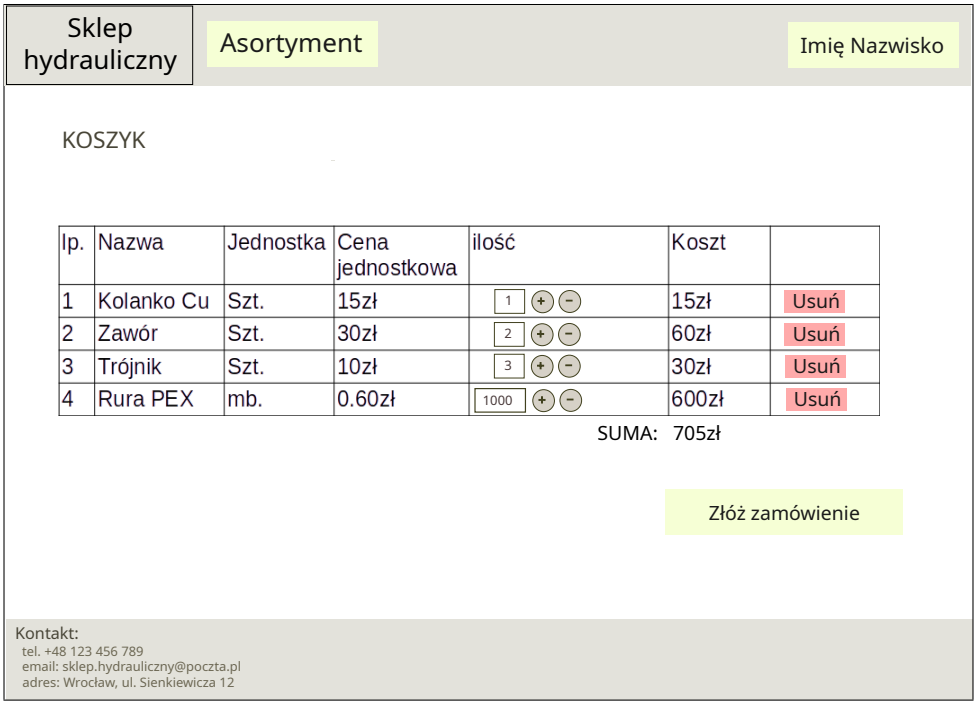
- Warunki końcowe: Wylogowanie i powrót do części aplikacji dostępnej dla gości.




Rysunek 4.12 Strona tytułowa dla użytkownika.




Rysunek 4.13 Ekran asortymentu dla użytkownika.




Rysunek 4.14 Ekran koszyka

### 4.3.2 PU „Dodaj produkt do koszyka”

- Aktor: Użytkownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Do koszyka”
- Warunki początkowe: Użytkownik w widoku asortymentu, patrz rysunek 4.13.
- Przebieg podstawowy realizacji PU:
  1. Wpisanie nazwy szukanego produktu w belce wyszukiwania.
  2. Wyświetlenie produktów spełniających kryterium wyszukiwania (nazwę).
  3. Określenie żądanej ilości produktu przez wpisanie liczby w polu wejściowym chcianego produktu lub wciskanie przycisków „+” i „–”, które odpowiednio zmniejszają i zwiększają ilość o jeden.
  4. Wciśnięcie przycisku „Do koszyka” przy produkcie z określoną ilością.
- Przebieg alternatywny 1. realizacji PU: użytkownik nie wpisuje nic w belce wyszukującej i przewija sam listę produktów. Wtedy od razu przechodzi do drugiego punktu podstawowego przebiegu PU.
- Przebieg alternatywny 2. realizacji PU: użytkownik wpisał ciąg znaków w belce wyszukiwania, do którego nie udało się dopasować żadnego produktu. Wtedy nie wyświetli się żaden produkt. Realizacja PU zatrzyma się na punkcie 1.
- Przebieg alternatywny 3. realizacji PU: użytkownik nie wciśnie przycisku „Do koszyka” po określeniu liczby produktów. Wtedy produkt nie zostanie dodany a relizacja PU zatrzyma się na punkcie 3.

- Warunki końcowe: użytkownik w widoku początkowym, produkt dodany do koszyka.

### 4.3.3 PU „Zobacz koszyk”

- Aktor: Użytkownik
- Zdarzenie inicjujące: kliknięcie przycisku „Koszyk”.
- Warunki początkowe: Użytkownik w dowolnym widoku.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „Koszyk”.
  2. Wyświetlenie widoku koszyka, patrz rysunek 4.14.
- Warunki końcowe: użytkownik w widoku koszyka. Można kontynuować dalsze przypdaki użycia opisane w sekcjach 4.3.3 i 4.3.3.

#### PU „Zmień ilość produktów w koszyku”

- Aktor: Użytkownik
- Zdarzenie inicjujące: kliknięcie przycisków lub wpisanie nowej wartości ilości na ekranie widocznym na rysunku 4.14.
- Warunki początkowe: Użytkownik w widoku koszyka. Kontynuacja PU opisanego w sekcji 4.3.3.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „+” lub „–” w wierszu produktu, aby odpowiednio zwiększyć lub zmniejszyć jego ilość.
  2. Sprawdzenie poprawności operacji przez aplikację.
  3. Wyświetlenie zaktualizowanej ilości.
- Przebieg alternatywny 1. realizacji PU: użytkownik klika przycisk „Usuń” i całkowicie usuwa pozycję z koszyka.
- Przebieg alternatywny 2. realizacji PU: użytkownik, zamiast klikać przyciski, wpisuje nową wartość do pola z ilością. Aplikacja wtedy sprawdza poprawność danych i aktualizuje.
- Warunki końcowe: użytkownik w widoku koszyka ze zmodyfikowaną liczbą produktów w koszyku.

**PU „Złóż zamówienie”**

- Aktor: Użytkownik
- Zdarzenie inicjujące: kliknięcie przycisku „Złóż zamówienie”.
- Warunki początkowe: Użytkownik w widoku koszyka, patrz rysunek 4.14. Kontynuacja PU opisanego w sekcji 4.3.3.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „Złóż zamówienie”.
- Warunki końcowe: użytkownik w widoku startowym ze złożonym zamówieniem.

### 4.3.4 PU „Zobacz stan swoich zamówień”

- Aktor: Użytkownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Moje zamówienia”.
- Warunki początkowe: Użytkownik w dowolnym widoku.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku z imieniem i nazwiskiem użytkownika w prawym górnym rogu.
  2. Wyświetlenie rozwijanego menu, patrz rysunek 4.11.
  3. Kliknięcie przycisku „Moje zamówienia”.
  4. Wyświetlenie widoku zamówień użytkownika, patrz rysunek 4.9.
- Warunki końcowe: użytkownik w widoku zamówień. Można kontynuować dalsze przypadki użycia opisane w sekcjach 4.3.4, 4.3.4, 4.3.4 i 4.3.4.

**PU „Odbierz zamówienie”**

- Aktor: Użytkownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Odbierz”.
- Warunki początkowe: Użytkownik w widoku zamówień, patrz rysunek 4.9. Kontynuacja PU opisanego w sekcji 4.3.4.
- Przebieg podstawowy realizacji PU:
  1. Dla zamówienia w stanie gotowym do odbioru kliknięcie przycisku „Odbierz”.
- Warunki końcowe: użytkownik odebrał zamówienie.

**PU „Poproś o rachunek”**

- Aktor: Użytkownik
- Zdarzenie inicjujące: Zaznaczenie opcji „Poproś o rachunek” w widoku zamówienia, podczas odbioru, patrz rysunek 4.9.
- Warunki początkowe: Użytkownik w widoku zamówień. Kontynuacja PU opisanego w sekcji 4.3.4.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „Poproś o rachunek” w zamówieniu, które zostanie odebrane.
  2. Kliknięcie przycisku „Odbierz” w zamówieniu, które zostanie odebrane.
- Warunki końcowe: użytkownik odebrał zamówienie i poprosił o rachunek, z pominięciem całej sekwencji zwrotu. Zamówienie jest w stanie gotowym do opłacenia.

**PU „Zapłać za zamówienie”**

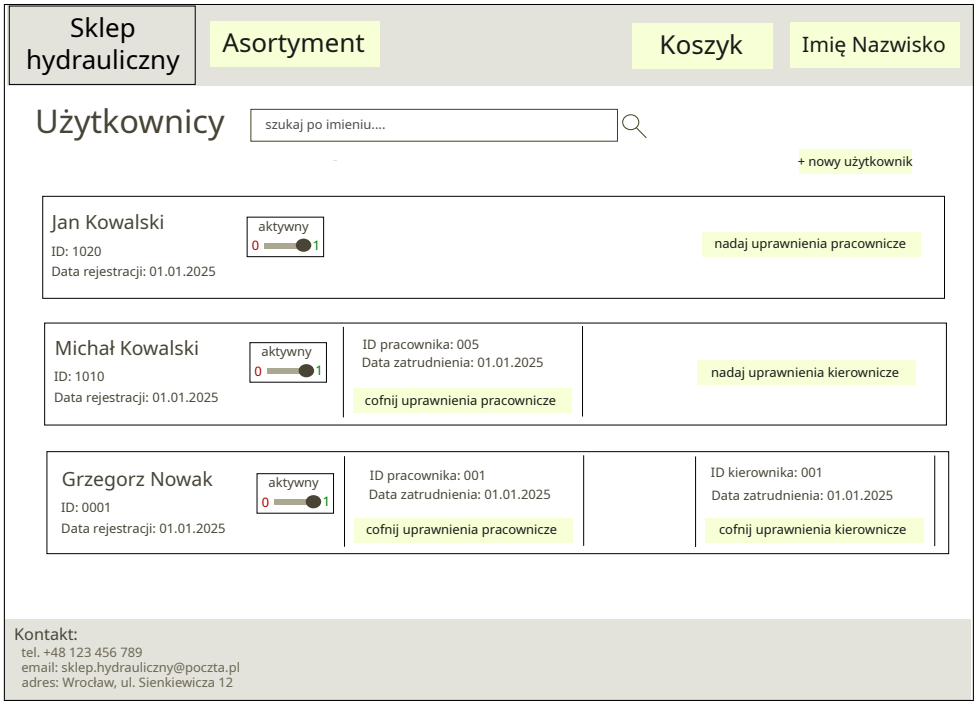
- Aktor: Użytkownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Zapłać”. Dla zamówienia w stanie gotowym do opłaty wyświetli się, w miejscu przycisku „Odbierz” z rysunku 4.9, przycisk „Zapłać”.
- Warunki początkowe: Użytkownik w widoku zamówień. Kontynuacja PU opisanego w sekcji 4.3.4.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „Zapłać” w zamówieniu, które zostanie opłacone.
- Warunki końcowe: zamówienie zostanie opłacone.

**PU „Wykonaj zwrot”**

- Aktor: Użytkownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Zwróć” w zamówieniu, które zostało odebrane. Dla zamówienia w stanie oczekiwania na zwrot wyświetli się, w miejscu przycisku „Odbierz” z rysunku 4.9, przycisk „Zwróć”.
- Warunki początkowe: Użytkownik w widoku zamówień. Kontynuacja PU opisanego w sekcji 4.3.4.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „Zwróć” w zamówieniu, które zostanie zwrócone.
  2. Otwarcie okna zwrotu, patrz rysunek 4.10.

3. Wypełnienie formularza zwrotu. Określenie ile produktów zostaje zwróconych.
4. Kliknięcie przycisku „Zatwierdź".
5. Zamówienie zostaje zwrócone.

- Przebieg alternatywny 1. realizacji PU: użytkownik klika przycisk „Anuluj" w oknie zwrotu. Wtedy zamówienie nie zostanie zwrócone, a użytkownik wróci do widoku zamówień.
- Przebieg alternatywny 2. realizacji PU: użytkownik klika przycisk „Zwróć", wtedy następuje automatyczne uzupełnienie kolumny zwróconych produktów do poziomu skompletowanych – czyli wszystkie produkty, które zostały skompletowane w zamówieniu, zostaną zwrócone. Użytkownik może zmniejszyć tę wartość, ale nie może jej zwiększyć. Wtedy aplikacja wyświetli komunikat o błędzie i nie pozwoli na wprowadzenie takiej wartości.
- Warunki końcowe: zamówienie zostanie zwrócone.




Rysunek 4.15 Ekran wyszukiwania użytkownika

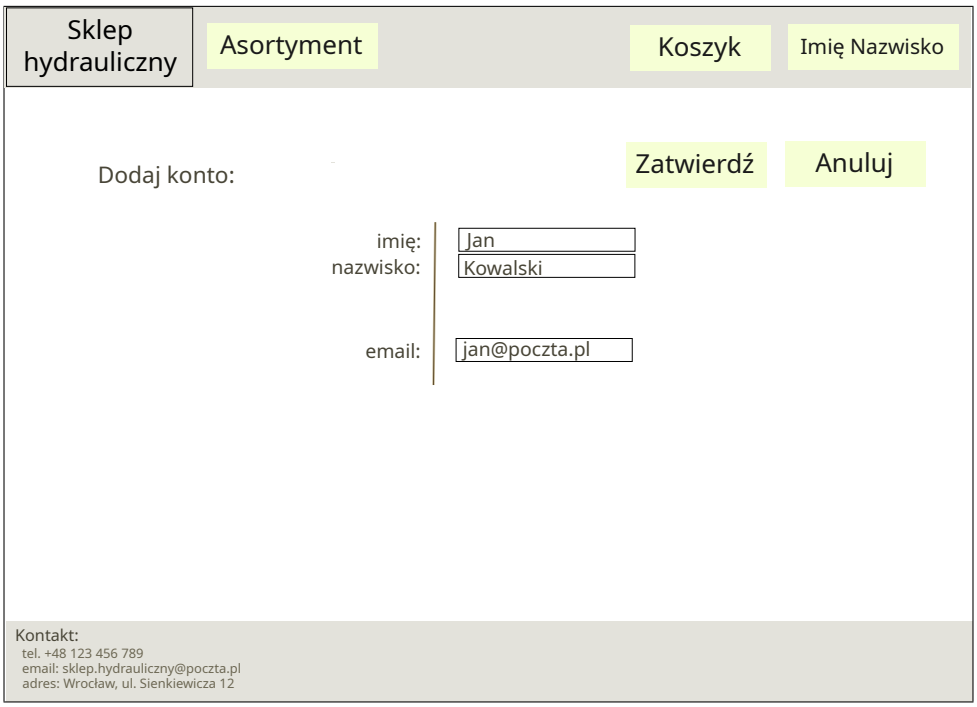
## 4.4 Przypadki użycia dla pracownika

### 4.4.1 PU „Wyszukaj użytkownika”

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Zarządzaj użytkownikami”.
- Warunki początkowe: Pracownik w dowolnym ekranie.
- Przebieg podstawowy realizacji PU:
  1. Rozwinięcie uniwersalnego menu (imię i nazwisko w prawym górnym rogu).
  2. Wyświetlenie menu kontekstowego dla pracownika, patrz rysunek 4.11.
  3. Kliknięcie przycisku „Zarządzaj użytkownikami”.
  4. Otwarcie widoku zarządzania użytkownikami, patrz rysunek 4.15.
- Przebieg alternatywny realizacji PU: pracownik wpisuje w belce wyszukiwania imię i nazwisko użytkownika, którego szuka. Aplikacja wyświetla użytkowników, którzy pasują do podanego kryterium.
- Warunki końcowe: Pracownik w widoku zarządzania użytkownikami. Można kontynuować dalsze przypadki użycia opisane w sekcjach 4.4.1 i 4.4.1.




Rysunek 4.16 Ekran do obsługi skompletowań.



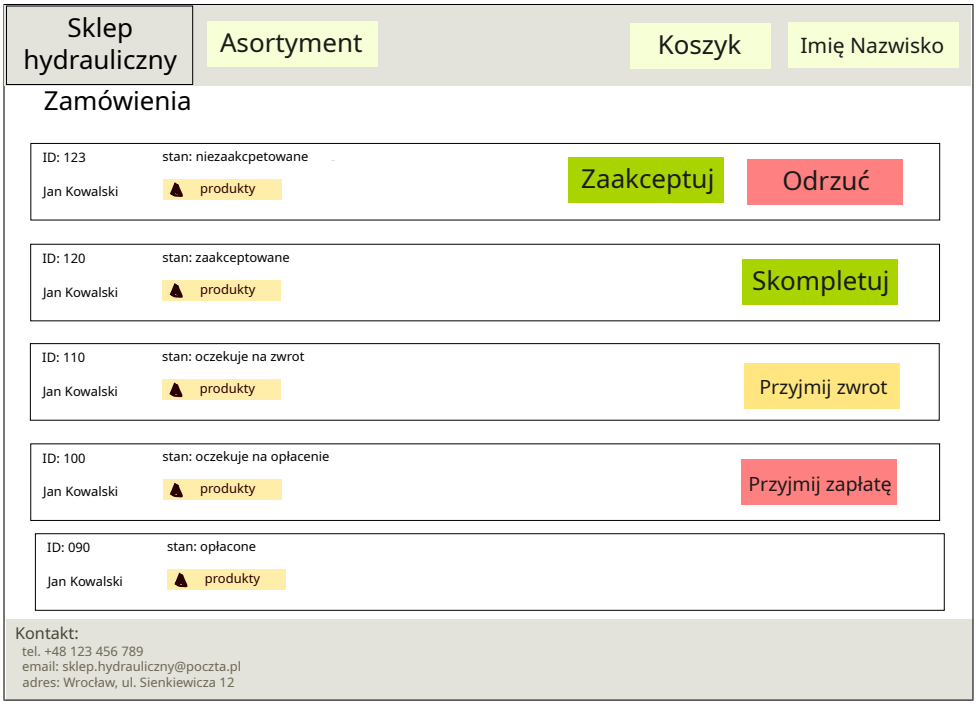

Rysunek 4.17 Ekran do dodawania nowych użytkowników.

### PU „Dodaj konto użytkownika”

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku „nowy użytkownik”.
- Warunki początkowe: Pracownik w widoku zarządzania użytkownikami, patrz rysunek 4.15. Kontynuacja PU opisanego w sekcji 4.4.1.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „nowy użytkownik”.
  2. Wyświetlenie formularza dodawania użytkownika, patrz rysunek 4.17.
  3. Wprowadzenie danych użytkownika.
  4. Kliknięcie przycisku „Zatwierdź”.
- Przebieg alternatywny 1. realizacji PU: pracownik nie wprowadza wszystkich danych. Wtedy aplikacja wyświetla komunikat o błędzie i nie dodaje użytkownika.
- Przebieg alternatywny 2. realizacji PU: pracownik wprowadza źle sformatowane dane. Wtedy aplikacja wyświetla komunikat o błędzie i nie dodaje użytkownika.
- Przebieg alternatywny 3. realizacji PU: pracownik klika przycisk „Anuluj”. Wtedy aplikacja nie dodaje użytkownika.
- Przebieg alternatywny 4. realizacji PU: użytkownik o podanym emailu już istnieje. Wtedy aplikacja wyświetla komunikat o błędzie i nie dodaje użytkownika.
- Warunki końcowe: Nowy użytkownik zostaje dodany do bazy danych.

### PU „Zablokuj/odblokuj konto użytkownika”

- Aktor: Pracownik
- Zdarzenie inicjujące: Przestawienie suwaka aktywacji użytkownika.
- Warunki początkowe: Pracownik w widoku zarządzania użytkownikami (patrz rysunek 4.15). Kontynuacja PU opisanego w sekcji 4.4.1.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie suwaka podpisanego „aktywny” w wierszu użytkownika, którego konto ma być zablokowane. Taki suwak będzie klikalny tylko dla użytkowników, którzy nie są pracownikami.
- Warunki końcowe: pracownik w widoku zarządzania użytkownikami z zablokowanym/odblokowanym kontem użytkownika.




Rysunek 4.18 Ekran zarządzania zamówieniami

### 4.4.2 PU „Zarządzaj zamówieniami”

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku „zarządzaj zmówieniami”
- Warunki początkowe: Pracownik w dowolnym ekranie,
- Przebieg podstawowy realizacji PU:
  1. Rozwinięcie uniwersalnego menu (imię i nazwisko w prawym górnym rogu).
  2. Wyświetlenie rozwijanego menu, patrz rysunek 4.11.
  3. Kliknięcie przycisku „zarządzaj zamówieniami”.
  4. Otwarcie ekranu zarządzania zamówieniami, patrz rysunek 4.18.
- Warunki końcowe: Otwarty ekran zarządzania zamówieniami. Można kontynuować dalsze przypadki użycia opisane w sekcjach 4.4.2, 4.4.2, 4.4.2 i 4.4.2.

#### PU „Zaakceptuj zamówienie”

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku zatwierdzania zamówienia.
- Warunki początkowe: Pracownik w panelu zarządzania zamówieniami. Kontynuacja PU opisanego w sekcji 4.4.2.

- Przebieg podstawowy realizacji PU:
  1. Dla zamówienia w stanie oczekującym na akceptację kliknięcie przycisku „Zaakceptuj".
- Przebieg alternatywny realizacji PU: pracownik klika przycisk produkty i następuje rozwinięcie tabeli z produktami. W sposób taki jak w tabelce na rysunku 4.9.
- Warunki końcowe: Nowe zamówienie zostaje zatwierdzone.

**PU „Odrzuć zamówienie"**

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku odrzucania zamówienia.
- Warunki początkowe: Pracownik w panelu zarządzania zamówieniami. Kontynuacja PU opisanego w sekcji 4.4.2.
- Przebieg podstawowy realizacji PU:
  1. Dla zamówienia w stanie oczekującym na akceptację kliknięcie przycisku „Odrzuć".
- Przebieg alternatywny realizacji PU: pracownik klika przycisk produkty i następuje rozwinięcie tabeli z produktami. W sposób taki jak w tabelce na rysunku 4.9.
- Warunki końcowe: Nowe zamówienie zostaje odrzucone.

**PU „Skompletuj zamówienie"**

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Skompletuj" dla zamówienia oczekującego na skompletowanie.
- Warunki początkowe: Pracownik w oknie zarządzania zamówieniami, patrz rysunek 4.18. Kontynuacja PU opisanego w sekcji 4.4.2.
- Przebieg podstawowy realizacji PU:
  1. Dla zamówienia w stanie oczekującym na skompletowanie kliknięcie przycisku „Skompletuj".
  2. Otwarcie okna skompletowania zamówienia, patrz rysunek 4.16.
  3. W odpowiednich wierszach tabeli pracownik wpisuje ilości produktów, które są dostępne w magazynie.
  4. Kliknięcie przycisku „Zatwierdź" w celu zapisania zmian.
  5. Wyświetlenie komunikatu o powodzeniu operacji.

6. Zamówienie zmienia status na „oczekujące na odbiór”.

- Przebieg alternatywny 1. realizacji PU: pracownik klika przycisk „Anuluj” i zamyka okno skompletowania zamówienia.

- Przebieg alternatywny 2. realizacji PU: pracownik klika przycisk „Skompletuj" w oknie skompletowania zamówienia. Wtedy następuje automatyczne uzupełnienie kolumny skompletowanych produktów do poziomu zamówionych, lub maksymalnie do poziomu dostępnych w magazynie – czyli wszystkie produkty, które zostały zamówione, zostaną skompletowane. Pracownik może zmniejszyć tę wartość, ale nie może jej zwiększyć. Wtedy aplikacja wyświetli komunikat o błędzie i nie pozwoli na wprowadzenie takiej wartości.

- Warunki końcowe: Zamówienie zmienia status na „oczekujące na odbiór”.

**PU „Przyjmij zwrot”**

- Aktor: Pracownik

- Zdarzenie inicjujące: Kliknięcie przycisku „Przyjmij zwrot” w zamówieniu, które zostało odebrane.

- Warunki początkowe: Pracownik w oknie zarządzania zamówieniami, patrz rysunek 4.18. Kontynuacja PU opisanego w sekcji 4.4.2.

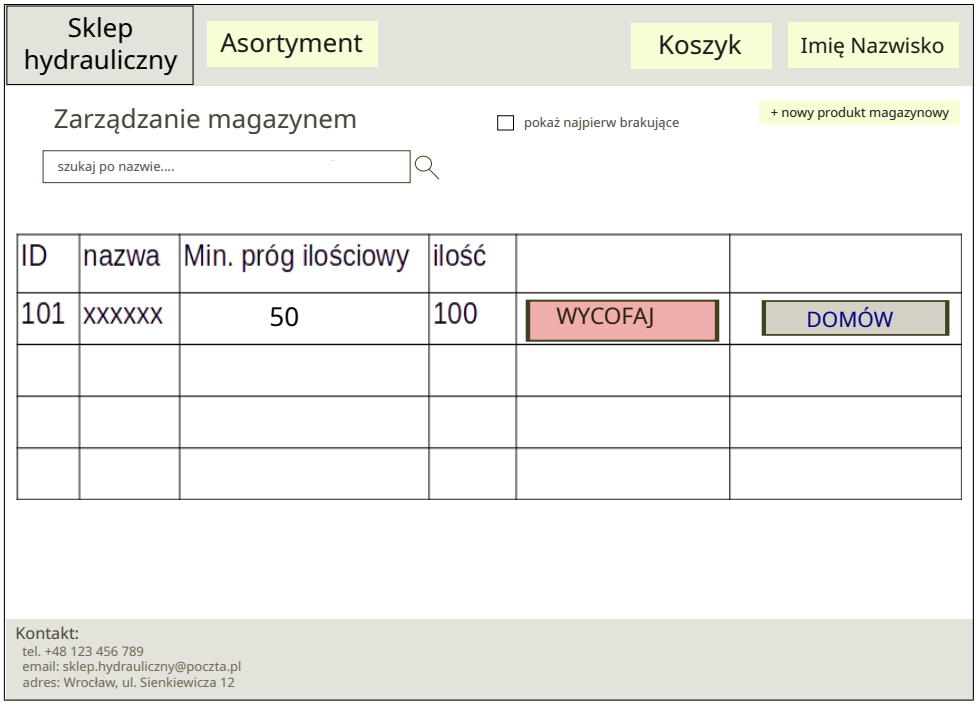
- Przebieg podstawowy realizacji PU:

  1. Kliknięcie przycisku „Przyjmij zwrot” w zamówieniu, które zostanie zwrócone.
  2. Otwarcie okna zwrotu zamówienia, patrz rysunek 4.10.
  3. Wypełnienie formularza zwrotu. Określenie ile produktów zostaje zwróconych.
  4. Kliknięcie przycisku „Zatwierdź”.
  5. Zamówienie zostaje zwrócone.

- Przebieg alternatywny 1. realizacji PU: pracownik klika przycisk „Anuluj” w oknie zwrotu. Wtedy zamówienie nie zostanie zwrócone, a pracownik wróci do widoku zamówień.

- Przebieg alternatywny 2. realizacji PU: pracownik klika przycisk „Zwróć”, wtedy następuje automatyczne uzupełnienie kolumny zwróconych produktów do poziomu skompletowanych – czyli wszystkie produkty, które zostały skompletowane w zamówieniu, zostaną zwrócone. Pracownik może zmniejszyć tę wartość, ale nie może jej zwiększyć. Wtedy aplikacja wyświetli komunikat o błędzie i nie pozwoli na wprowadzenie takiej wartości.

- Warunki końcowe: zamówienie zostaje zwrócone.




Rysunek 4.19 Ekran edycji stanu magazynowego

**PU „Przyjmij zapłatę”**

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku „Przyjmij zapłatę” w zamówieniu, które zostało odebrane.
- Warunki początkowe: Pracownik w oknie zarządzania zamówieniami, patrz rysunek 4.18. Kontynuacja PU opisanego w sekcji 4.4.2.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „Przyjmij zapłatę” w zamówieniu, które zostanie opłacone.
  2. Zamówienie zostaje opłacone.
- Warunki końcowe: zamówienie zostaje opłacone.

### 4.4.3 PU „Edytuj stan magazynowy”

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku „zarządzaj magazynem”.
- Warunki początkowe: Pracownik w dowolnym ekranie.
- Przebieg podstawowy realizacji PU:




Rysunek 4.20 Formularz danych produktu.

1. Rozwinięcie uniwersalnego menu (imię i nazwisko w prawym górnym rogu).
2. Wyświetlenie rozwijanego menu, patrz rysunek 4.11.
3. Kliknięcie przycisku „zarządzaj magazynem".
4. Otwarcie widoku edycji stanu magazynowego, patrz rysunek 4.19.

- Warunki końcowe: pracownik w ekranie zarządzania magazynem. Można kontynuować dalsze przypadki użycia opisane w sekcjach 4.4.3, 4.4.3, 4.4.3 i 4.4.3.

### PU „Domów produkty"

- Aktor: Pracownik
- Zdarzenie inicjujące: kliknięcie przycisku „domów".
- Warunki początkowe: Pracownik w oknie zarządzania magazynem, patrz rysunek 4.19. Kontynuacja PU opisanego w sekcji 4.4.3.
- Przebieg podstawowy realizacji PU:

1. Kliknięcie przycisku „domów" w wierszu produktu, który ma być domówiony.
2. Otwarcie formularza domówienia produktu, patrz rysunek 4.22.
3. Wpisanie ilości produktów, które mają być domówione.
4. Kliknięcie przycisku „Zatwierdź".




Rysunek 4.21 Formularz danych produktu magazynowego.




Rysunek 4.22 Formularz domawiania produktu.

5. Produkt zostaje domówiony.

- Warunki końcowe: Zwiększony stan magazynowy produktu. Pracownik w oknie edycji zarządzania magazynem.
- Przebieg alternatywny 1. realizacji PU: Pracownik klika przycisk „Anuluj" i zamyka formularz domówienia produktu.
- Przebieg alternatywny 2. realizacji PU: Pracownik nie wypełnia formularza poprawnie i klika przycisk „Zatwierdź". Wtedy aplikacja wyświetli komunikat o błędzie.
- Przebieg alternatywny 3. realizacji PU: Wpisanie wartości ujemnej w formularzu domówienia produktu. Wtedy aplikacja wyświetli komunikat o błędzie.

**PU „Wycofaj produkty"**

- Aktor: Pracownik
- Zdarzenie inicjujące: kliknięcie przycisku „wycofaj".
- Warunki początkowe: Pracownik w oknie zarządzania magazynem, patrz rysunek 4.19. Kontynuacja PU opisanego w sekcji 4.4.3.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „wycofaj" w wierszu produktu, który ma być wycofany.
  2. Produkt zostaje wycofany z magazynu.
- Przebieg alternatywny realizacji PU: jeżeli ilość produktów w magazynie jest większa od 0, to aplikacja wyświetli komunikat o błędzie.
- Warunki końcowe: Pozycja magazynowa ma zmieniony status aktywności.

**PU „Dodaj nowe produkty"**

- Aktor: Pracownik
- Zdarzenie inicjujące: Kliknięcie przycisku „nowy produkt magazynowy".
- Warunki początkowe: Pracownik w oknie edycji stanu magazynowego. Kontynuacja PU opisanego w sekcji 4.4.3.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „nowy produkt magazynowy".
  2. Otwarcie formularza dodawania nowego produktu magazynowego, patrz rysunek 4.20.
  3. Wypełnienie formularza danymi nowego produktu.

4. Kliknięcie przycisku „Zatwierdź”.

- Przebieg alternatywny 1. realizacji PU: Pracownik klika przycisk „Anuluj” i zamyka formularz dodawania nowego produktu.
- Przebieg alternatywny 2. realizacji PU: Pracownik nie wypełnia formularza poprawnie i klika przycisk „Zatwierdź”. Wtedy aplikacja wyświetli komunikat o błędzie.
- Przebieg alternatywny 3. realizacji PU: Pracownik nie klika przycisku „Zatwierdź” i zamyka formularz dodawania nowego produktu.

1. Kliknięcie przycisku „nowy produkt”.
2. Otwarcie formularza dodawania nowego produktu, patrz rysunek 4.20.
3. Pracownik wprowadza dane nowego produktu.
4. Kliknięcie przycisku „Zatwierdź”.
5. Produkt zostaje dodany do bazy danych.
6. Pracownik wraca do poprzedniego widoku, patrz rysunek 4.19.
7. Kliknięcie przycisku „Zatwierdź”.
8. Dodanie nowego produktu magazynowego do bazy danych.

- Warunki końcowe: Dodany nowy produkt magazynowy.

**PU „Edytuj minimalny próg ilościowy”**

- Aktor: Pracownik
- Zdarzenie inicjujące: kliknięcie przycisku „edytuj progi”.
- Warunki początkowe: Pracownik w oknie edycji stanu magazynowego, patrz rysunek 4.19. Kontynuacja PU opisanego w sekcji 4.4.3.
- Przebieg podstawowy realizacji PU:

1. Wpisanie nowej wartości minimalnego progu ilościowego w wierszu produktu, który ma być edytowany.

- Warunki końcowe: Edytowany minimalny próg ilościowy produktu.

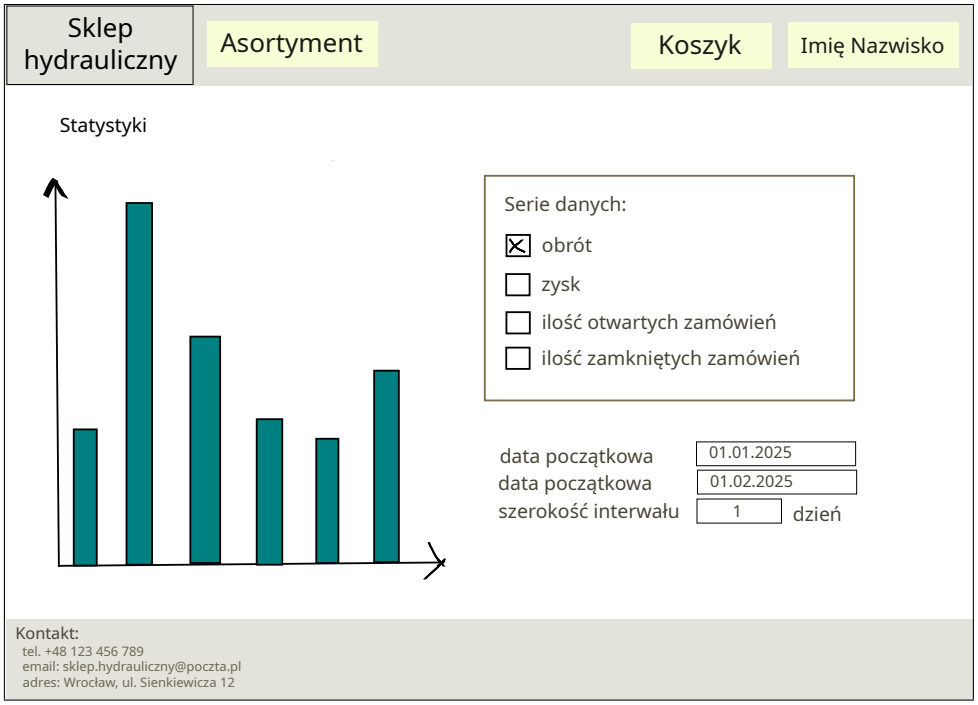
## 4.5 Przypadki użycia dla kierownika

### 4.5.1 PU „Nadaj/odbierz rolę pracownika”

- Aktor: Kierownik
- Zdarzenie inicjujące: kliknięcie przycisku „nadaj uprawnienia pracownicze” lub „odbierz uprawnienia pracownicze”.
- Warunki początkowe: Kierownik w ekranie zarządzania użytkownikami, patrz rysunek 4.15.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „nadaj uprawnienia pracownicze” lub „odbierz uprawnienia pracownicze” przy żądanym użytkowniku.
  2. Obsługa procesu przez aplikację.
- Warunki końcowe: Kierownik w ekranie wyszukiwania użytkownika, użytkownik ze zmienioną rolą pracowniczą.

### 4.5.2 PU „Nadaj/odbierz rolę kierownika”

- Aktor: Kierownik
- Zdarzenie inicjujące: kliknięcie przycisku „nadaj uprawnienia kierownicze” lub „odbierz uprawnienia kierownicze”.
- Warunki początkowe: Kierownik w ekranie zarządzania użytkownikami, patrz rysunek 4.15.
- Przebieg podstawowy realizacji PU:
  1. Kliknięcie przycisku „nadaj uprawnienia kierownicze” lub „odbierz uprawnienia kierownicze” przy żądanym użytkowniku.
  2. Obsługa procesu przez aplikację.
  3. Wyświetlenie informacji o powodzeniu operacji.
- Przebieg alternatywny realizacji PU: W przypadku cofania uprawnień kierowniczych możliwe jest, że po wykonaniu tej operacji w systemie nie pozostanie żaden kierownik. Taka sytuacja jest niedopuszczalna, wtedy aplikacja w punkcie 3. realizacji PU zostanie wyświetlony komunikat o powodzie odrzucenia operacji.
- Warunki końcowe: Kierownik w ekranie wyszukiwania użytkownika, użytkownik ze zmienioną rolą pracowniczą.




Rysunek 4.23 Okno generowania statystyk.

### 4.5.3 PU „Generuj statystyki dla sklepu w określonym czasie”

- Aktor: Kierownik
- Zdarzenie inicjujące: Kliknięcie przycisku generowania statystyk
- Warunki początkowe: Kierownik w dowolnym ekranie.
- Przebieg podstawowy realizacji PU:
  1. Rozwinięcie uniwersalnego menu (imię i nazwisko w prawym górnym rogu).
  2. Wyświetlenie rozwijanego menu, patrz rysunek 4.11.
  3. Kliknięcie przycisku „GENERUJ STATYSTYKI”.
  4. Otwarcie okna generacji statystyk, patrz rysunek 4.23.
  5. Wybór opcji serii danych.
  6. Wybór początku i końca wizualizowanego przedziału czasu, wraz z wyborem długości interwału/szerokości pojedynczego paska histogramu.
- Warunki końcowe: Kierownik w oknie generowania statystyk, wygenerowany histogram. Można kontynuować dalsze przypadki użycia opisane w sekcjach 4.5.3, 4.5.3, 4.5.3 i 4.5.3.

#### PU „Całkowity zysk”

- Aktor: Kierownik

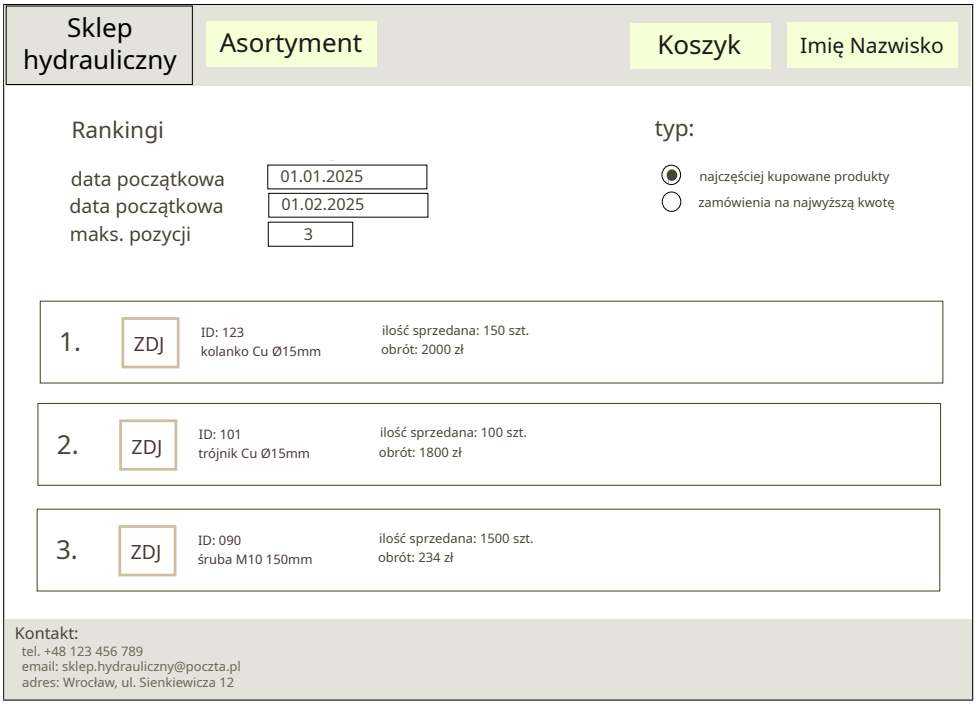
- Zdarzenie inicjujące: Zaznaczenie opcji dodania serii całkowitego zysku na histogramie,
- Warunki początkowe: Kierownik w oknie generacji statystyk. Kontynuacja PU opisanego w sekcji 4.5.3.
- Przebieg podstawowy realizacji PU:
  1. Zaznaczenie opcji „Całkowity zysk" w podmenu „serie danych", patrz rysunek 4.23.
- Warunki końcowe: Generacja statystyk całkowitego zysku.

**PU „Ilość otwartych zamówień"**

- Aktor: Kierownik
- Zdarzenie inicjujące: Zaznaczenie opcji dodania serii ilości otwartych zamówień na histogramie,
- Warunki początkowe: Kierownik w oknie generacji statystyk. Kontynuacja PU opisanego w sekcji 4.5.3.
- Przebieg podstawowy realizacji PU:
  1. Zaznaczenie opcji „Ilość otwartych zamówień" w podmenu „serie danych", patrz rysunek 4.23.
- Warunki końcowe: Generacja statystyk ilości otwartych zamówień.

**PU „Całkowity obrót"**

- Aktor: Kierownik
- Zdarzenie inicjujące: Zaznaczenie opcji dodania serii całkowitego obrotu na histogramie,
- Warunki początkowe: Kierownik w oknie generacji statystyk. Kontynuacja PU opisanego w sekcji 4.5.3.
- Przebieg podstawowy realizacji PU:
  1. Zaznaczenie opcji „Całkowity obrót" w podmenu „serie danych", patrz rysunek 4.23.
- Warunki końcowe: Generacja statystyk całkowitego obrotu.




Rysunek 4.24 Ono generowania rankingów

**PU „Ilość zamkniętych zamówień”**

- Aktor: Kierownik
- Zdarzenie inicjujące: Zaznaczenie opcji dodania serii ilości zamkniętych zamówień na histogramie,
- Warunki początkowe: Kierownik w oknie generacji statystyk. Kontynuacja PU opisanego w sekcji 4.5.3.
- Przebieg podstawowy realizacji PU:
  1. Zaznaczenie opcji „Ilość zamkniętych zamówień” w podmenu „serie danych”, patrz rysunek 4.23.
- Warunki końcowe: Generacja statystyk ilości zamkniętych zamówień.

### 4.5.4 PU „Generuj rankingi”

- Aktor: Kierownik
- Zdarzenie inicjujące: Kliknięcie przycisku generowania rankingów
- Warunki początkowe: Kierownik w dowolnym ekranie.
- Przebieg podstawowy realizacji PU:
  1. Rozwinięcie uniwersalnego menu (imię i nazwisko w prawym górnym rogu).

2. Wyświetlenie rozwijanego menu, patrz rysunek 4.11.
3. Kliknięcie przycisku „GENERUJ RANKINGI".
4. Otwarcie okna generacji statystyk, patrz rysunek 4.24.
5. Wybór opcji.
6. Wyświetlenie rankingu.

- Warunki końcowe: Kierownik w oknie generowania rankingów, wygenerowany ranking. Można kontynuować dalsze przypadki użycia opisane w sekcjach 4.5.4 i 4.5.4.

**PU „Zamówienia na najwyższą kwotę"**

- Aktor: Kierownik
- Zdarzenie inicjujące: Wybór opcji zamówień na najwyższą kwotę.
- Warunki początkowe: Kierownik w oknie generowania rankingów. Kontynuacja PU opisanego w sekcji 4.5.4.
- Przebieg podstawowy realizacji PU:

1. Zaznaczenie odpowiedniej opcji 4.24.
2. Wybór daty początkowej i końcowej.
3. Wybór ilości pozycji w rankingu.
4. Wyświetlenie rankingu.

- Warunki końcowe: Kierownik w oknie generowania rankingów, wygenerowany ranking.

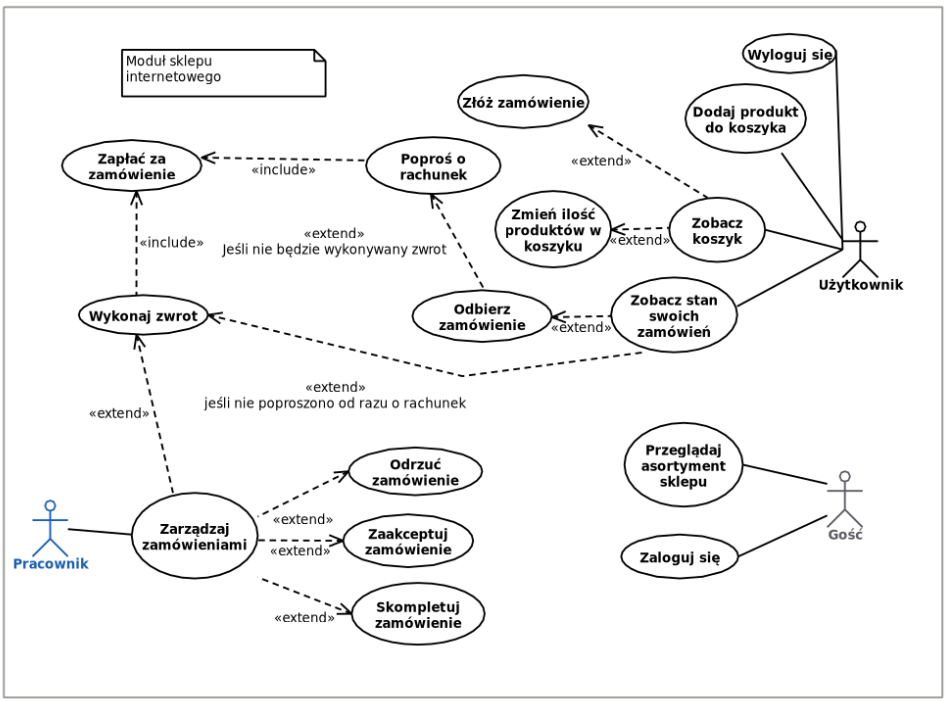
**PU „Najczęściej kupowane produkty"**

- Aktor: Kierownik
- Zdarzenie inicjujące: Wybór opcji najczęściej kupowanych produktów
- Warunki początkowe: Kierownik w oknie generowania rankingów. Kontynuacja PU opisanego w sekcji 4.5.4.
- Przebieg podstawowy realizacji PU:

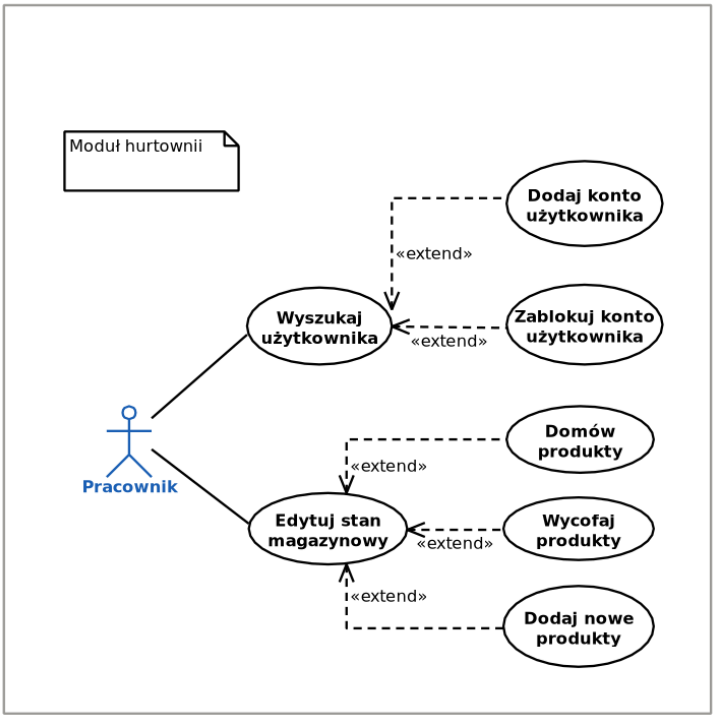
1. Zaznaczenie odpowiedniej opcji 4.24.
2. Wybór daty początkowej i końcowej.
3. Wybór ilości pozycji w rankingu.
4. Wyświetlenie rankingu.

- Warunki końcowe: Kierownik w oknie generowania rankingów, wygenerowany ranking.

## 4.6 Grafiki




Rysunek 4.25 Wcześniejsza wersja diagramu przypadków użycia dla modułu *Sklep internetowy*.




Rysunek 4.26 Wcześniejsza wersja diagramu przypadków użycia dla modułu *Hurtownia*.

# Rozdział 5

# Modele bazy danych

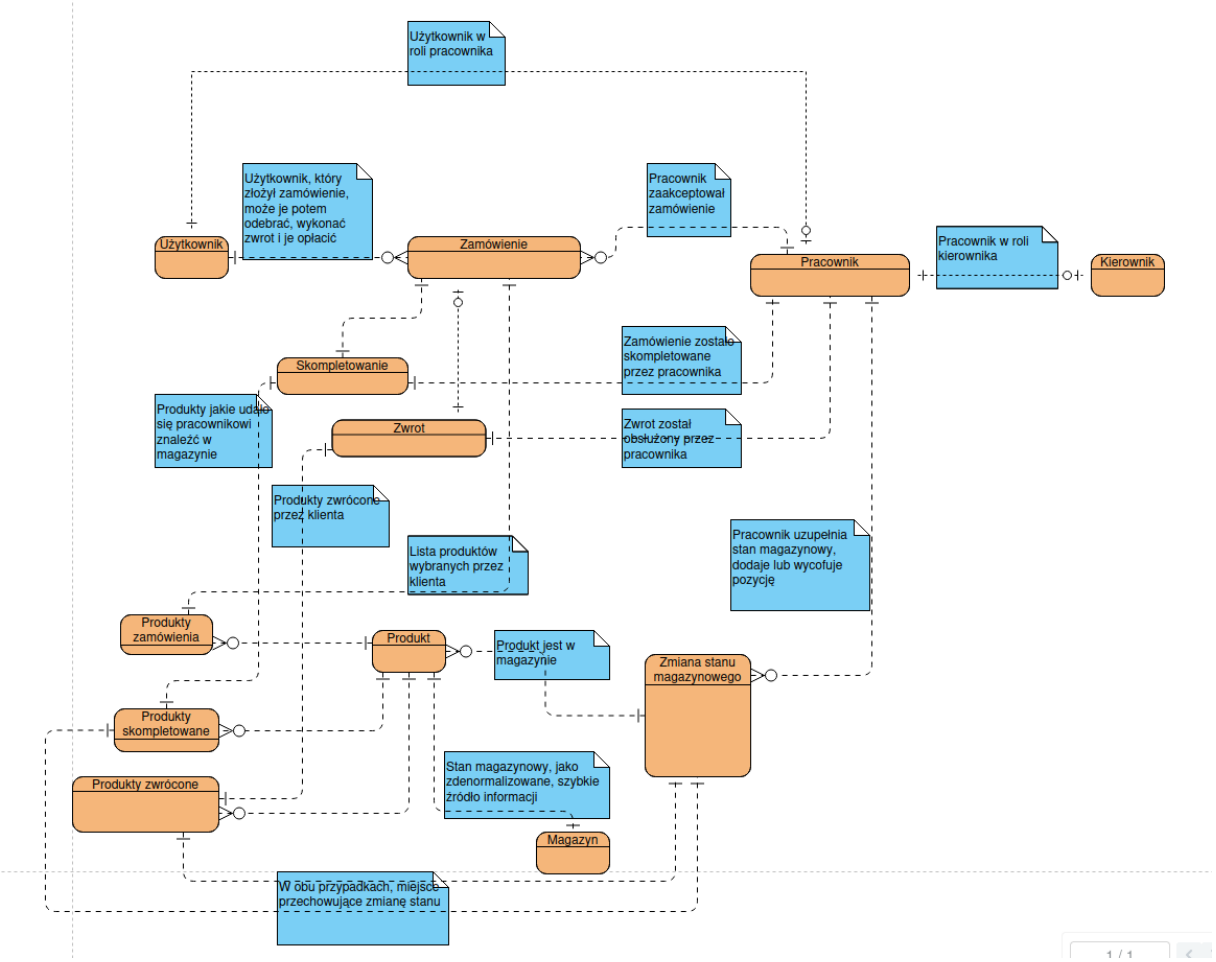
Normalizacja to proces organizowania danych w bazie w taki sposób, aby zminimalizować nadmiarowość oraz wyeliminować niepożądane zależności między atrybutami. Dzięki stosowaniu postaci normalnych możliwe jest stworzenie bazy danych, która jest spójna, przejrzysta i odporna na tzw. anomalie (np. aktualizacji, wstawiania, usuwania). Poniżej przedstawiono trzy podstawowe postacie normalne, które stosuje się w projektowaniu relacyjnych baz danych:

Tabela 5.1 Opis postaci normalnych w relacyjnej bazie danych

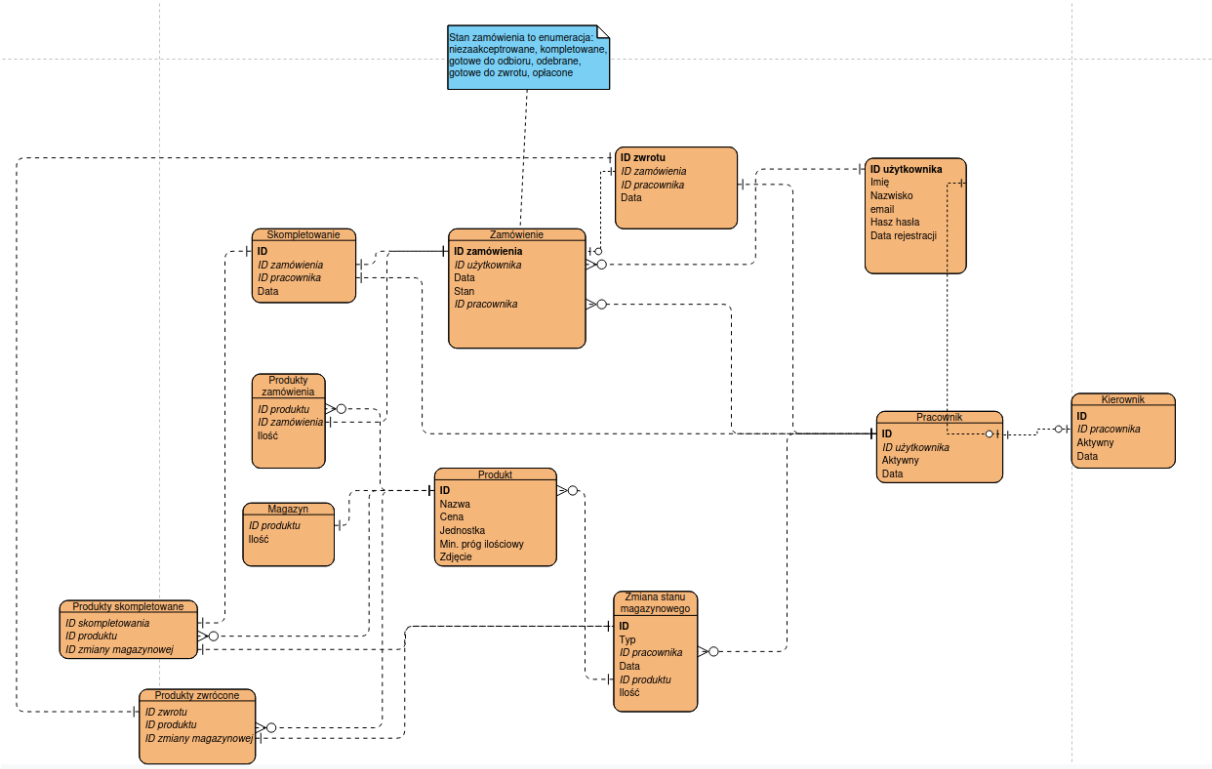
| Postać normalna | Warunki konieczne | Opis / Cel normalizacji |
|---|---|---|
| **1NF** | • Wszystkie atrybuty zawierają wartości atomowe (niepodzielne)<br>• Brak powtarzających się grup danych (np. kolumn listowych)<br>• Każdy rekord jednoznacznie identyfikowalny (klucz główny) | Zapewnia podstawową strukturę relacyjną – każdy atrybut reprezentuje dokładnie jedną wartość, co eliminuje zagnieżdżone lub powtarzalne dane w kolumnach. |
| **2NF** | • Spełnienie 1NF<br>• Brak częściowych zależności między atrybutami niekluczowymi a fragmentem klucza głównego (jeśli klucz złożony) | Redukuje redundancję poprzez eliminację zależności atrybutów od części klucza głównego. Zapewnia, że każdy atrybut niekluczowy zależy od całego klucza głównego. |
| **3NF** | • Spełnienie 2NF<br>• Brak zależności przechodnich: atrybut niekluczowy nie zależy od innego atrybutu niekluczowego | Eliminuje ukryte zależności między atrybutami niekluczowymi, zwiększając spójność i odporność bazy na anomalie aktualizacji. |

## 5.1 Wersja pierwsza

Po analizie modelu okazało się, że był błędny. Nie spełniał on zasad pierwszej postaci normalnej, niektóre encje miały nazwy będące w liczbie mnogiej. Samych encji było też za dużo – jednym z objawów jest trudność ich umiejscowienia na diagramie bez dużej ilości przecięć.




Rysunek 5.1 Model encji i relacji w wersji 1.



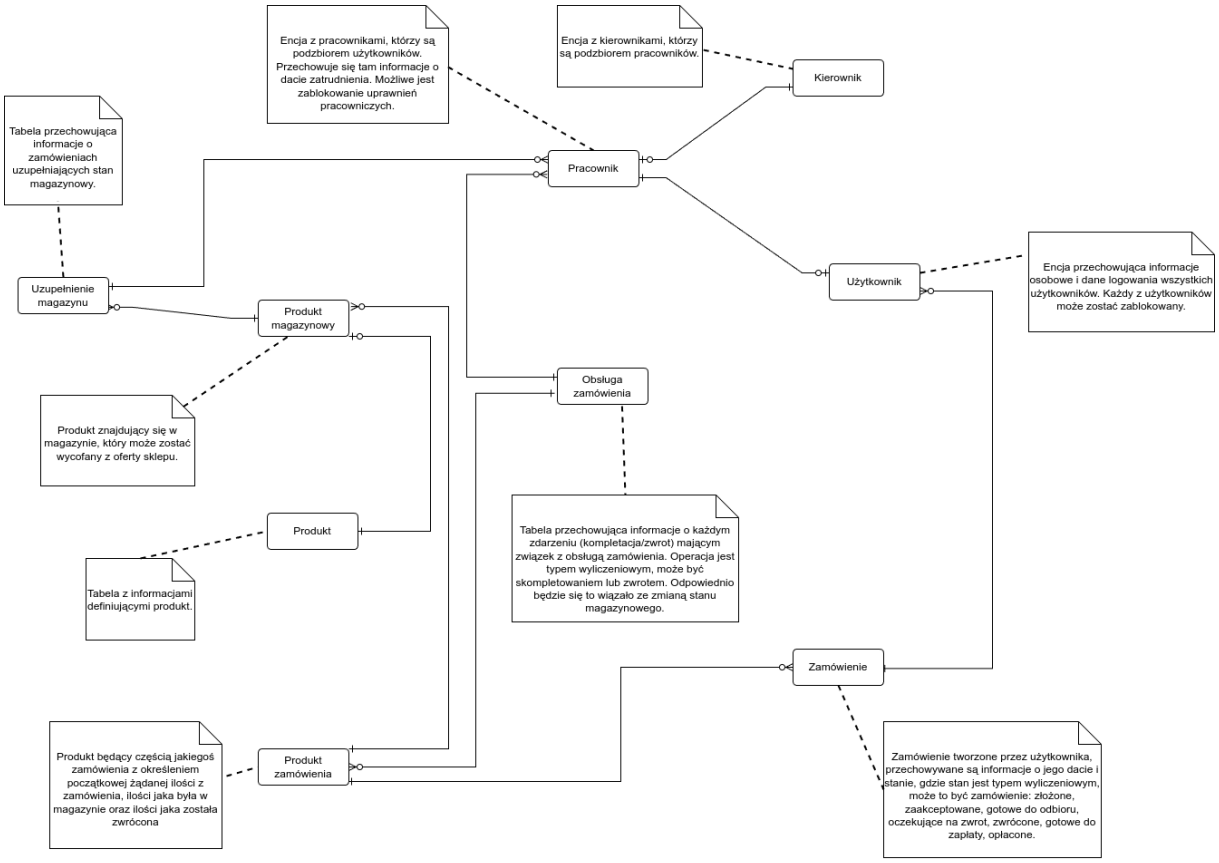

Rysunek 5.2 Model konceptualny w wersji 1.

Ze względu na brak spełnienia pierwszej postaci normalnej, model został wycofany z użycia.

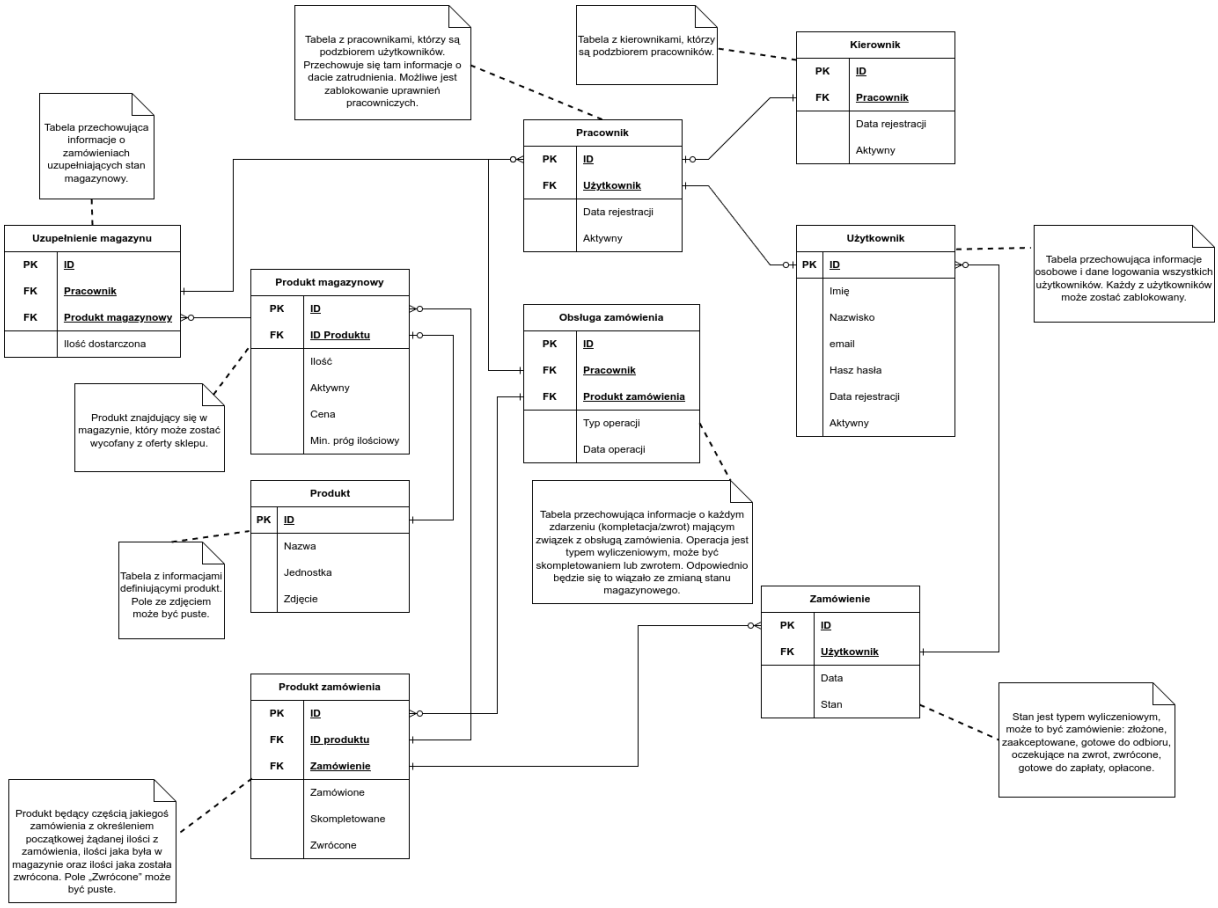
## 5.2 Wersja druga

Druga wersja modelu stanowi odpowiedź na problemy zidentyfikowane w poprzednim etapie. Najważniejszą zmianą było ograniczenie liczby encji do tych, które rzeczywiście odzwierciedlają kluczowe obszary funkcjonowania sklepu hydraulicznego. Przebudowano również struktury związane z procesem realizacji zamówień klientów oraz zarządzaniem stanem magazynowym. Wprowadzono podejście oparte na atomowości danych – każdy atrybut w encjach opisuje pojedynczą, niepodzielną cechę, co zapewnia zgodność z wymaganiami 1NF.

Dodatkowo, każda encja została jednoznacznie zidentyfikowana poprzez nadanie jej klucza głównego. Model ten stanowi istotne uproszczenie względem poprzedniej wersji, a zarazem wprowadza klarowną strukturę logiczną. Model spełnia wymagania pierwszej postaci normalnej.



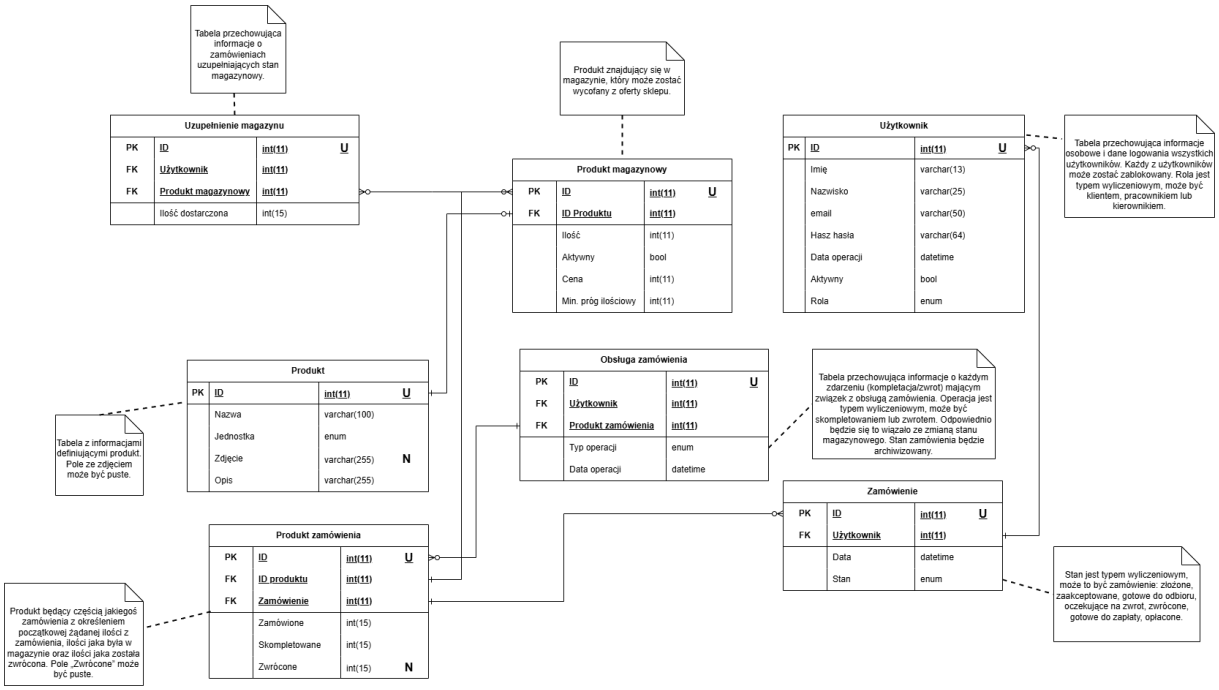

Rysunek 5.3 Model encji i relacji w wersji 2.




Rysunek 5.4 Model konceptualny w wersji 2.

## 5.3 Wersja trzecia

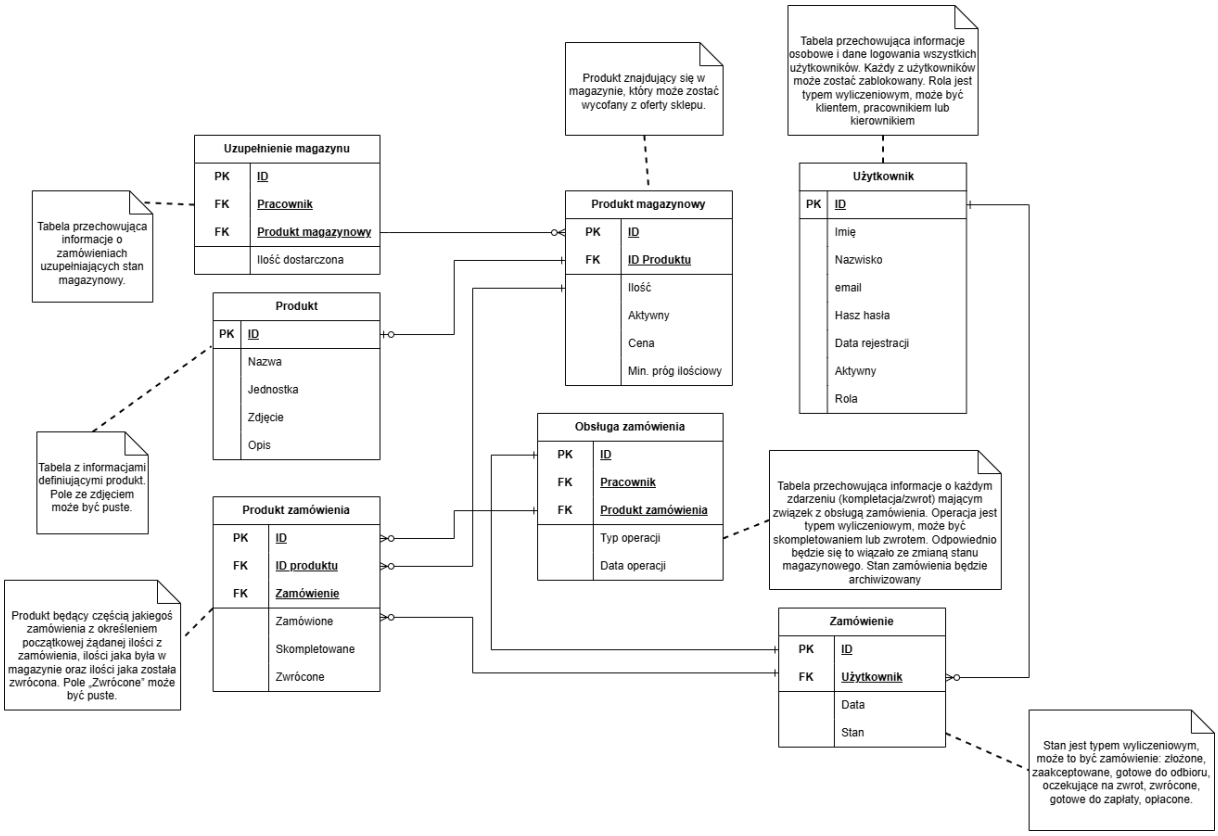
Trzecia wersja modelu to przejście do modelu fizycznego, którego celem było przygotowanie struktury pod kątem implementacji w środowisku bazodanowym. Jedną z kluczowych zmian było uproszczenie sposobu zarządzania użytkownikami systemu – zamiast odrębnych encji dla „Pracownika" i „Kierownika", zdecydowano się na jedną uniwersalną encję „Użytkownik", w której przypisywana jest konkretna rola. Takie podejście zwiększa elastyczność zarządzania uprawnieniami i ułatwia rozwój systemu w przyszłości. Mimo tego, model w tej wersji nie został jeszcze uznany za w pełni poprawny, głównie ze względu na niedoprecyzowanie relacji oraz brak pełnej zgodności z kolejnymi postaciami normalnymi. Wersja ta miała więc charakter przejściowy i stanowiła podstawę do dalszych udoskonaleń. Model spełnia wymagania pierwszej postaci normalnej.



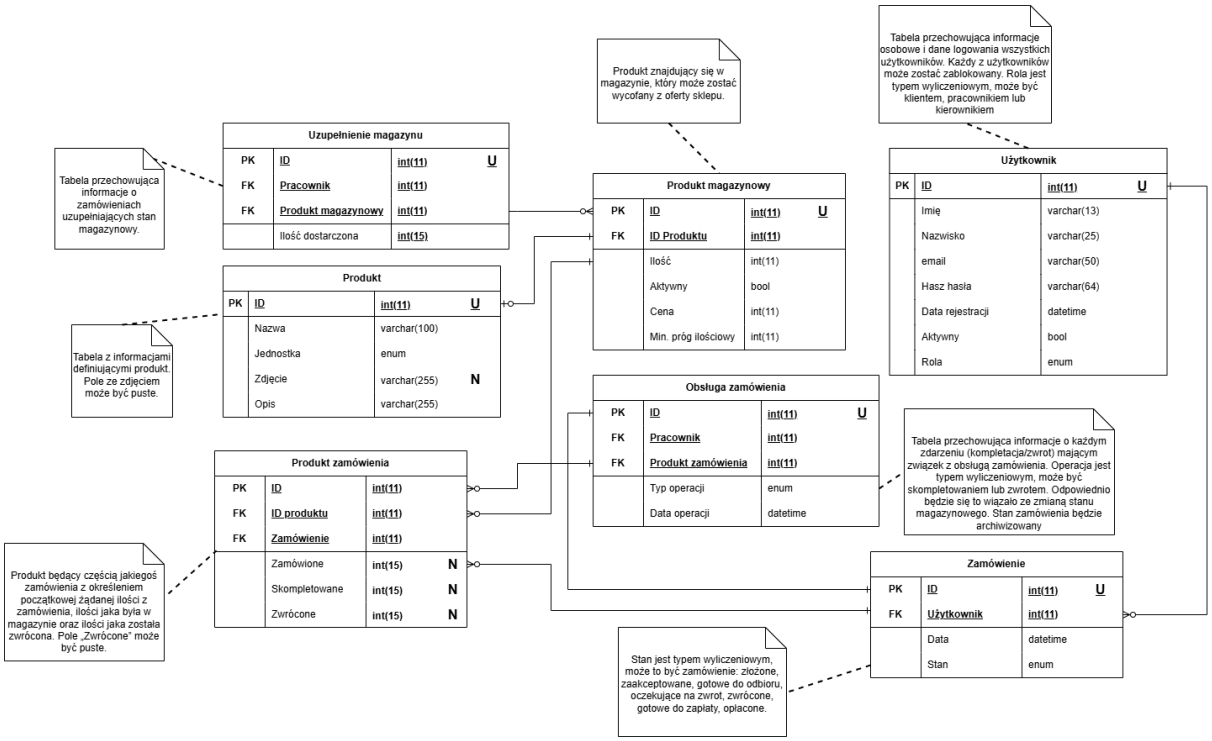

Rysunek 5.5 Model fizyczny.

## 5.4 Wersja czwarta

Czwarta wersja modelu stanowiła kontynuację prac nad udoskonaleniem struktury bazy danych. Zarówno model konceptualny, jak i fizyczny zostały uszczegółowione, a relacje między encjami zostały poprawione i dostosowane do wymagań wynikających z wcześniejszych etapów analizy. Widoczna jest znaczna poprawa spójności logicznej oraz strukturalnej modelu. Wersja ta skupia się na ujednoliceniu struktury danych i przygotowaniu do stworzenia wersji finalnej. Model spełnia wymagania drugiej postaci normalnej.




Rysunek 5.6 Model konceptualny.

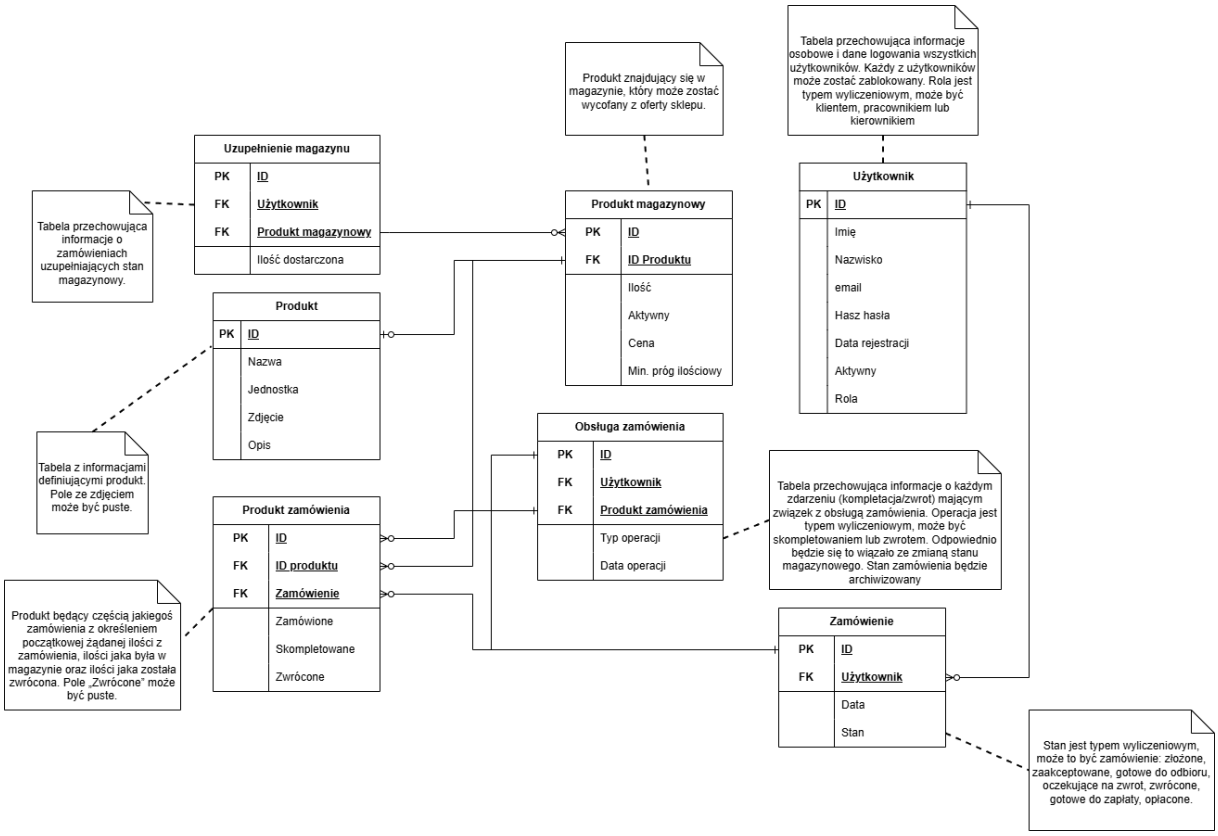



Rysunek 5.7 Model fizyczny.

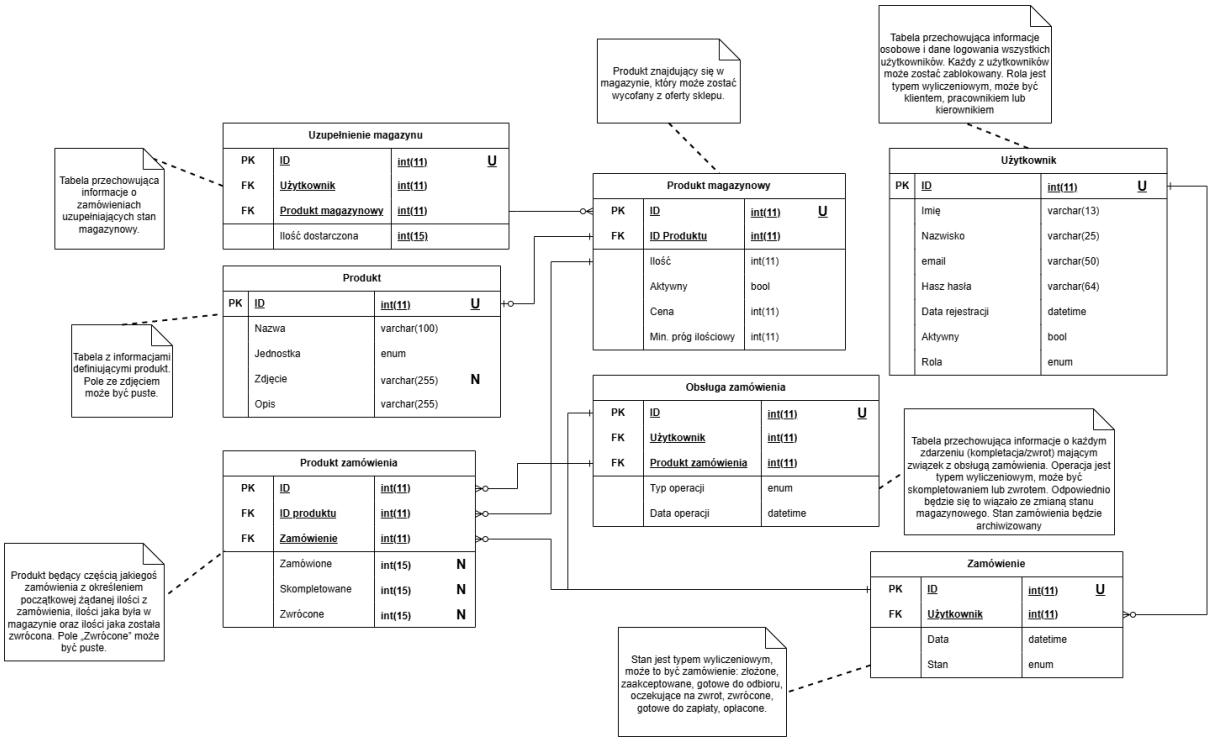
## 5.5 Wersja piąta

Względem wersji czwartej zostały zmienione nazwy pól w encji "Uzupełnienie magazynu oraz "Obsługa zamówienia" z "Pracownik" na "Użytkownik", co wprowadza spójność nomenklatury. Ponadto zostały zmienione niektóre relacje pól w celu zachowania spójności bazy danych.

Zaprezentowany w tej wersji model jest finalnym modelem bazy danych. Model spełnia wymagania trzeciej postaci normalnej.




Rysunek 5.8 Model konceptualny.

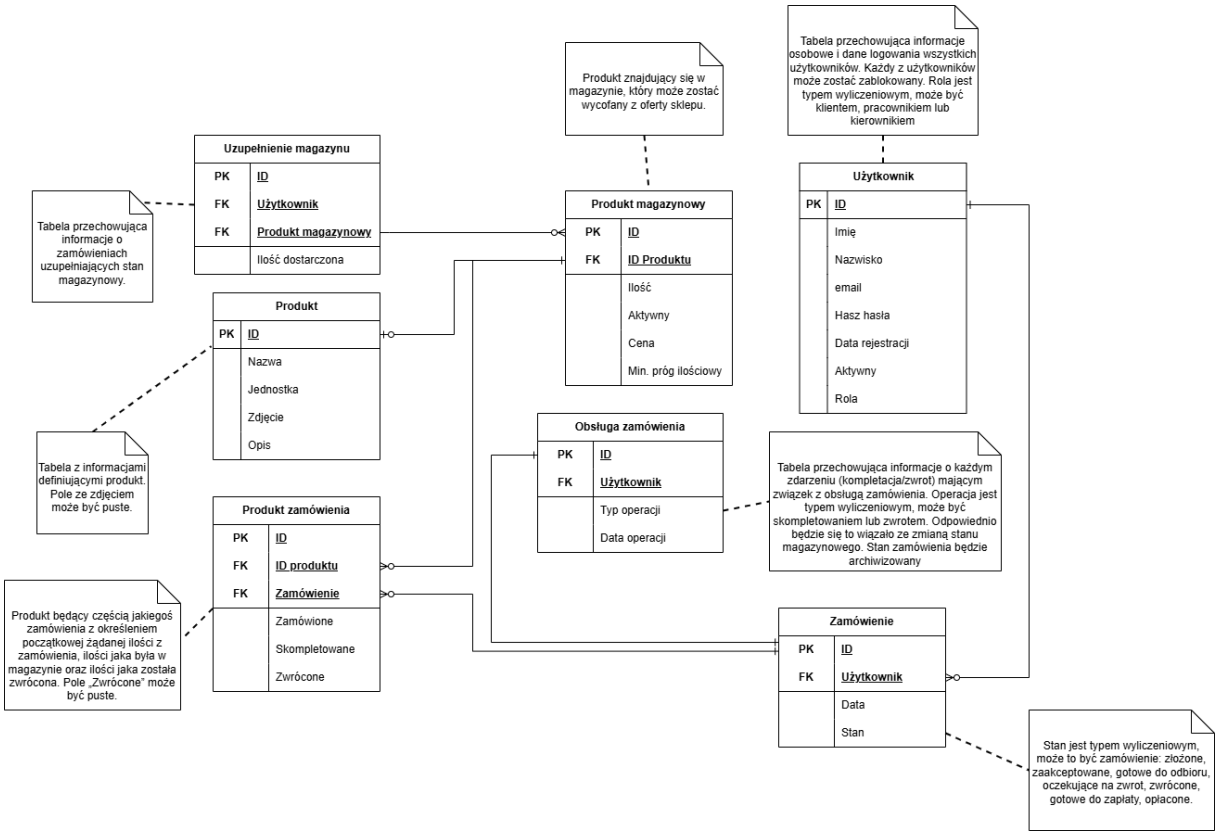



Rysunek 5.9 Model fizyczny.

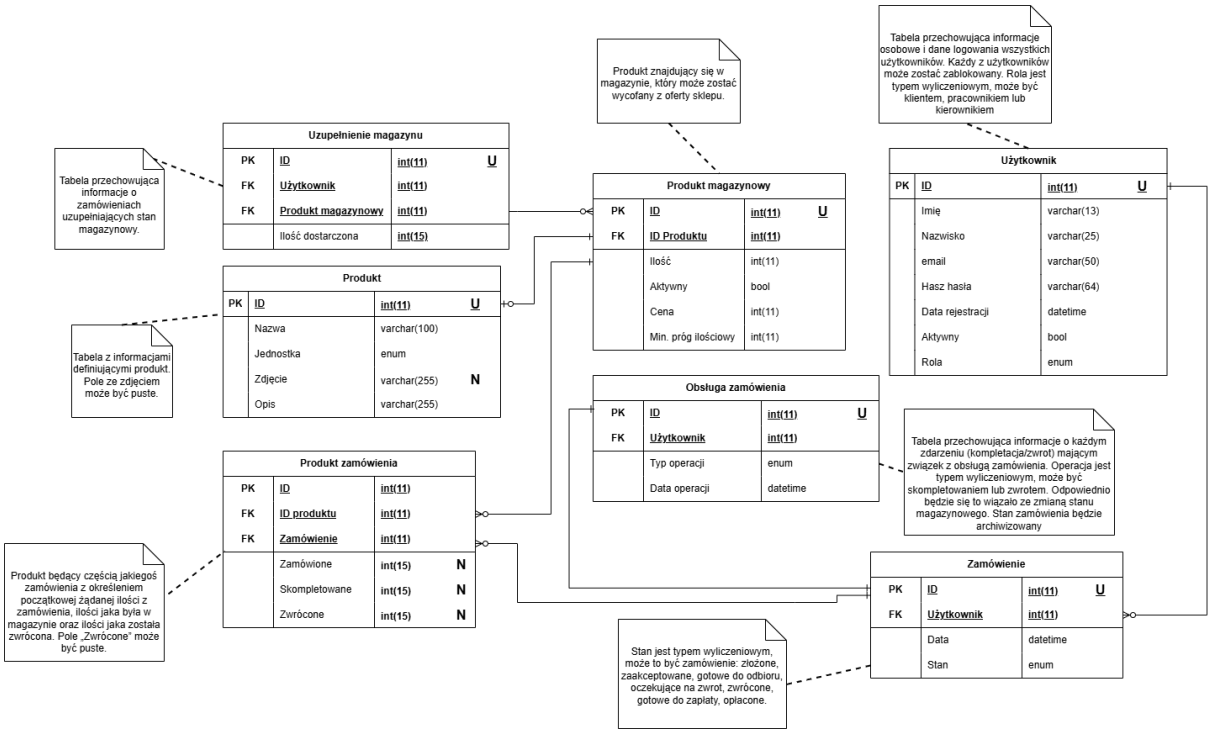
## 5.6 Wersja szósta

względem wersji piątej zostały wprowadzone nieznaczne poprawki. Usunięto relację między encjami "Produkt zamówienia" i "Obsługa zamówienia". Ponadto zmieniono położenie relacji między encjami "Obsługa zamówienia" i "Zamówienie" oraz "Produkt zamówienia" i "Zamówienie" w celu poprawy widoczności diagramu.

Zaprezentowany w tej wersji model jest finalnym modelem bazy danych. Model spełnia wymagania trzeciej postaci normalnej.



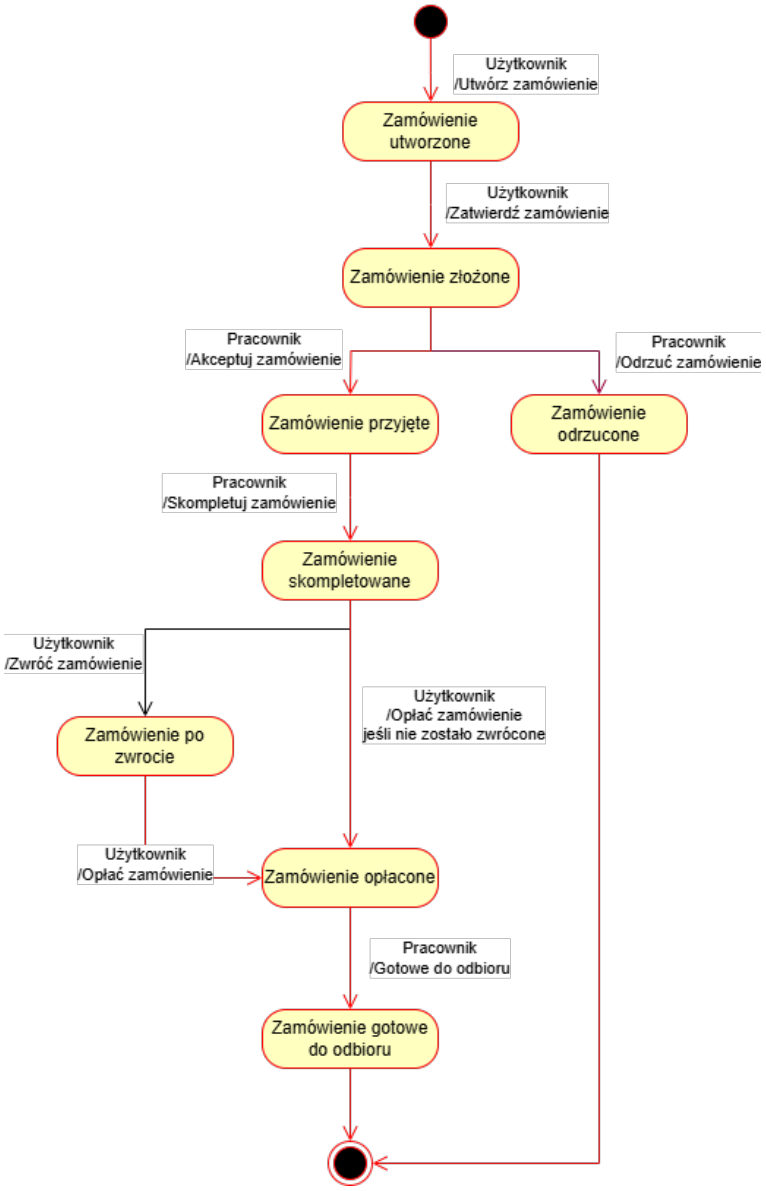

Rysunek 5.10 Model konceptualny.




Rysunek 5.11 Model fizyczny.

## 5.7 Graf stanów zamówienia

Diagram stanów obrazuje zmiany stanu obiektu zamówienia w systemie – czyli jak zamówienie przechodzi przez różne etapy, od momentu jego utworzenia aż do zakończenia (zrealizowania lub anulowania).




Rysunek 5.12 Graf stanów zamówienia (Machine State Diagram).

Stany i przejścia:

1. Początek — Stan początkowy. Oznacza, że obiekt Zamówienie zostaje utworzony.

2. Zamówienie złożone – użytkownik zatwierdza zamówienie akceptując wybrane produkty. Możliwe przejście do stanów "Zamówienie przyjęte" lub "Zamówienie odrzucone".

3. Zamówienie odrzucone – w przypadku odrzucenia zamówienia przez pracownika przechodzi ono w stan końcowy.

4. Zamówienie przyjęte – w przypadku akceptacji zamówienia przez pracownika może ono zostać skompletowane.

5. Zamówienie skompletowane – zamówienie kompletuje pracownik i oznacza to, że wszystkie, bądź część, produktów z zamówienia zostały skompletowane w ramach zamówienia. Możliwe przejście do stanów "Zamówienie po zwrocie" lub od razu do stanu "Zamówienie opłacone" w przypadku w którym zamówienie nie zostało zwrócone.

6. Zamówienie po zwrocie – w przypadku zwrócenia zamówienia przez użytkownika zwrócona zostaje część produktów zamówienia. Następnie użytkownik może je opłacić.

7. Zamówienie opłacone – jeśli użytkownik opłaci zamówienie, przechodzi ono w stan oczekiwania na gotowość do odbioru.

8. Zamówienie gotowe do odbioru – po opłaceniu zamówienia pracownik zmienia jego stan na gotowe do odbioru. Takie zamówienie może być odebrane przez użytkownika na terenie sklepu.

9. Koniec – zamówienie zostało odrzucone przez pracownika lub odebrane przez użytkownika.

# Rozdział 6

# Opis implementacji

## 6.1 Opis implementacji

System został zrealizowany jako aplikacja webowa oparta na frameworku **Spring Boot** z wykorzystaniem **Vaadin Flow** do budowy warstwy frontendowej. Zastosowana architektura opiera się na klasycznym podziale aplikacji na warstwy: prezentacji, logiki biznesowej i dostępu do danych.

### 6.1.1 Technologie

- Backend: **Spring Boot** (Java)
- Frontend: **Vaadin Flow**
- Baza danych: relacyjna My SQL
- ORM: JPA/Hibernate

### 6.1.2 Struktura systemu

Aplikacja została podzielona na następujące warstwy:

- **Repozytoria** — warstwa dostępu do danych, oparta o interfejsy JPA.
- **Serwisy** — warstwa logiki biznesowej, pośrednicząca między interfejsem użytkownika a bazą danych.
- **Warstwa prezentacji** — zrealizowana za pomocą Vaadin Flow, zapewnia interaktywne GUI dostępne z poziomu przeglądarki.

### 6.1.3 Moduły funkcjonalne

System realizuje następujące funkcjonalności:

- Rejestracja użytkowników
- Logowanie z uwierzytelnianiem i haszowaniem haseł

- Zarządzanie użytkownikami
- Tworzenie i zarządzanie zamówieniami
- Dodawanie, edycja i usuwanie produktów
- Zarządzanie stanami magazynowymi

### 6.1.4 Model danych

W systemie wyróżniono następujące encje:

- **Uzytkownik** — dane użytkownika (login, zahaszowane hasło)
- **Produkt** — podstawowe informacje o produkcie
- **ProduktMagazynowy** — stan magazynowy konkretnego produktu
- **Zamowienie** — dane dotyczące zamówienia złożonego przez użytkownika
- **ProduktZamowienia** — powiązanie produktów z zamówieniem
- **UzupelnienieMagazynu** — operacje uzupełniania stanu magazynowego
- **ObslugaZamowienia** — procesy realizacji zamówienia

### 6.1.5 Bezpieczeństwo

W aplikacji zastosowano następujące środki bezpieczeństwa:

- Hasła przechowywane są w postaci haszowanej, z wykorzystaniem algorytmu zgodnego z aktualnymi standardami.
- Dostęp do poszczególnych funkcji kontrolowany jest poprzez mechanizmy uwierzytelniania użytkownika.
- Zastosowano standardowe zabezpieczenia warstwy aplikacyjnej, minimalizujące ryzyko nieautoryzowanego dostępu.

# Rozdział 7

# Główne moduły i klasy

## 7.1 Główne moduły i klasy

Poniżej przedstawiono kluczowe moduły oraz klasy aplikacji, uporządkowane według struktury pakietów projektu.

### 7.1.1 Pakiet `com.kselomiel.app`

- **Application.java** — klasa główna uruchamiająca aplikację Spring Boot.

### 7.1.2 Pakiet `com.kselomiel.app.base`

- **domain.AbstractEntity** — abstrakcyjna klasa bazowa dla encji JPA, zawiera m.in. identyfikator.
- **ui.component.ViewToolbar** — komponent Vaadin używany w widokach do nawigacji i operacji.
- **ui.view.MainLayout** — główny layout aplikacji, definiujący układ interfejsu.
- **ui.view.MainView** — widok startowy aplikacji.
- **ui.view.MainErrorHandler** — globalny handler błędów.

### 7.1.3 Pakiet `com.kselomiel.app.model`

- **Product, Order, OrderProduct, MagazineProduct, OrderHandling, Unit, User, UserRole** — główne encje reprezentujące produkty, zamówienia, magazyn, użytkowników i uprawnienia.
- **enumerations.OrderState, OrderHandlingOperation** — enumeratory stanu zamówienia i operacji na zamówieniach.
- **converters.OrderStateConverter, OrderHandlingOperationConverter** — konwertery wartości enumeracji dla JPA.

### 7.1.4 Pakiet `com.kselomiel.app.repository`

- **ProductRepository, OrderRepository, OrderProductRepository, MagazineProductRepository, OrderHandlingRepository, UserRepository** — repozytoria JPA umożliwiające operacje CRUD na encjach.

### 7.1.5 Pakiet `com.kselomiel.app.security`

- **SecurityConfig** — konfiguracja zabezpieczeń aplikacji, definiuje m.in. ścieżki chronione.
- **UserDetailsServiceImpl, CustomUserDetails** — logika ładowania danych użytkownika oraz integracja z mechanizmem uwierzytelniania.
- **ui.view.LoginView, LogoutView, RegistrationView** — widoki obsługujące logowanie, rejestrację i wylogowanie.
- **ui.components.PasswordDialog** — komponent do zmiany hasła.

### 7.1.6 Pakiet `com.kselomiel.app.service`

- **ProductService, OrderService, UserService, MagazineProductService, ShoppingCartService** — warstwa logiki biznesowej obsługująca odpowiednio produkty, zamówienia, użytkowników, magazyn oraz koszyk zakupowy.
- **exception.InsufficientStockException, OrderStateException** — wyjątki związane z nieprawidłowym stanem magazynu i zamówienia.

### 7.1.7 Pakiet `com.kselomiel.app.sklep`

- **cart.ShoppingCart** — implementacja koszyka zakupowego.
- **Routes** — klasa definiująca ścieżki URL w aplikacji.

### 7.1.8 Widoki użytkownika (`com.kselomiel.app.sklep.ui.view`)

- **PublicAsortimentView** — publiczny widok asortymentu.
- **AsortimentView, ProductForm** — zarządzanie produktami (dla uprawnionych użytkowników).
- **CartView, AddToCartForm** — koszyk zakupowy.
- **MyAccountView, AccountView, PersonalDetailsDialog** — panel użytkownika i dane osobowe.
- **MyOrdersView, OrderForm** — przegląd i składanie zamówień.
- **ReturnView** — zwroty towarów.

- **employee.ManageOrdersView, ManageWarehouseView** — widoki pracownika: zarządzanie zamówieniami i magazynem.
- **manager.RankingView, StatisticsView** — panel menadżera: statystyki, ranking sprzedaży.

### 7.1.9 Formularze i dialogi

- **MagazineProductForm** — formularz zarządzania stanem magazynowym.
- **ManageOrderForm** — panel edycji zamówienia.
- **PersonalDetailsDialog** — dialog zmiany danych osobowych.

### 7.1.10 Testy

- **ArchitectureTest** — test zgodności architektury.
- **TestApplication** — klasa pomocnicza do testów.
- **TestcontainersConfiguration** — konfiguracja środowiska testowego z wykorzystaniem kontenerów.

### 7.1.11 Pozostałe pliki

- **Dockerfile, development-compose.yml, production-compose.yml** — pliki konfiguracyjne dla środowisk Docker.
- **db_backup/sklep_*.sql, sklep.sql** — kopie zapasowe bazy danych.

# Rozdział 8

# Opis wybranego fragmentu kodu źródłowego

Wybranym fragmentem kodu, który zostanie poddany głębszej analizie jest klasa usługi zamówień. Jest to tak naprawdę serce całego systemu informatycznego, gdyż obsługuje ona charakterystyczny dla tej aplikacji przepływ stanów zamówień. Na listingu 8.1 można zobaczyć, że ta klasa korzysta z mapowania relacyjno-obiektowego. Używa repozytoriów produktów magazynowych oraz zamówieniowych. Repozytoriów zamówień procesów obsługi zamówienia i użytkowników. Widać też adnotację `@Service`, która pozwala frameworkowi SprngBoot wykonać trochę niejawnego kodu.

```java
package com.kselomiel.app.service;

import com.kselomiel.app.model.Order;
import com.kselomiel.app.model.MagazineProduct;
import com.kselomiel.app.model.OrderProduct;
import com.kselomiel.app.model.User;
import com.kselomiel.app.model.OrderHandling;
import com.kselomiel.app.model.enumerations.OrderState;
import com.kselomiel.app.model.enumerations.OrderHandlingOperation;
import com.kselomiel.app.repository.*; // Zaimportuj wszystkie potrzebne
    repozytoria
import com.kselomiel.app.service.exception.InsufficientStockException;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.Optional;


@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final OrderProductRepository orderProductRepository;
    private final MagazineProductRepository magazineProductRepository;

    @SuppressWarnings("unused")
    private final UserRepository userRepository;
    private final OrderHandlingRepository orderHandlingRepository;
// ...
};
```

Wydruk 8.1 Zależności i zmienne klasy `OrderService`.

### 8.0.1 Konstruktor klasy

Jak widać na listingu 8.2, to ta klasa potrzebuje bardzo wielu parametrów. Jednak użytkownik jej nigdy nie tworzy. To SpringBoot dzięki adnotacjom automatycznie ją tworzy i podeje jej odpowiednie obiekty do konstruktora. Potem może tenże serwis przekazać innemu obiektowi.

```java
    /**
     * Serwis odpowiedzialny za obsługę operacji związanych z zamówieniami.
     *
     * @param orderRepository repozytorium do zarządzania encjami zamówień
     * @param orderProductRepository repozytorium do zarządzania produktami
     * zamówień
     * @param magazineProductRepository repozytorium do zarządzania produktami
     * magazynowymi
     * @param userRepository repozytorium do zarządzania użytkownikami
     * @param orderHandlingRepository repozytorium do obsługi realizacji zamówień
     */
    public OrderService(OrderRepository orderRepository,
            OrderProductRepository orderProductRepository,
            MagazineProductRepository magazineProductRepository,
            UserRepository userRepository,
            OrderHandlingRepository orderHandlingRepository) {
        this.orderRepository = orderRepository;
        this.orderProductRepository = orderProductRepository;
        this.magazineProductRepository = magazineProductRepository;
        this.userRepository = userRepository;
        this.orderHandlingRepository = orderHandlingRepository;
    }
```

Wydruk 8.2 Konstruktor klasy `OrderService`

### 8.0.2 Metoda tworzenia zamówień

Zamówienie ma w tabeli przydzielone swoje ID. W tabeli ProduktZamówienia tworzone jest wiele rekordów referujacych to samo zamówienie – one wszystkie są częścią tego zamówienia. Zamowienie potrzebuje jako argumentu użytkownika – zamawiającego, oraz mapy ID-int. Ta struktura jest koszykiem, ile jakiego produktu należy dodać.

```java
    /**
     * Tworzy nowe zamówienie dla podanego klienta na podstawie zawartości koszyka.
     * * @param customer Użytkownik składający zamówienie. Jeśli null, metoda
     * zwraca null.
     * @param shoppingCart Mapa produktów magazynowych oraz ich ilości zamawianych
     * przez klienta.
     * @return Utworzone zamówienie lub null, jeśli klient jest nieprawidłowy.
     * @throws InsufficientStockException Jeśli ilość produktu w magazynie jest
     * niewystarczająca.
     */
    @Transactional
    public Order createOrder(User customer, Map<MagazineProduct, Integer>
        shoppingCart) {

        if (customer == null) {
            return null;
        }

        LocalDateTime creationTime = LocalDateTime.now();
        Order order = new Order(customer, creationTime);

        Order savedOrder = orderRepository.save(order);

        for (Map.Entry<MagazineProduct, Integer> entry : shoppingCart.entrySet()) {
            MagazineProduct magazineProduct = entry.getKey();
            int quantity = entry.getValue();
            if (magazineProduct.getQuantity() < quantity) {
                throw new InsufficientStockException(
                        "Niewystarczająca ilość produktu: " + magazineProduct.
                            getProduct().getName());
            }

            OrderProduct orderProduct = new OrderProduct(
                    magazineProduct, savedOrder, quantity);

            orderProductRepository.save(orderProduct);
        }
        return savedOrder;
    }
```

Wydruk 8.3 Metoda tworzenia nowych zamówień.

## 8.1 Metody akceptacji i odrzucenia zamówienia

Dalsze metody służą głównie obsłudze procesu zamówienia, tj. przepływowi jego stanu. Metody akceptacji i odrzucenia są bardzo podobne, zasadniczo zmieniają w bazie jeden parametr i nie potrzebują rozległych transakcji do poprawnego wykonania.

```java
    /**
     * Akceptuje zamówienie, zmieniając jego stan na {@link OrderState#ACCEPTED}.
     * * <p>
     * Metoda wykonuje następujące operacje:
     * <ul>
     * <li>Zmienia stan zamówienia na {@code ACCEPTED}.</li>
     * <li>Tworzy i zapisuje wpis obsługi zamówienia typu {@code ACCEPTATION} przez
     * wskazanego użytkownika.</li>
     * <li>Zapisuje zaktualizowane zamówienie w repozytorium.</li>
     * </ul>
     * </p>
     *
     * @param order Zamówienie, które ma zostać zaakceptowane.
     * @param handler Użytkownik obsługujący operację akceptacji.
     * @return {@code true} jeśli stan zamówienia został zaktualizowany i zmiany
     * zostały zapisane; {@code false} w przeciwnym razie.
     */
    @Transactional
    public Boolean acceptOrder(Order order, User handler) { // Dodaj parametr
        handler
        if (order.updateState(OrderState.ACCEPTED)) {
            OrderHandling orderHandling = new OrderHandling(order, handler,
                OrderHandlingOperation.ACCEPTATION);
            orderHandlingRepository.save(orderHandling);
            orderRepository.save(order); // Zapisz zmieniony stan zamówienia
            return true;
        }
        return false;
    }

    /**
     * Odrzuca zamówienie i rejestruje operację odrzucenia.
     *
     * @param order Zamówienie, które ma zostać odrzucone.
     * @param handler Użytkownik obsługujący odrzucenie zamówienia.
     * @return true jeśli zamówienie zostało pomyślnie odrzucone, w przeciwnym
         razie
     * false.
     */
    @Transactional
    public Boolean rejectOrder(Order order, User handler) { // Dodaj parametr
        handler
        if (order.updateState(OrderState.REJECTED)) {
            OrderHandling orderHandling = new OrderHandling(order, handler,
                OrderHandlingOperation.REJECTION);
            orderHandlingRepository.save(orderHandling);
            orderRepository.save(order); // Zapisz zmieniony stan zamówienia
            return true;
        }
        return false;
    }
```

Wydruk 8.4 Akceptowanie i odrzucanie zamówienia.

### 8.1.1 Kompletowanie zamówienia

Ten proces jest już znacznie ciekawszy. Tu należy przekazać więcje informacji, nie tylko, które zamówienie przez kogo. Jeszcze trzeba przekazać co. Tu również używana jest mapa jak w koszyku, która przechowuje informację ile z danych produktów zostanie skompletowanych.

```
1
2   @Transactional
3   public Boolean markOrderAsReady(Order order, User handler, Map<Long, Integer>
        assembledQuantities) {
4       // assembledQuantities to mapa: klucz to id_produktu_zamowienia, wartość to
5       // skompletowana ilość
6       if (order.updateState(OrderState.READY)) {
7           OrderHandling orderHandling = new OrderHandling(order, handler,
                OrderHandlingOperation.ASSEMBLING);
8           orderHandlingRepository.save(orderHandling);
9           orderRepository.save(order);
10
11          for (Map.Entry<Long, Integer> entry : assembledQuantities.entrySet()) {
12              Long orderProductId = entry.getKey();
13              Integer assembledQuantity = entry.getValue();
14
15              OrderProduct orderProduct = orderProductRepository.findById(
                    orderProductId).orElse(null);
16              if (orderProduct != null) {
17                  orderProduct.setAssembled(assembledQuantity);
18                  orderProductRepository.save(orderProduct);
19
20                  // Opcjonalnie: Aktualizacja stanu magazynowego (ważne!)
21                  MagazineProduct magazineProduct = orderProduct.getMagazineProduct
                        ();
22                  magazineProduct.setQuantity(magazineProduct.getQuantity() -
                        assembledQuantity); // Zmniejszamy o // skompletowaną
23                                          // ilość
24                  magazineProductRepository.save(magazineProduct);
25
26              } else {
27                  // Log the error if the order product is not found
28                  System.out.println("OrderProduct with ID " + orderProductId + "
                        not found during assembling.");
29              }
30          }
31          return true;
32      }
33      return false;
34  }
```

Wydruk 8.5 Kompletowanie zamówienia.

### 8.1.2 Odbiór zamówienia, oznaczanie gotowości opłaty oraz opłata zamówienia

Te metody są bardzo podobne do tych od akceptacji i odrzucania zamówienia. Są bardzo proste, potrzeba jedynie powiedzieć, które zamówienie ma być w ten sposób potraktowane i przez kogo.

```
1
2     /**
3      * Oznacza zamówienie jako odebrane przez klienta, zmieniając jego stan na
4      * {@link OrderState#FETCHED}.
5      * * <p>
6      * Metoda wykonuje następujące operacje:
7      * <ul>
8      * <li>Zmienia stan zamówienia na {@code FETCHED} (odebrane).</li>
9      * <li>Tworzy i zapisuje wpis obsługi zamówienia typu {@code FETCHING} przez
10     * wskazanego użytkownika.</li>
11     * <li>Zapisuje zaktualizowane zamówienie w repozytorium.</li>
12     * </ul>
13     * </p>
14     *
15     * @param order Zamówienie, które ma zostać oznaczone jako odebrane.
16     * @param handler Użytkownik obsługujący operację odbioru.
17     * @return {@code true} jeśli stan zamówienia został zaktualizowany i zmiany
18     * zostały zapisane; {@code false} w przeciwnym razie.
19     */
20    @Transactional
21    public Boolean markOrderAsFetched(Order order, User handler) {
22        if (order.updateState(OrderState.FETCHED)) {
23            OrderHandling orderHandling = new OrderHandling(order, handler,
                  OrderHandlingOperation.FETCHING);
24            orderHandlingRepository.save(orderHandling);
25            orderRepository.save(order);
26            return true;
27        }
28        return false;
29    }
```

Wydruk 8.6 Odbiór zamówienia

```java
    /**
     * Oznacza zamówienie jako wymagające płatności, zmieniając jego stan na
     * {@link OrderState#TO_PAY}.
     * * @param order Zamówienie, które ma zostać oznaczone do opłacenia.
     * @param handler Użytkownik wykonujący operację.
     * @return true jeśli stan zamówienia został pomyślnie zmieniony i zapisany, w
     * przeciwnym razie false.
     */
    @Transactional
    public Boolean markOrderToPay(Order order, User handler) {
        if (order.updateState(OrderState.TO_PAY)) {

            orderRepository.save(order);
            return true;
        }
        return false;
    }
```

Wydruk 8.7 Oznaczanie gotowości opłaty

```java
    /**
     *
     * Oznacza podane zamówienie jako opłacone i rejestruje operację obsługi
     * płatności.
     *
     * Jeśli stan zamówienia zostanie pomyślnie zaktualizowany na
     * {@link OrderState#PAID},
     * metoda tworzy nowy wpis {@link OrderHandling} z podanym użytkownikiem
     * obsługującym i typem operacji
     * {@link OrderHandlingOperation#PAYMENT}, zapisuje go w repozytorium
     * oraz zapisuje zaktualizowane zamówienie.
     *
     * @param order zamówienie do oznaczenia jako opłacone
     * @param handler użytkownik obsługujący operację płatności
     * @return {@code true} jeśli stan zamówienia został zaktualizowany i zmiany
     * zostały zapisane; {@code false} w przeciwnym razie
     */
    @Transactional
    public Boolean markOrderAsPaid(Order order, User handler) {
        if (order.updateState(OrderState.PAID)) {
            OrderHandling orderHandling = new OrderHandling(order, handler,
                OrderHandlingOperation.PAYMENT);
            orderHandlingRepository.save(orderHandling);
            orderRepository.save(order);
            return true;
        }
        return false;
    }
```

Wydruk 8.8 Opłata zamówienia

## 8.2 Zwrot zamówienia

Metoda analogiczna do tej kompletującej. Parametry ma te same. Zwracając oddaje produkty do magazynu.

```java

    @Transactional
    public Boolean markOrderAsReturned(Order order, User handler, Map<Long, Integer
        > returnedQuantities) {
        // returnedQuantities to mapa: klucz to id_produktu_zamowienia, wartość to
        // zwrócona ilość
        // Ta metoda powinna być wywoływana, gdy zamówienie jest zwracane (część
            ciowo lub
        // w całości)
        if (order.updateState(OrderState.RETURNED)) { // Upewnij się, że ORDERED
            lub PAID przechodzi w RETURNED w
                                                // Order.updateState
            OrderHandling orderHandling = new OrderHandling(order, handler,
                OrderHandlingOperation.RETURNING);
            orderHandlingRepository.save(orderHandling);
            orderRepository.save(order);

            for (Map.Entry<Long, Integer> entry : returnedQuantities.entrySet()) {
                Long orderProductId = entry.getKey();
                Integer returnedQuantity = entry.getValue();

                OrderProduct orderProduct =
                        orderProductRepository.findById(orderProductId).orElse(null)
                            ;
                if (orderProduct != null) {
                    orderProduct.setReturned(returnedQuantity);
                    orderProductRepository.save(orderProduct);

                    // Opcjonalnie: Aktualizacja stanu magazynowego (ważne!)
                    MagazineProduct magazineProduct = orderProduct.getMagazineProduct
                        ();
                    magazineProduct.setQuantity(magazineProduct.getQuantity()
                    + returnedQuantity); // Zwiększamy o
                                        // zwróconą ilość
                    magazineProductRepository.save(magazineProduct);

                } else {
                    // Obsługa błędu: Produkt zamówienia nie istnieje
                    // Można rzucić wyjątek lub zalogować
                }
            }
            return true;
        }
        return false;
    }
```

Wydruk 8.9 Zwrot produktów.

## 8.3 Metody pomocnicze

Metody powstałe z wygody. W paru miejscach potrzeba było tej usługi zamówień. Ona sama w sobie posiada repozytoria zamówień, produktów, itp. Dlatego, jeżeli już jakiś obiekt ma referencję na tą usługę, to może ją poprosić o dostęp do tychże repozytoriów.

```
    /**
     * Zwraca zamówienie po jego ID, zapewniając dostęp do najnowszych danych.
     * Jest to operacja odczytu, która nie wpływa na logikę biznesową serwisu.
     * * @param id ID zamówienia.
     * @return Optional zawierający zamówienie, jeśli znaleziono.
     */
    public Optional<Order> findOrderById(Long id) {
        return orderRepository.findById(id);
    }

    public MagazineProductRepository getMagazineProductRepository() {
        return magazineProductRepository;
    }
```

Wydruk 8.10 Metody pomocnicze

# Rozdział 9

# Podręcznik dla użytkownika

## 9.1 Uruchomienie aplikacji

Aby uruchomić aplikację **Sklep Hydrauliczny**, należy przygotować środowisko zgodnie z poniższą instrukcją.

#### Tryb deweloperski

1. Importuj projekt do środowiska IDE (np. IntelliJ IDEA, Eclipse).
2. Uruchom klasę `Application` z pakietu `com.kselomiel.app`.
3. Alternatywnie, można uruchomić aplikację z terminala poleceniem:

   ```
   ./mvnw
   ```

W trybie deweloperskim aplikacja działa bez konieczności budowania pełnej paczki produkcyjnej.

#### Środowisko kontenerowe (opcjonalne)

Projekt zawiera konfigurację `.devcontainer`, umożliwiającą uruchomienie środowiska deweloperskiego w kontenerze Dockera z siecią hosta:

1. Otwórz projekt w Visual Studio Code z zainstalowaną wtyczką *Dev Containers*.
2. Skorzystaj z opcji **Reopen in Container**.
3. W kontenerze można kompilować aplikację do pliku `.jar`, który można uruchamiać także poza kontenerem.

**Uwaga:** Kontener deweloperski nie zawiera narzędzi Dockera. Budowanie obrazu Dockera należy wykonać z systemu hosta.

### 9.1.1 Budowanie aplikacji (Tryb produkcyjny)

Aby przygotować aplikację do środowiska produkcyjnego:

1. Zbuduj paczkę `.jar` poleceniem:

   ```
   ./mvnw -Pproduction package
   ```

2. Alternatywnie, w kontenerze deweloperskim:

   ```
   ./build_package.sh
   ```

### 9.1.2 Budowanie obrazu Docker

Po utworzeniu paczki `.jar` można zbudować obraz Dockera, wykonując w katalogu projektu:

```
./build_image.sh
```

Operacja ta musi zostać wykonana na systemie hosta.

### 9.1.3 Uruchamianie bazy danych

Projekt zawiera dwa pliki `docker-compose`:

- `development-compose.yml` — środowisko deweloperskie (baza danych i PHPMyAdmin),
- `production-compose.yml` — środowisko produkcyjne (baza danych oraz aplikacja).

Uruchomienie:

```
# Środowisko produkcyjne
docker compose -f production-compose.yml up

# Środowisko deweloperskie
docker compose -f development-compose.yml up
```

W trybie deweloperskim aplikację należy uruchomić ręcznie (`./mvnw`).

### 9.1.4 Dostęp do aplikacji

- Aplikacja dostępna jest pod adresem:

  `http://localhost:8001`

- Panel *PHPMyAdmin* do zarządzania bazą danych:

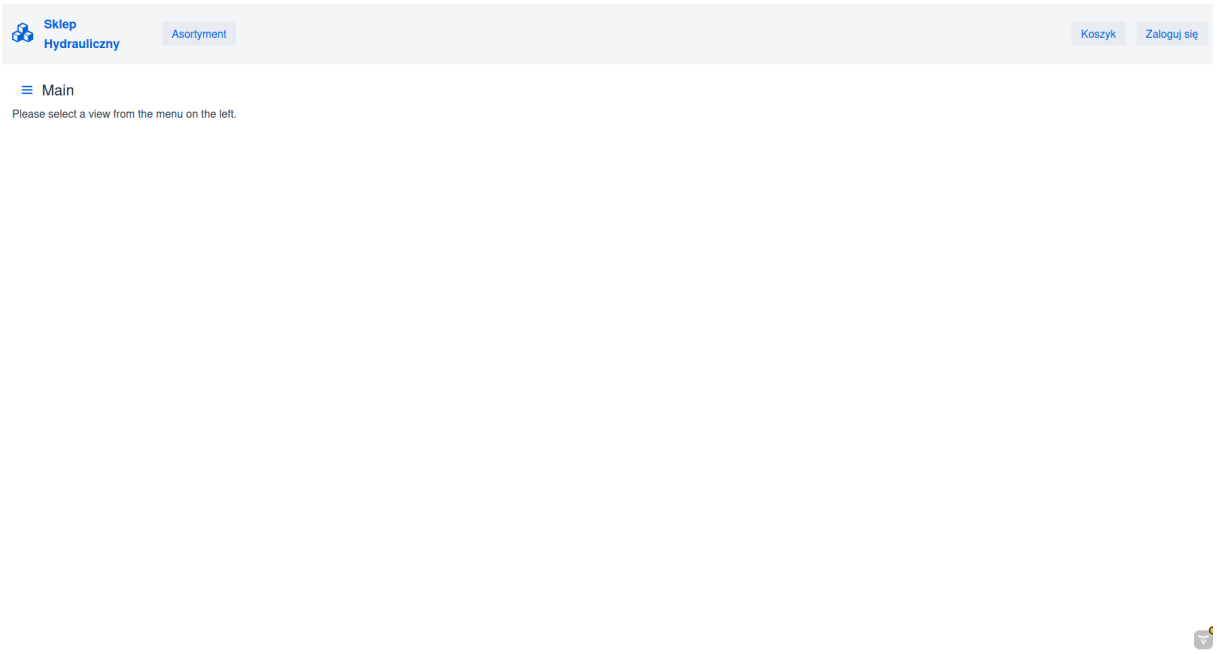
  `http://localhost:8081`

## 9.2 Przewodnik po rolach użytkowników

System sklepu hydraulicznego przewiduje kilka poziomów dostępu w zależności od roli użytkownika. W niniejszym rozdziale przedstawiono widoki oraz funkcjonalności przypisane poszczególnym rolom.

### 9.2.1 Gość

#### Strona główna (widok publiczny)



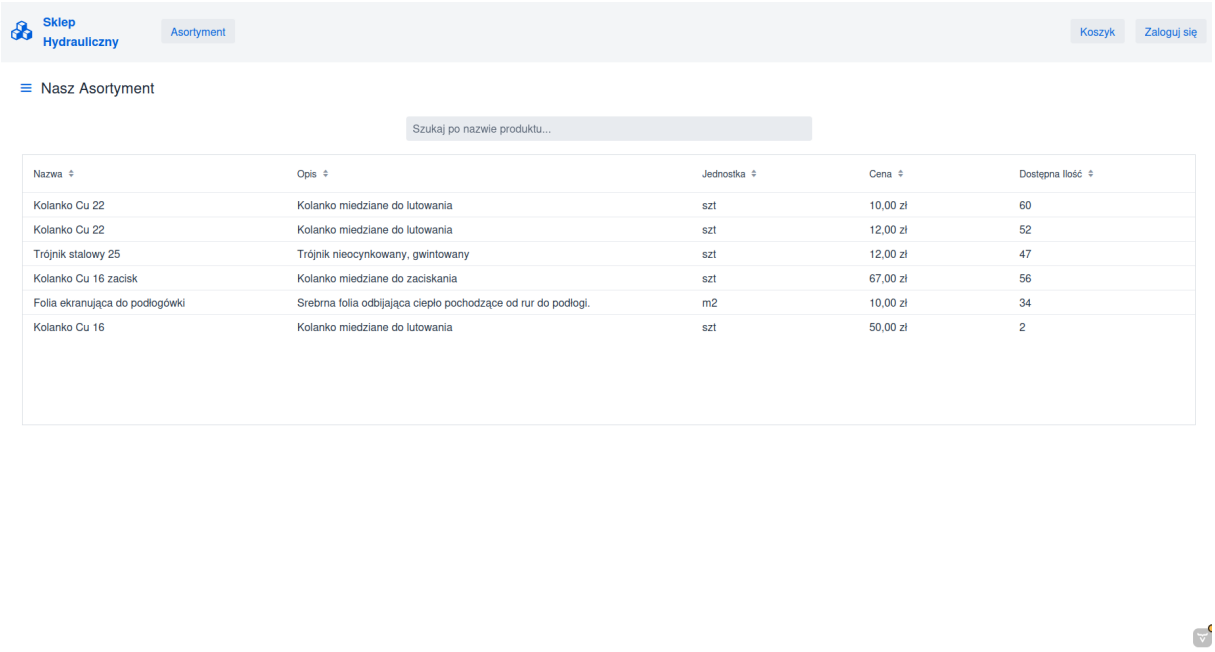

Rysunek 9.1 Widok główny dostępny dla niezalogowanego użytkownika (gościa).

Po wejściu na stronę aplikacji użytkownik niezalogowany widzi podstawowy widok systemu. Do dyspozycji posiada:

- **Asortyment** — możliwość przeglądania dostępnych produktów bez konieczności logowania.
- **Zaloguj się** — przekierowanie do formularza logowania.

W nagłówku strony umieszczono logo systemu oraz odnośnik do strony głównej.

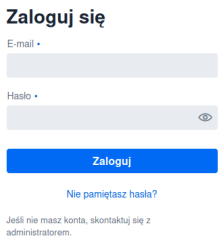
## Asortyment




Rysunek 9.2 Widok publicznego asortymentu produktów dostępnego dla gościa.

Gość ma możliwość przeglądania listy produktów wraz z podstawowymi informacjami o każdym z nich, bez opcji zakupu.

### Logowanie

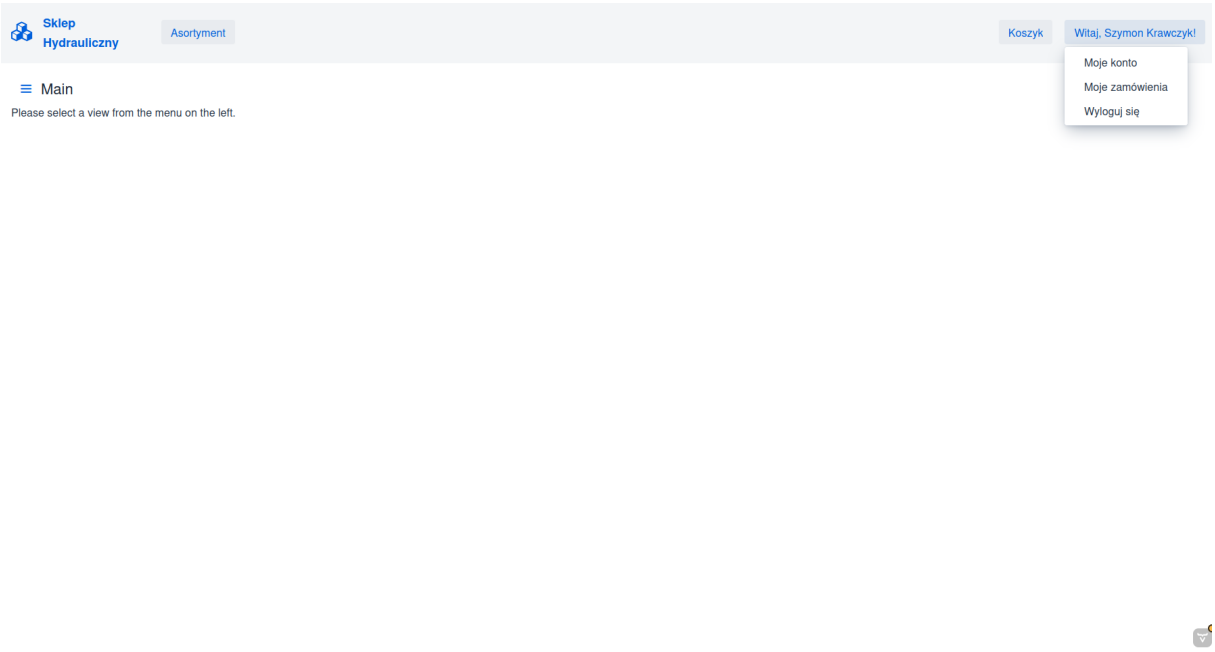



Rysunek 9.3 Widok formularza logowania do systemu.

Logowanie umożliwia przejście do widoków przypisanych odpowiedniej roli po uwierzytelnieniu.

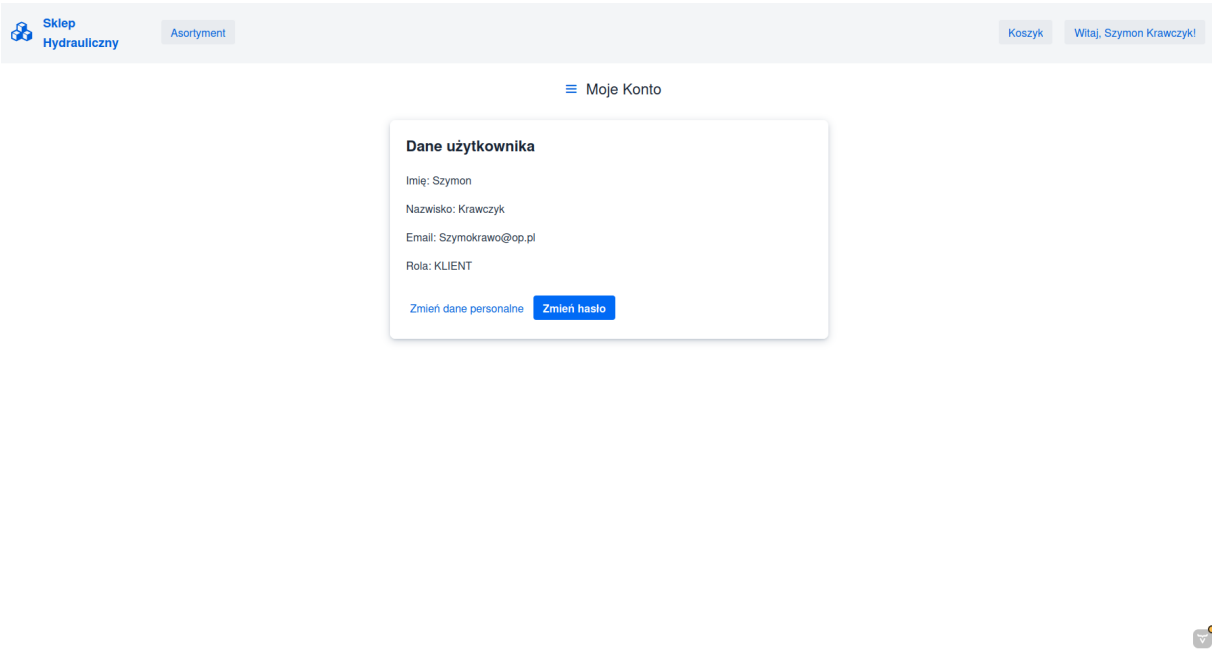
## 9.2.2 Klient

Klient po zalogowaniu posiada dostęp do wszystkich funkcjonalności gościa oraz dodatkowo:




Rysunek 9.4 Widok strony domowej klienta z rozwiniętym menu.

## Dane konta

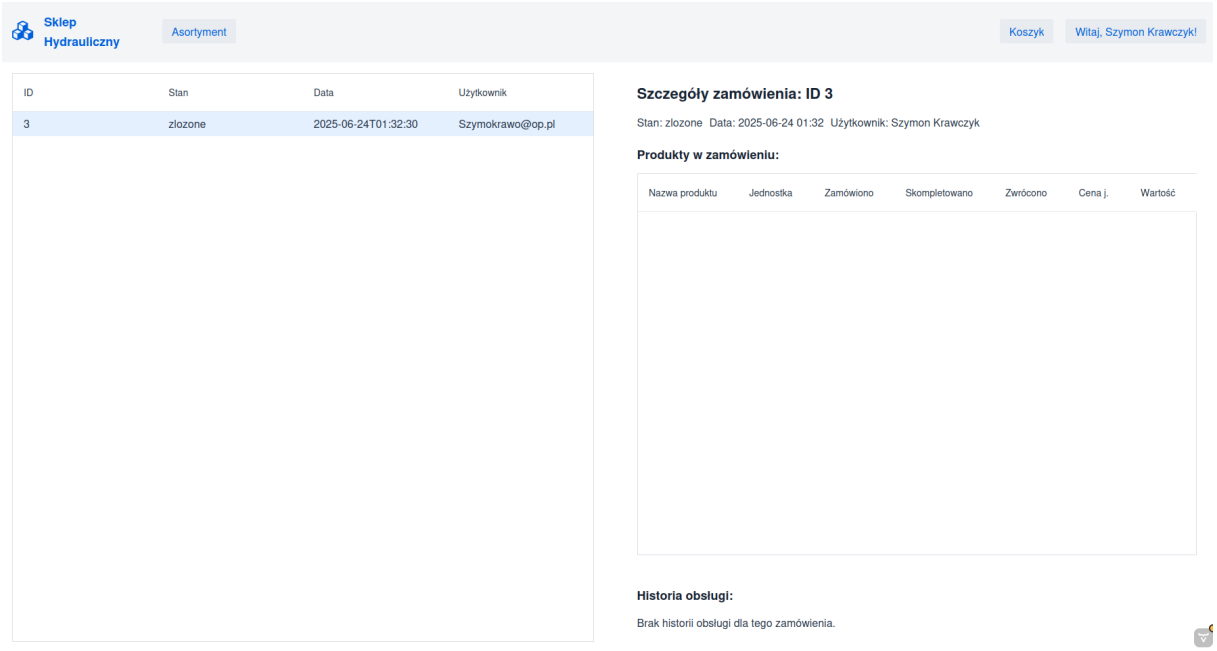



Rysunek 9.5 Widok danych konta klienta.

Każdy zalogowany użytkownik, niezależnie od roli, posiada możliwość podglądu i edycji własnych danych osobowych (np. imię, nazwisko, adres, hasło).

*Widok danych konta jest identyczny dla każdej roli, z tą różnicą, że zawartość zależy od konkretnego użytkownika.*

### Moje zamówienia

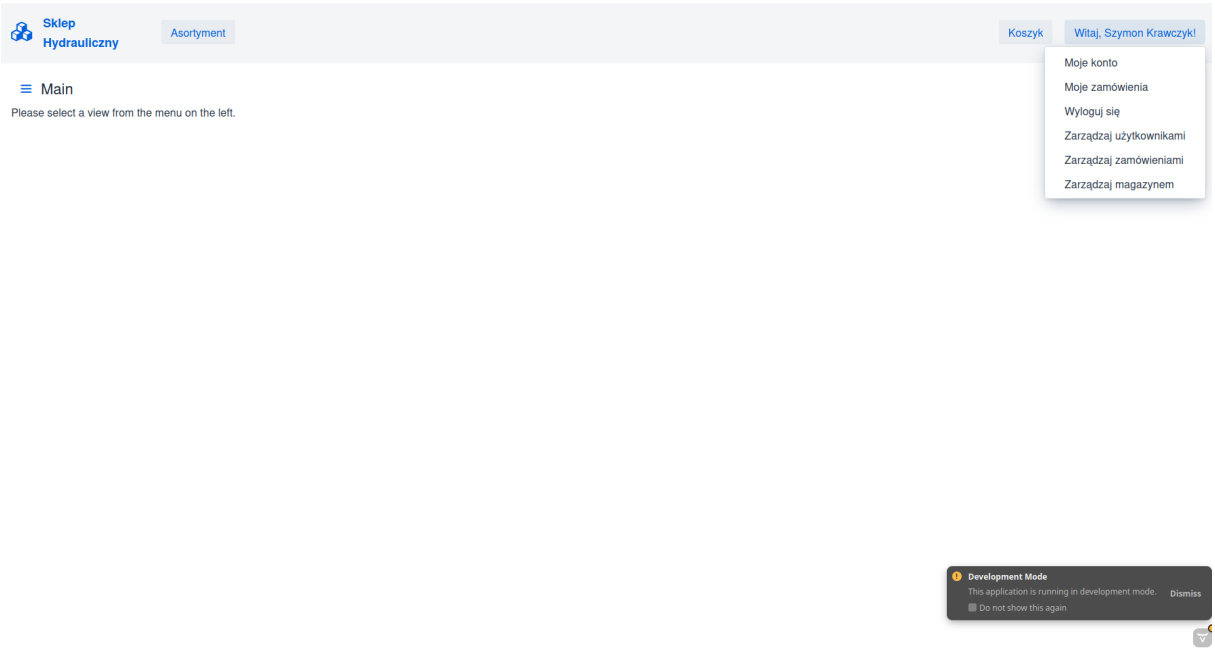



Rysunek 9.6 Widok listy zamówień klienta.

Klient może przeglądać historię własnych zamówień, sprawdzać ich status oraz szczegóły.

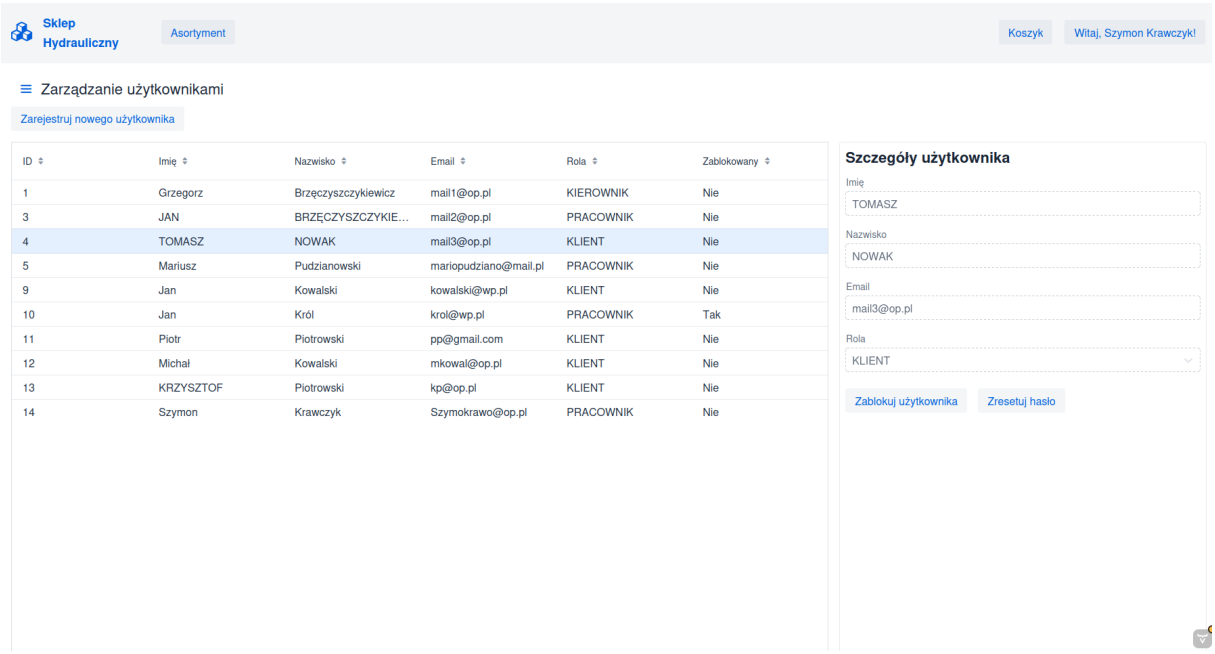
## 9.2.3 Pracownik

Pracownik posiada wszystkie uprawnienia klienta oraz dodatkowe moduły administracyjne:




Rysunek 9.7 Strona główna pracownika z rozbudowanym menu.

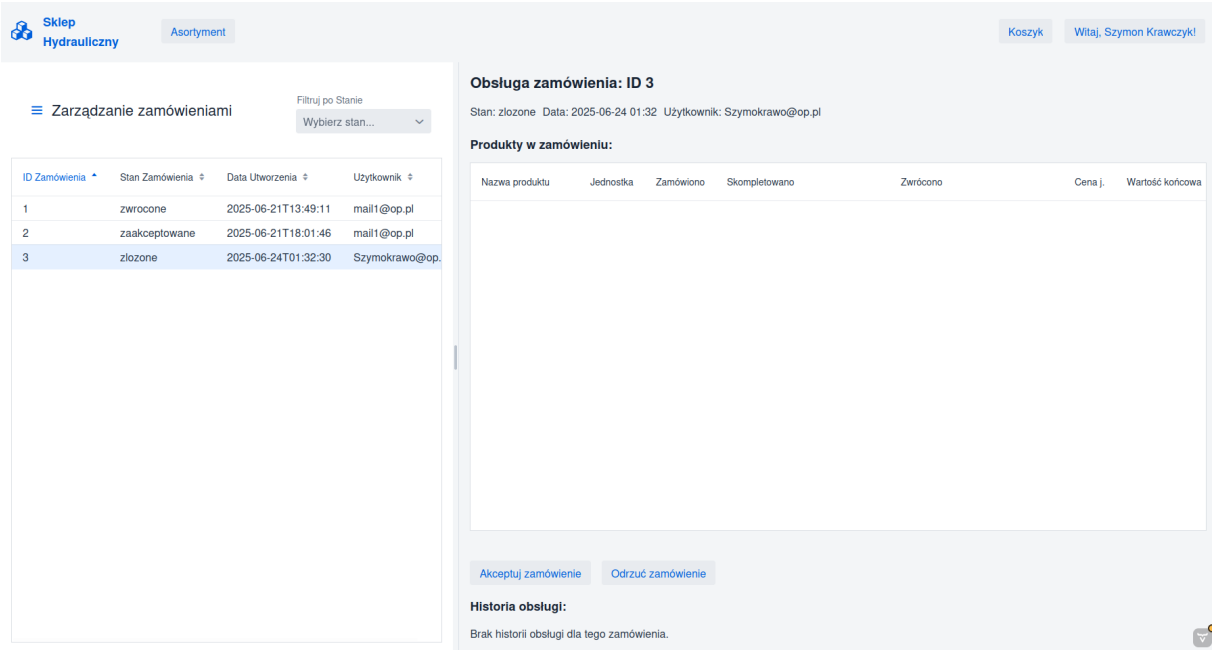
## Zarządzanie użytkownikami




Rysunek 9.8 Widok panelu zarządzania użytkownikami przez pracownika.

Pracownik ma możliwość przeglądania listy użytkowników, zmiany hasła użytkownika oraz blokowania kont.

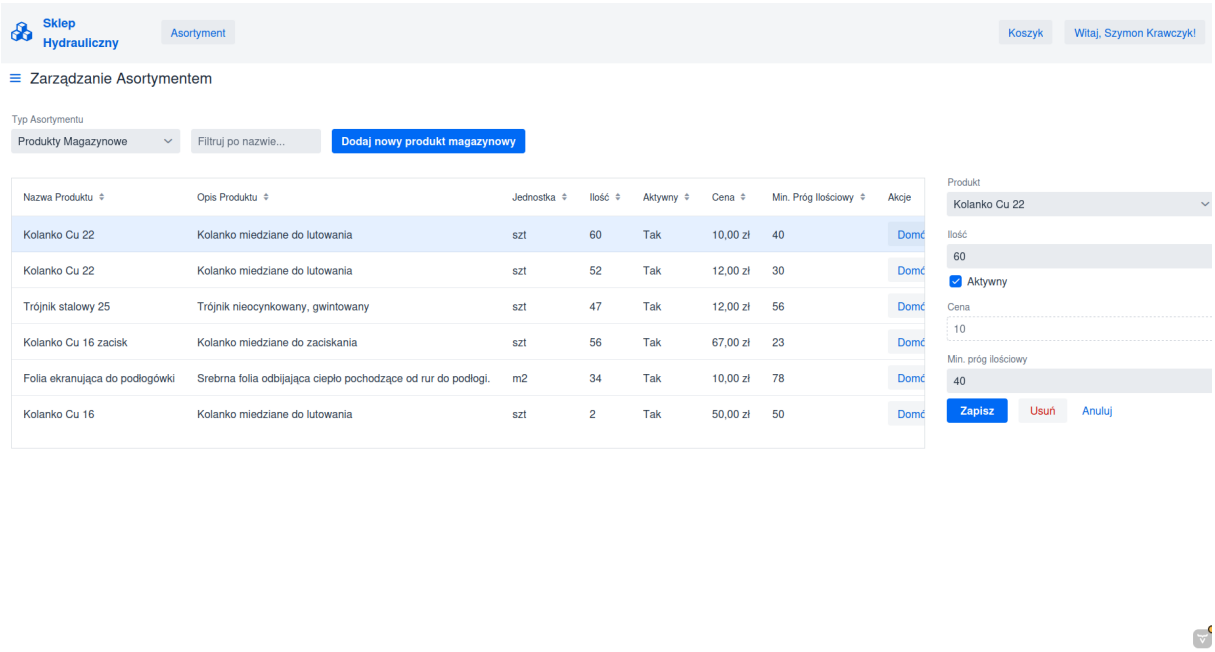
## Zarządzanie zamówieniami



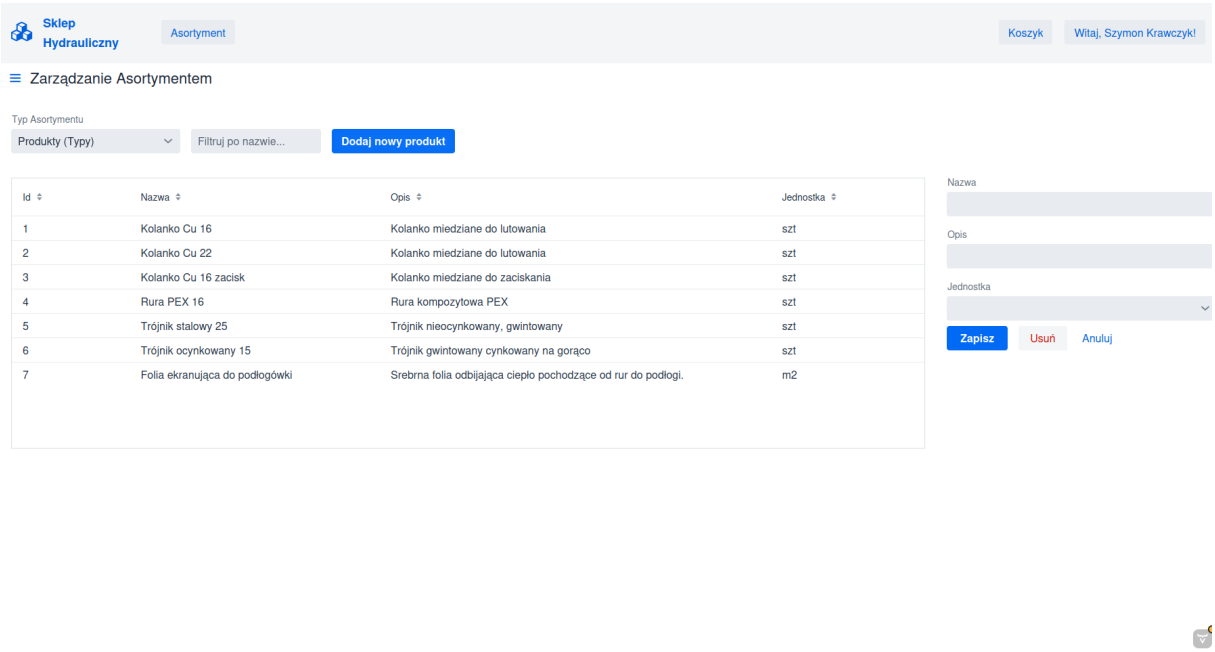

Rysunek 9.9 Widok zarządzania zamówieniami.

Pracownik obsługuje zamówienia klientów — zmienia ich status, zatwierdza lub odrzuca.

## Zarządzanie magazynem i produktami




Rysunek 9.10 Widok zarządzania stanem magazynowym.

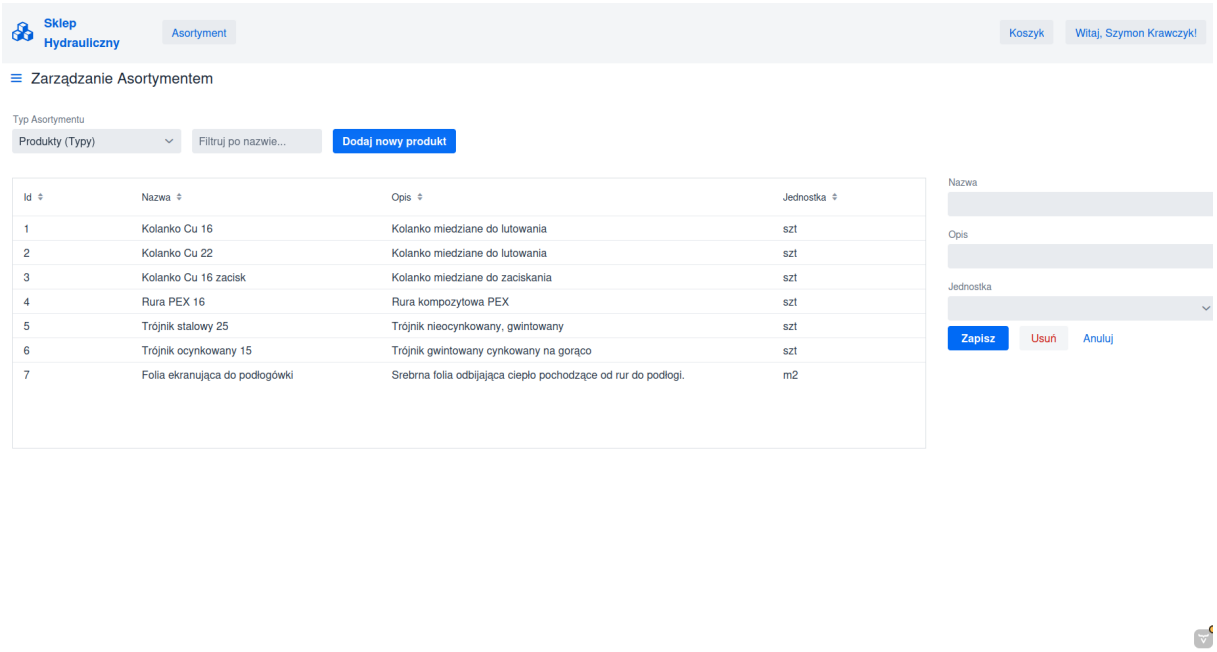



Rysunek 9.11 Widok formularza dodawania nowych produktów.

Pracownik może dodawać nowe produkty do oferty, aktualizować ich dane oraz zarządzać stanami magazynowymi.

### 9.2.4 Kierownik

Kierownik posiada pełne uprawnienia pracownika oraz dodatkowe funkcje zarządcze:




Rysunek 9.12 Widok panelu zarządzania użytkownikami kierownika (analogiczny do pracownika).

W przypadku kierownika zarządzanie użytkownikami obejmuje również możliwość nadawania lub odbierania uprawnień pracownikom oraz innym kierownikom.

# Rozdział 10

# Wnioski

1. W obecnej wersji aplikacji nie zaimplementowano jeszcze możliwości tworzenia statystyk dotyczących sprzedaży, najczęściej zamawianych produktów czy zestawień klientów z największymi zamówieniami. Wdrożenie tych funkcji ułatwiłoby analizę danych i podejmowanie decyzji biznesowych.

2. Wersja responsywna, umożliwiająca wygodne korzystanie z aplikacji na smartfonach i tabletach, nie została jeszcze zaimplementowana. Dostosowanie interfejsu do urządzeń mobilnych jest istotne, zwłaszcza dla pracowników hurtowni w terenie.

3. Aktualna wersja aplikacji wymaga ręcznego nadzorowania niektórych procesów, takich jak uzupełnianie stanów magazynowych po przekroczeniu progów minimalnych. Automatyzacja takich zadań w przyszłości przyczyni się do ograniczenia ryzyka błędów ludzkich i zwiększy efektywność działania systemu.

4. Framework Vaadin udostępnia wiele gotowych rozwiązań, które mogłyby przyspieszyć rozwój systemu oraz poprawić jego wygląd i funkcjonalność, zwłaszcza w kontekście dostosowania do urządzeń mobilnych. W dalszych etapach projektu warto rozważyć szersze wykorzystanie bibliotek Vaadina, co umożliwi szybkie wdrożenie responsywnych widoków i zwiększy dostępność aplikacji.

5. Podczas projektowania modelu danych nie udało się od razu przewidzieć wszystkich zależności i scenariuszy biznesowych. W trakcie implementacji okazało się, że niektóre relacje i pola wymagały korekt, co pokazuje, jak ważne jest iteracyjne podejście do projektowania bazy danych, szczególnie w bardziej złożonych systemach.

# Spis rysunków


3.1 Aktorzy w systemie zarządzania sklepem hydraulicznym. . . . . 14
3.2 Diagram przypadków użycia dla modułu *Administracja.* . . . . . 15
3.3 Diagram przypadków użycia dla modułu *Hurtownia.* . . . . . 16
3.4 Diagram przypadków użycia dla modułu *Sklep internetowy.* . . . . . 17

4.1 Aktorzy. . . . . 19
4.2 Diagram przypadków użycia dla modułu *Administracja.* . . . . . 20
4.3 Diagram przypadków użycia dla modułu *Hurtownia.* . . . . . 21
4.4 Diagram przypadków użycia dla modułu *Sklep internetowy.* . . . . . 21
4.5 Strona tytułowa dla gościa. . . . . 22
4.6 Ekran logowania . . . . . 22
4.7 Przegląd asortymentu przez gościa . . . . . 24
4.8 Ekran z informacjami o koncie . . . . . 24
4.9 Ekran z informacjami o zamówieniach użytkownika . . . . . 25
4.10 Ekran do wykonywania zwrotu towarów. . . . . 25
4.11 Uniwersalne rozwijane menu. . . . . 26
4.12 Strona tytułowa dla użytkownika. . . . . 27
4.13 Ekran asortymentu dla użytkownika. . . . . 27
4.14 Ekran koszyka . . . . . 28
4.15 Ekran wyszukiwania użytkownika . . . . . 33
4.16 Ekran do obsługi skompletowań. . . . . 34
4.17 Ekran do dodawania nowych użytkowników. . . . . 34
4.18 Ekran zarządzania zamówieniami . . . . . 36
4.19 Ekran edycji stanu magazynowego . . . . . 39
4.20 Formularz danych produktu. . . . . 40
4.21 Formularz danych produktu magazynowego. . . . . 41
4.22 Formularz domawiania produktu. . . . . 41
4.23 Okno generowania statystyk. . . . . 45
4.24 Ono generowania rankingów . . . . . 47
4.25 Wcześniejsza wersja diagramu przypadków użycia dla modułu *Sklep internetowy.* . . . . . 49
4.26 Wcześniejsza wersja diagramu przypadków użycia dla modułu *Hurtownia.* . . . . . 50

5.1 Model encji i relacji w wersji 1. . . . . 53
5.2 Model konceptualny w wersji 1. . . . . 54
5.3 Model encji i relacji w wersji 2. . . . . 55

5.4 Model konceptualny w wersji 2. . . . . 56
5.5 Model fizyczny. . . . . 57
5.6 Model konceptualny. . . . . 58
5.7 Model fizyczny. . . . . 59
5.8 Model konceptualny. . . . . 60
5.9 Model fizyczny. . . . . 61
5.10 Model konceptualny. . . . . 62
5.11 Model fizyczny. . . . . 63
5.12 Graf stanów zamówienia (Machine State Diagram). . . . . 64

9.1 Widok główny dostępny dla niezalogowanego użytkownika (gościa). . 87
9.2 Widok publicznego asortymentu produktów dostępnego dla gościa. . 88
9.3 Widok formularza logowania do systemu. . . . . 89
9.4 Widok strony domowej klienta z rozwiniętym menu. . . . . 90
9.5 Widok danych konta klienta. . . . . 90
9.6 Widok listy zamówień klienta. . . . . 91
9.7 Strona główna pracownika z rozbudowanym menu. . . . . 92
9.8 Widok panelu zarządzania użytkownikami przez pracownika. . . . . 92
9.9 Widok zarządzania zamówieniami. . . . . 93
9.10 Widok zarządzania stanem magazynowym. . . . . 94
9.11 Widok formularza dodawania nowych produktów. . . . . 94
9.12 Widok panelu zarządzania użytkownikami kierownika (analogiczny do pracownika). . . . . 95